



中国研究生创新实践系列大赛
“华为杯”第二十届中国研究生
数学建模竞赛

学 校 国防科技大学

参赛队号 23910020096

1.龙洗

队员姓名 2.陈思雅

3.乔琛远

中国研究生创新实践系列大赛

“华为杯”第二十届中国研究生

数学建模竞赛

题 目： 基于因果推理的强对流降水临近预报问题研究

摘 要：

我国地域辽阔，自然条件复杂，灾害性天气种类繁多，地区差异性大。雷雨大风、冰雹、短时强降水等强对流天气是造成经济损失、危害生命安全最严重的一类灾害性天气。由于强对流降水具有高强度、小空间尺度等特点，一直是气象预报领域的一大挑战。本文从时序数据普遍存在的因果关系入手，研究了基于因果推理的强对流降水临近预报问题。

针对问题一：为解决有效应用双偏振变量改进强对流预报问题，从“时序因果性”与“空间关联性”两个维度出发，提出了基于结构因果模型的 Z_H 临近预报方法。首先，统计不同区域的总降水量，采用最大互信息与皮尔逊相关系数对不同区域之间的关联进行量化，并引入“区域半径”的概念确定区域的空间关联范围。然后，为了确定关联区域变量 Z_H 、 Z_{DR} 、 K_{DP} 之间的因果关系，提出了基于多元传递熵的因果贝叶斯网络构建方法。最后，为了定量表示 Z_H 、 Z_{DR} 、 K_{DP} 之间的因果关系，建立可提取用于强对流临近预报双偏振雷达资料中微物理特征信息的数学模型，根据多元传递熵的计算过程，推导了其结构因果模型，对 Z_H 进行定量预报。该模型以确定性方程的形式表达了 Z_H 、 Z_{DR} 、 K_{DP} 的物理关联，可以实现对 Z_H 的精确临近预报，还具备传统机器学习方法不具备的可解释性。**主要结论如下：**1. 相邻区域之间具有很强的空间关联性，随着区域半径的增加，这种关联性逐渐减弱。2. 所提方法在 3km/5km 等高面情况下优于核回归、多层感知机（MLP）、长短记忆神经网络（LSTM）等基线方法，但在 1km 等高面时其预测效果略低于 MLP。3. 不同区域的降水规律对预测结果有较大影响，当有突发性降水时， Z_H 预报偏差大。

针对问题二：为解决当前数据驱动算法中存在的“回归到平均”问题，将其分解为两类子问题。一是无法预测极端降水事件的发生，二是无法准确预测极端降水事件发生之后的降水数据。针对子问题一，使用极值理论拟合极端降水事件分布，并采用 peaks-over-threshold（POT）方法得到极端降水事件的发生概率与极值。针对子问题二，在问题一结构因果模型的基础上，将极端降水事件抽象成干预，并基于卡尔曼滤波原理，对极端气象事件所带来的影响进行动态建模，得到了基于干预影响动态建模的 Z_H 预报模型，从而缓解预报的模糊效应。**主要结论如下：**1. 当突发性降水不规律时，POT 方法能够有效地预测到该极端降水事件。相反，POT 方法的性能会有所下降。2. 基于干预影响动态建模的 Z_H 预报模型能够对极端事件之后的 Z_H 值进行有效预测，相比较典型的 Causal Transfer（CF）算法，其最小平均误差（MAE）和最小均方误差（MSE）降低了 14.76% 和 66.37%。但是，该方法只能对“回归到平均”的问题进行缓解，并不能完全消除。

针对问题三：为解决利用 Z_H 和 Z_{DR} 进行定量降水估计的问题，考虑降水的“时序因果性”与“空间关联性”，提出了基于时空因果图神经网络的定量降水估计模型。在第二章因果贝叶斯网络的基础上，计算 Z_H 和 Z_{DR} 之间的因果强度。然后，使用图卷积模块提取空间相关性，使用时序卷积模块提取时序因果性，从而实现对降水量的定量估计。**主要结论如下：**

1. 时空图神经网络在求解降水量这种带有复杂时空关系的问题上具有一定优越性，与 Deep Switching Auto-Regression Factorization (DSARF)、LSTM 等基线方法相比，MAE 提升了 **25.79%** 和 **44.82%**。2. 将时序因果信息与空间关联信息引入到时空图神经网络中能够提高模型估计精度，与没有添加因果信息的时空图神经网络相比，其 MAE 和 MSE 分别提升了 **1.83%** 和 **0.15%**。

针对问题四：为解决双偏振雷达在强对流降水临近预报中的贡献度评估，优化数据融合策略的问题，提出了**基于反事实推理的数据融合策略优化方法**。在第四章时空因果图神经网络的基础上，将模型的输入替换为反事实样本，计算模型输出的变化，从而对双偏振雷达资料在强对流降水临近预报中的贡献进行评估。为了更好地应对突发性和局地性强的强对流天气，提出了**基于元学习的数据融合策略优化方法**。该方法以第四章模型为基本框架，通过输入不同组合的双偏振雷达观测数据 ($\{Z_H\}$, $\{Z_H, Z_{DR}\}$) 来训练模型得到基学习器。然后，以不同基学习器的降雨预报值作为输入，训练一个元学习器来输出不同模型预报值的集成结果。当输出误差达到最小时，理论上此时元学习模型代表最优融合策略。**主要结论如下：** Z_H 和 Z_{DR} 在强对流降水临近预报中都起到了重要作用，但是相对来说， Z_H 在强对流降水临近预报中起到的作用更为显著。2. 基于元学习的数据融合策略方法能够提高预测效果，但该效果只体现在 MSE 显著下降，而 MAE 却略有上升。主要原因为：元学习在学习过程中会把预测的数据作为训练集，这回导致降水量为 0 值区域的误差不断累加从而导致 MAE 的上升。同时，这种累计误差可能会使强降水区域的预测值不断趋近于真实值，导致 MSE 显著下降。因此，这种方法能更好地应对突发性和局地性强的强对流天气。

关键词： 强对流降水预报；因果推理；时序因果性；空间关联性；时空图神经网络；元学习

目 录

摘 要:	1
第 1 章 绪论	5
1.1 研究背景与意义	5
1.2 国内外研究现状	5
1.2.1 强对流降水临近预报方法.....	5
1.2.2 回归到平均缓解方法.....	6
1.2.3 特征重要性评估.....	6
1.2.4 因果推理.....	6
1.3 本文的主要内容与结构安排	7
1.3.1 论文主要研究内容.....	7
1.3.2 论文组织结构.....	7
1.4 本章小结	8
第 2 章 基于结构因果模型的 Z_H 临近预报.....	9
2.1 问题一分析	9
2.2 区域关联性分析	10
2.2.1 不同区域降水量统计	10
2.2.2 区域半径选择.....	10
2.3 临近预报模型	11
2.3.1 因果贝叶斯网络构建.....	11
2.3.2 基于随机过程的结构因果方程.....	14
2.4 模型求解	15
2.4.1 数据选择.....	15
2.4.2 实验设置.....	15
2.4.3 对比算法.....	15
2.5 结果展示与分析	15
2.5.1 因果贝叶斯网络.....	15
2.5.2 算法有效性分析.....	16
2.5.3 不同区域降水规律对算法性能影响.....	17
2.6 本章小结	19
第 3 章 基于极值理论与干预影响动态建模的预报模糊效应缓解.....	20
3.1 问题二分析	20
3.2 极端事件发生规律统计	21
3.2.1 极值理论.....	21
3.2.2 区域降水量分析.....	21
3.2.3 基于 POT 的极端降水事件分布拟合	21
3.3 干预影响动态模型	23
3.3.1 干预影响计算.....	23
3.3.2 融入动态干预影响的卡尔曼滤波.....	23
3.3.3 基于极值理论与干预影响动态建模的预报模糊效应缓解	24
3.4 模型求解	25
3.4.1 数据选择.....	25
3.4.2 实验设置.....	25

3.4.3 对比算法.....	25
3.5 结果展示与分析	25
3.5.1 极端气象事件 Z_H 预报结果.....	25
3.5.2 极端气象事件 1 小时后 Z_H 预报	25
3.6 本章小结	28
第 4 章 基于时空因果图神经网络的定量降水估计	29
4.1 问题三分析	29
4.2 Z_H 与 Z_{DR} 因果强度分析	29
4.3 定量降水估计模型	30
4.3.1 因果图结构提取.....	30
4.3.2 时序因果性-时序卷积	30
4.3.3 空间关联性-图卷积	31
4.4 模型求解	31
4.4.1 数据选择.....	31
4.4.2 对比算法.....	31
4.5 结果展示与分析	31
4.5.1 方法有效性分析.....	32
4.5.2 不同区域降水量估计值.....	32
4.6 本章小结	32
第 5 章 基于反事实推理的数据融合策略优化.....	33
5.1 问题四分析	33
5.2 基于反事实推理的双偏振雷达重要性评估.....	34
5.3 数据融合策略优化	34
5.3.1 常见的数据融合策略优化方法.....	34
5.3.2 基于元学习的数据融合优化策略.....	35
5.4 模型求解与分析	36
5.4.1 数据选择.....	36
5.4.2 双偏振雷达资料雷达重要性评估.....	36
5.4.3 数据融合策略有效性分析.....	36
5.5 本章小结	37
第 6 章 模型的改进与推广	38
6.1 模型的优点	38
6.2 模型的缺点	38
6.3 模型的改进与推广	39
参考文献	40
附录	42

第 1 章 绪论

1.1 研究背景与意义

强对流降水作为一种具有高强度、短时程、小空间尺度的降水现象，一直是气象预报领域的一大挑战。这种降水形式通常与雷暴、短时强降水、冰雹和龙卷风等灾害性天气现象相关联，对人们的日常生活和经济活动产生巨大的影响。因此，对强对流降水的准确预报对于减少其带来的损失和灾害至关重要。

在过去的几十年中，强对流降水预报的方法和技术经历了巨大的变革。传统的预报方法主要依赖于气象专家的经验 and 直觉，利用他们丰富的经验得出预测结果。这种主观预报方法虽然在某些情况下效果显著，但也存在局限性，尤其是在天气系统较复杂和不确定性较大的情况下。随着理论方法的进步，客观预报方法逐渐崭露头角。这些方法主要依赖于数值天气预报模型，基于大气的物理和动力学方程通过计算机模拟大气的运动和变化，从而预测未来的天气情况。然而，由于强对流降水的小尺度和短时程特性，传统的数值模型往往难以捕捉其细节和变化，导致预报出现不准确的问题^[1]。

近年来，随着深度学习和人工智能技术的发展，通过有效结合气象大数据，这些技术开始被应用于强对流天气预报领域。基于深度学习的预报方法，通过训练大量的历史天气数据，学习天气的模式和规律，从而提高预报的准确性。例如，基于深度学习的短时定量降水预测研究，通过训练雷达和卫星数据，可以更准确地预测降水的位置、强度和移动方向。此外，新的技术和方法，如基于双偏振雷达的降水短临预报技术、基于 TrajGRU 的短临预报实现等，也为强对流降水的临近预报带来了新的机会和挑战。这些技术结合了传统的数值模型和现代的深度学习方法，能够有效提高强对流天气预报的准确性和时效性。

强对流降水的临近预报是一个复杂而又具有挑战性的问题，本文在双偏振雷达获取的水平反射率因子、差分反射率和比差分相移等数据的基础上，针对提高预报精度、缓解模糊效应，优化数据融合策略的问题开展研究，旨在更好地应用双偏振雷达改进强对流降水短临预报效果，从而及时做到防灾减灾，为人们的生命和财产提供更好的保护。

1.2 国内外研究现状

1.2.1 强对流降水临近预报方法

强对流降水的临近预报方法是气象学中的重要研究方向，其目的是为了更准确、更及时地预测强对流降水事件。传统的强对流降水临近预报依靠数值天气预报模型开展，这些模型通过数理方程来模拟大气的运动和演变。目前，常用的数值模型包括 WRF、MM5 等，这些模型能够模拟大气中的水汽、温度、风速等参数，从而预测强对流降水的发生。近年来，人工智能技术在强对流降水临近预报中的应用逐渐增多。通过对大量的气象数据进行学习，机器学习模型能够发现数据中的模式和规律，从而实现强对流降水的预测。深度学习，尤其是卷积神经网络（CNN）和循环神经网络（RNN），在图像和序列数据分析中表现出色，因此在雷达和卫星数据分析中具有广泛的应用前景。CNN 通过卷积层提取空间特征，适合处理具有空间结构的气象图像数据^[2]；而 RNN 则能够处理时序数据，捕捉时间序列中的动态变化，适用于时序气象数据的分析^[3]。当前的研究重点聚焦于利用历史数据进一步提高预报的准确性，为了达到这一目的需要研究如何对原始数据进行转换和处理以提取出对模型学习有用的特征，以及使用数据融合方法整合来自不同源的气象数据以提供更为全面和准确的输入特征。

1.2.2 回归到平均缓解方法

回归到平均 (Regression to the Mean) 是一个统计学概念, 指的是在一系列随机变量中, 一个极端的观测值往往会被一个更接近平均值的观测值所跟随。选择合适的统计方法和模型可以缓解回归到平均的影响, 常见的方法有多层模型 (Multilevel Models)、贝叶斯方法、协变量调整、随机效应模型、交叉验证、引导法 (Bootstrapping) 等^[4]。多层模型可以处理具有层次结构或分组结构的数据, 能够考虑到组内和组间的差异, 从而减少误差和回归到平均的影响, 但是计算复杂度高, 对模型的假设和参数的解释较为复杂; 贝叶斯方法通过引入先验知识和后验分布, 提供了一种以概率的形式处理不确定性的方式, 方法较为直观且易于解释, 但是对于大型模型计算上可能非常复杂, 并且选择合适的先验分布时可能会引入主观性; 协变量调整可以帮助控制潜在的混杂变量, 增强因果关系的解释能力, 从而减少回归到误差的影响, 其缺点在于对选取协变量的要求较高, 如果协变量选择不当可能会引入偏见; 随机效应模型考虑了观测值之间的随机变异, 通过引入随机效应, 研究者可以更好地捕捉数据中的随机性, 从而减少回归到平均的影响, 但是其计算复杂度高, 并且需要足够的数据来估计随机效应; 交叉验证是一种评估模型预测性能的方法, 通过将数据分为训练集和测试集并多次重复这一过程, 可以评估模型的稳健性和预测能力, 但是计算成本高且分割数据可能导致某些子集的信息丢失; 引导法是一种基于重采样的统计方法, 可以用来估计参数的分布和置信区间, 通过多次重采样, 引导法可以提供对参数估计的不确定性的更好的了解, 从而减少回归到平均的影响。

1.2.3 特征重要性评估

特征重要性评估是机器学习中的一个关键步骤, 目的在于避免无用、冗余的特征, 使用易于理解的简单特征。它有助于我们理解模型的工作原理, 优化模型性能, 以及解释模型的预测。特征重要性评估方法可以大致分为三类: 过滤方法、包装方法和嵌入方法。过滤方法主要基于统计学的方法, 在模型训练之前评估特征的重要性, 这类方法的主要优点是计算速度快, 不依赖于任何机器学习算法, 因此它们通常被视为预处理步骤, 其缺点是它们可能会忽略特征之间的相互作用; 包装方法通过训练模型来评估特征的重要性, 这类方法考虑了特定模型的性能, 因此通常能够找到对给定模型最有用的特征子集, 例如递归特征消除 (RFE)、前向和后向特征选择, 但是包装方法的计算成本较高, 尤其是当特征数量很大时; 嵌入方法在模型训练过程中直接评估特征的重要性, 可以考虑特定模型的性能和特征之间的相互作用, 而且通常比包装方法更为高效, 例如 LASSO 回归^[5]、决策树和随机森林^[6]以及神经网络等。

1.2.4 因果推理

因果推理的目标是发现变量背后的因果关系。图灵奖获得者 Judea Pearl 将因果关系分为三层即: “因果发现”、“干预”和“反事实推理”^[7]。20 世纪 80 年代, 许多学者研究了图与概率论之间的关系^[10], 并且在时间序列数据中实现因果发现、干预估计和反事实推理方面取得很大的进展, 这给本文提供了良好的启发。

因果发现主要指挖掘数据之间的因果关系, 剔除冗余的相关关系, 得到更具有可解释性、更鲁棒的关联关系; 干预影响评估主要是指评价治疗手段、政策、环境变化等治疗措施对系统所产生的影响; 反事实推理用来回答“如果怎样将会怎样”的问题, 如果当初事情发生的方式与现实不一样, 那么结构将会如何, 是一种精确的归因分析方法。

本文主要解决四个问题, **如何有效应用双偏振变量改进强对流预报? 如何缓解预报的模糊效应? 如何进行定量降水估计? 以及如何评估双偏振雷达资料在强对流降水临近预报中的贡献?** 在前两个问题中, 使用因果发现方法可以获取目标变量的直接原因, 建立直

接原因与目标变量之间的确定性函数关系，获得更精确、更鲁棒的结果；在如何缓解预报的模糊效应中，预报的模糊效应是指当极端事件发生后，数据驱动方法的预测却更接近于平均值，这对于强对流预报来说是不合理的。因此可以把极端事件抽象成干预，来计算干预的影响，并对干预的影响进行动态建模，最后将干预的影响融入预测模型中，以缓解预报的模糊效应；对于评估双偏振雷达资料在强对流降水临近预报中的贡献这一问题，可以通过使用反事实推理来回答，如果使用的是传统雷达，那么估计的降水量将会如何？通过这种对比来评估双偏振雷达资料在强对流降水临近预报中的贡献。

1.3 本文的主要内容与结构安排

1.3.1 论文主要研究内容

论文针对强对流降水临近预报问题，从时序数据普遍存在的因果关系入手，研究了基于因果推理的强对流降水临近预报问题。首先，为解决有效应用双偏振变量改进强对流预报问题，从“时序因果性”与“空间关联性”两个维度入手，提出了基于结构因果模型的 Z_H 临近预报方法。然后，针对当前数据驱动算法存在的“回归到平均”问题，使用极值理论拟合极端事件的分布，得出极端时间的发生概率与极值。然后将极端天气抽象为“干预”，在结构因果模型的基础上，提出了基于干预影响动态建模的预报模糊效应缓解方法。再次，针对定量降水估计问题，考虑区域降水的“时序因果性”与“空间关联性”，提出了基于时空因果图神经网络的定量降水估计模型。最后，为解决双偏振雷达在强对流降水临近预报中的贡献度问题，优化数据融合策略，提出了基于反事实推理的数据融合策略优化方法。

1.3.2 论文组织结构

论文主要分 7 章对强对流降水临近预报问题展开研究，其组织结构图 1 所示，下面就论文各章的研究内容安排如下：

第 1 章：“绪论”。本章主要介绍了论文的研究背景与意义；分析了与论文研究内容相关工作的国内研究现状，主要包括强对流降水临近预报方法、回归到平均缓解方法、特征重要性评估方法以及因果推理研究现状；最后介绍了论文的主要内容及结构安排。

第 2 章：“基于结构因果模型的 Z_H 临近预报”。本章主要解决有效应用双偏振变量改进强对流预报问题。首先进行区域关联性分析，找到区域之间的空间关联性，其次提出了基于多元传递熵的因果贝叶斯网络构建方法，构造变量之间的因果关联；最后为了定量表示 Z_H 、 Z_{DR} 、 K_{DP} 之间的因果关联，根据多元传递熵的计算过程，推导出了结构因果模型，对 Z_H 进行定量预报。

第 3 章：“基于极值理论与干预影响动态建模的预报模糊效应缓解”。本章主要解决数据驱动算法中存在的“回归到平均”问题。在第 2 章结构因果模型的基础上，根据极值理论拟合极端事件的分布，得出极端事件发生的规律。然后，将极端气象事件抽象成干预，并对极端气象事件所带来的影响进行动态建模，将这种影响冗余结构因果模型中，得到了其随时间动态变化的状态空间模型，从而缓解预报模糊效应。

第 4 章：“基于时空因果图神经网络的定量降水估计”。本章主要解决利用 Z_H 和 Z_{DR} 进行定量降水估计的问题。根据第 2 章推导的因果贝叶斯网络，考虑区域降水的“时序因果性”与“空间关联性”，提出了基于时空因果图神经网络的定量降水估计模型。使用时序卷积模块提取 Z_H 和 Z_{DR} 之间的时序因果性，使用图卷积模块提取不同区域之间的空间关联性，从而提高估计模型精度。

第 5 章：“基于反事实推理的数据融合策略优化”。本章主要解决评估双偏振雷达资料在强对流降水临近预报中的贡献及如何数据融合策略优化的问题。根据第 4 章得到的定量降水估计模型，采用反事实推理方法，将输入更换为传统雷达信息数据，对二者情况进

行差异化表征，从而对双偏振雷达数据特征的重要性进行评估。为了更好地应对突发性和局地性强的强对流天气，提出了基于元学习的数据融合策略优化方法。该方法以第四章模型为基本框架，通过输入不同组合的双偏振雷达观测数据（ $\{Z_H\}$ ， $\{Z_H, Z_{DR}\}$ ）来训练模型预报降水量。然后，以不同模型的预报值作为输入，训练一个元学习器来输出不同模型预报值的集成结果。当输出误差达到最小时，理论上此时元学习模型代表最优融合策略。

第6章：“模型改进与推广”。对本文提出的模型的优点与缺点进行总结。

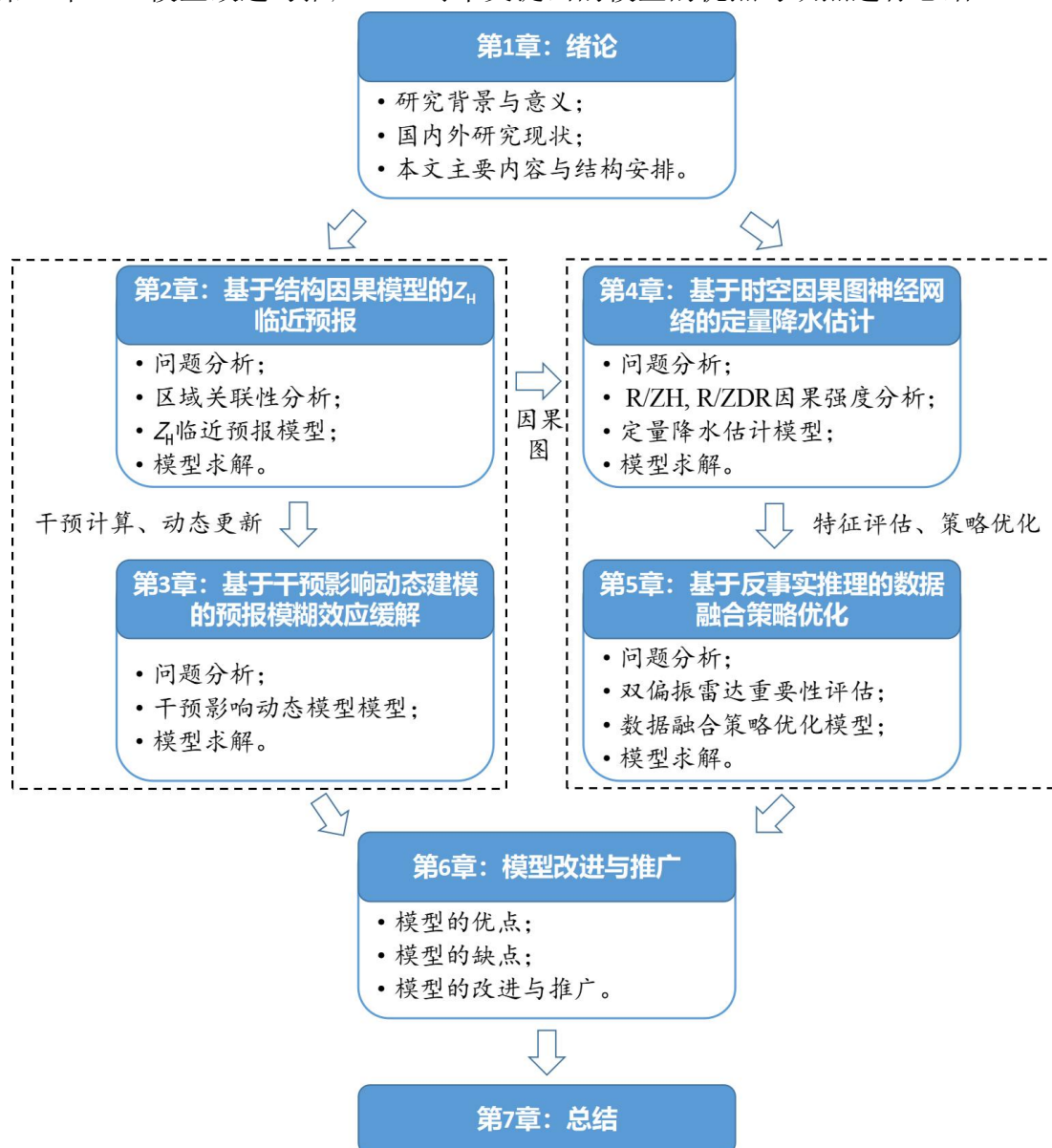


图 1 论文组织结构图

1.4 本章小结

本章介绍了强对流降水临近预报的研究背景与研究意义；并针对提出的四个问题，对强对流降水临近预报、回归到平均缓解方法、特征重要性评估这三个内容的研究现状进行了总结，指明了可以进一步研究的方法，并对贯穿全文的因果推理方法进行了介绍，并说明了技术路线需要使用因果推理的理由；最后介绍了本文的主要内容与结构安排。

第 2 章 基于结构因果模型的 Z_H 临近预报

2.1 问题一分析

问题一要求有效应用双偏振变量改进强对流预报，建立可用于强对流临近预报双偏振雷达资料中微物理特征信息的数学模型。临近预报的输入为前面一小时时（10 帧）的雷达观测量（ Z_H 、 Z_{DR} 、 K_{DP} ），输出为后续一小时（10 帧）的 Z_H 临近预报。本节从时序数据普遍存在的因果关系入手，结合关联区域历史降水数据信息，提出了一种基于结构因果模型的 Z_H 临近预报方法，主要流程如图 2。

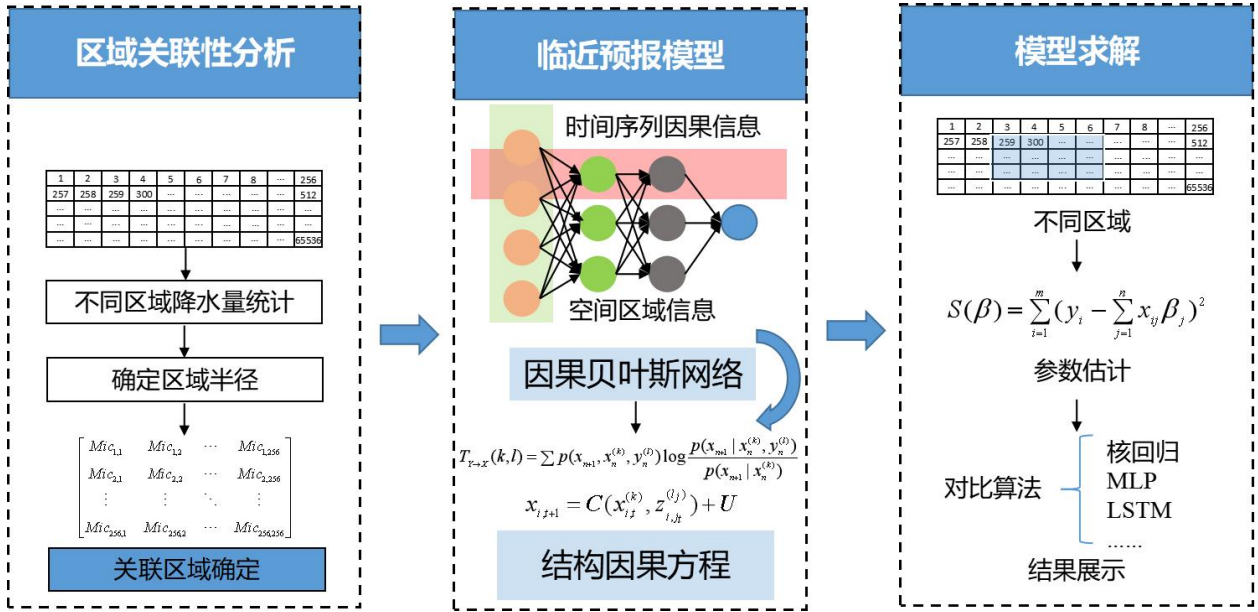


图 2 问题 1 求解思路

(1) **区域关联性分析**：每个地区的降水量不仅只受自身地区的影响，相邻地区的历史降水规律也能够提供一定借鉴意义（具有某些关联的地区）。因此，本节首先对 256×256 个平面区域每次降水的总量进行了统计，并定义“区域半径”来确定与之间的空间相关性。分别采用最大互信息系数（Maximal Information Coefficient, MIC）和皮尔逊相关系数并结合“区域半径”对不同区域的降水量关联联系进行计算，总结出不同区域的降水关联规律，找到与区域具有显著影响的关联区域。

(2) **临近预报模型**：1. 因果贝叶斯网络构建，在得到每个区域的“关联区域”后，使用多元传递熵构造自身区域与降水关联区域的 3 个等高面（1km, 3km, 7km）的 Z_H 、 Z_{DR} 、 K_{DP} 参数的因果动态贝叶斯网络，因果动态贝叶斯网络可以描述数据内涵的因果机制，可以用于提取强对流临近预报双偏振雷达资料中微物理特征信息的数学模型。2. **结构因果方程推导**，基于因果贝叶斯网络，从时间序列的因果关系入手，推导了 Z_H 、 Z_{DR} 、 K_{DP} 参数的结构因果模型（SCM）。该模型以确定性方程的形式表达了 Z_H 、 Z_{DR} 、 K_{DP} 的物理关联，可以实现对 Z_H 的精确临近预报，还具备传统机器学习方法不具备的可解释性。

(3) **模型求解**：以不同区域为背景，结合参数估计对临近预报模型进行求解，并与 MLP、核回归、长短记忆神经网络（Long Short Term Memory networks）等预测算法进行对比，得出相应结论。

2.2 区域关联性分析

2.2.1 不同区域降水量统计

为了总结出区域相关性，提取区域之间的相关特征，统计出了 256×256 （共计 65536）个区域的所有降水次数的降水量总和，如图 3 所示。

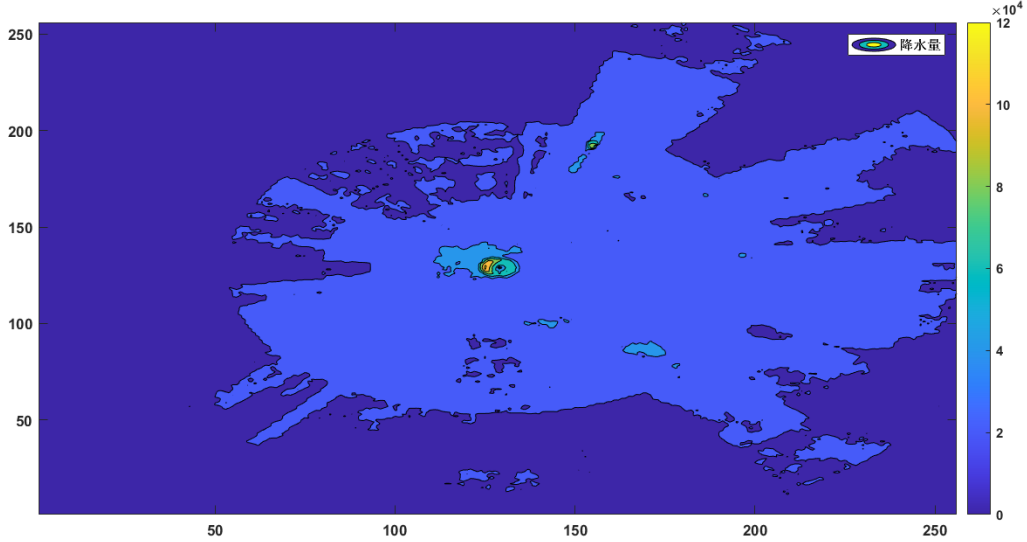


图 3 不同区域降水量

从图 3 可以看出，这一 $256\text{km} \times 256\text{km}$ 区域的降水主要集中在中心靠右区域，不同地方之间的降水差别较大，区域之间可能存在某种空间相关性，为了衡量不同区域之间的空间相关性，下面首先定义“区域半径”，然后结合最大互信息与皮尔逊相关系数对不同区域之间的关联进行量化。

2.2.2 区域半径选择

区域半径的定义如图 4 所示，假设被关联的区域为第 K 个区域，当它的区域半径为 $R = 2$ 时，其“邻居区域”如蓝色区域所示。相应的计算公式为：

$$Ty_top/bottom = K_{1 \times R} \mp 256 * [1:R], \quad Ty_left/right = [Ty_top, Ty_bottom] \mp ones(2R,1) * [1:R] \quad (1)$$

1	2	3	4	5	6	7	8	...	256
257	258	259	300	...	...	...	...	...	512
...	...	...	...	K	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	65536

1	2	3	4	5	6	7	8	...	256
257	258	259	300	...	...	...	...	...	512
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	65536

图 4 区域半径定义

需要说明的是，当 K 位于两侧时，其计算公式需要考虑一定约束。例如，当 $K = 258$ ，且 $R = 2$ 时，其邻居区域如图 4（右），相关计算代码见附录，在此不做赘述。

为了选择合适的区域半径，本节设计了基于贪婪搜索的区域半径选择方法，力求在得到最佳能够体现空间依赖性的区域半径，以减少计算时间。基本思路为：

- （1）给定区域半径 R ，计算区域的“邻居区域”；
- （2）计算区域与邻居区域的降水量关系；
- （3）遍历 R ，找到最佳半径（通过不同区域降水量关系求得）。

最大互信息系数和皮尔逊相关系数在计算变量关系当面起到了重要的作用。为了避免单一变量导致的计算结果偏差，本节选择最大互信息系数和皮尔逊相关系数来计算区域与邻居区域的降水量关系。

最大互信息系数：衡量两个变量之间的关联程度、线性和非线性关系，相比较互信息具有更高的准确度，且具有一定的普适性、公平性和对称性。其计算公式如下：

$$Mic(x, y) = \max_{a \leq b \leq B} \frac{I(x; y)}{\log_2 \min(a, b)} \quad (2)$$

其中 a, b 是在 x, y 方向上划分格子的个数。 B 是变量， $I(x; y)$ 表示 x, y 的互信息。

皮尔逊相关系数：表示两个变量之间协方差和标准差的商，用来表示两者间线性相关性，其值介于-1 到 1 之间，公式如下：

$$\rho_{x,y} = \frac{cov(X, Y)}{\sigma_x \sigma_y} = \frac{E[(X - \mu_x)(Y - \mu_y)]}{\sigma_x \sigma_y} \quad (3)$$

其中 σ 为标准差， cov 为协方差， E 为数学期望， μ 为均值。

随机选择八个区域(坐标分别为[35,175], [38, 210], [85, 179], [128, 125], [130, 126], [180, 150], [191, 155], [225, 170])及邻域半径 R 为输入(考虑计算时间与空间纬度, R 选择 1 至 20)，在不同邻域半径下计算不同区域与其邻居区域的最大互信息系数和皮尔逊相关系数，力求获得不同区域的最佳区域半径。得到结果如图 5，具体数据见附录。

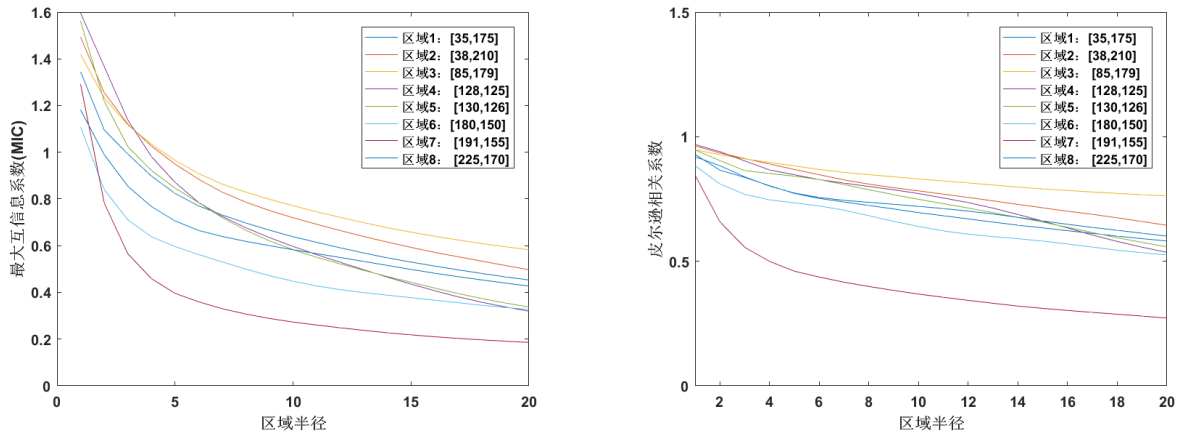


图 5 不同区域半径下区域与其邻居区域的最大互信息系数和皮尔逊相关系数

图 5 不同区域半径下区域与其邻居区域的最大互信息系数和皮尔逊相关系数，根据图 5 可知，当区域半径为 1 时，每种区域与其邻居区域相关性非常强，这证明相邻区域之间具有很强的相关性。同时，随着区域半径的增加，其相关性逐渐减弱。因此，我们可以得出如下结论：区域与区域之间是存在空间相关性的，当区域半径为 1 时，区域之间的关联性很强。并且随着区域半径增加，这种关系单调递减。

为了得到不同区域的关联区域，我们认为当最大互信息系数与皮尔逊相关系数均大于 0.8 时，两个区域之间是存在显著关联的。基于这种约束，便可得到不同区域的关联区域。因此，在对某个区域的 Z_H 进行预报时，不仅要考虑自身区域的双偏振雷达信息，还需要考虑关联区域的双偏正雷达信息。

2.3 临近预报模型

在 2.2 节中分析出的区域关联性的基础上，提出了一种基于结构因果模型的 Z_H 临近预报方法。首先基于多元传递熵构造了能够体现双偏振雷达资料中微物理特征的因果贝叶斯网络；随后基于多元传递熵的计算过程，推导出了结构因果方程来对 Z_H 进行定量预报。

2.3.1 因果贝叶斯网络构建

(1) 因果贝叶斯网络

贝叶斯网络由概率有向无环图（DAG）表示，其中顶点是随机变量，弧表示对应于弧的两端节点的两个变量之间的概率依赖关系^[12]。在图中，一个顶点 v 有一个条件概率表 $P(v | \text{Parent}(v))$ ，描述了变量与其父节点之间的关系。贝叶斯网络的一个重要特征是局部有向马尔可夫条件，这意味着，以其父节点为条件， v 独立于网络中既不是 v 的后代也不是父节点的变量，这样的特征使得联合概率分布表示为式(4)：

$$P(v_1, v_2, \dots, v_n) = \prod_{i=1}^n P(v_i | v_{i-1}, \dots, v_1) = \prod_{i=1}^n P(v_i | \text{Parent}(v_i)) \quad (4)$$

在贝叶斯网络中，弧的方向不需要有意义，其中一些甚至可能不遵循时间顺序。这是因为两个网络 $A \rightarrow B$ 和 $A \leftarrow B$ 在概率模型中是等价的，产生相同的边际分布并对相同的查询（例如 $P(A|B)$ ）产生相同的概率推断。

因果贝叶斯网络（Causal Bayesian Network, CBN）与贝叶斯网络有相同的形式，不同之处在于，在因果网络中，变量 A 的父节点是 A 的直接原因。换句话说，当且仅当 A 是指定总体中 B 的直接原因（描述为给定数据集）时，弧 $A \rightarrow B$ 存在因果贝叶斯网络中。因果贝叶斯网络采用贝叶斯网络来描述因果信息和传统的条件依赖，它们通过概率演算来计算因果效应，因此贝叶斯网络是因果网的基础^[13]。因果动态贝叶斯网络在因果贝叶斯网络的基础上考虑了因果参数之间的时间滞后，如图 6。

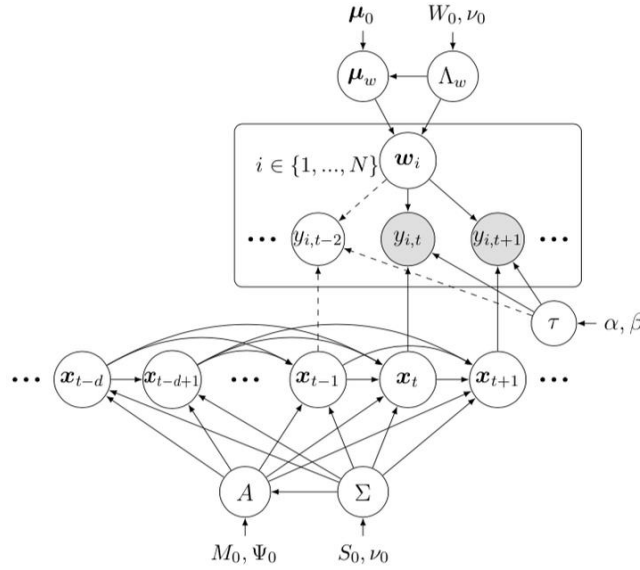


图 6 因果动态贝叶斯网络

因果动态贝叶斯网络不仅直观的展示了目标变量与其因变量的定性关系，而且反映了其时间相关性，所以可以使用因果贝叶斯网络对双偏振雷达资料中微物理特征之间的因果关联进行建模。

(2) 基于多元传递熵的因果贝叶斯网络构建

基于多元传递熵的因果贝叶斯网络构建方法的输入为 $Z_{\text{self}} = \{Z_H, Z_{DR}, K_{DP}\}$ 、 $Z_{\text{other}} = \{Z_H, Z_{DR}, K_{DP}\}$ ，其中 Z_{self} 代表本区域的双偏振雷达信息， Z_{other} 代表相关区域的双偏振雷达信息， $Z_H = \{Z_{H1}, Z_{H3}, Z_{H7}\}$ 代表 1km、3km、7km 三个等高面的 Z_H 参数， $Z_{DR} = \{Z_{DR1}, Z_{DR3}, Z_{DR7}\}$ 代表 1km、3km、7km 三个等高面的 Z_{DR} 参数， $K_{DP} = \{K_{DP1}, K_{DP3}, K_{DP7}\}$ 代表 1km、3km、7km 三个等高面的 K_{DP} 参数，输出为因果动态贝叶斯网络 G_{CDBN} 。

传递熵定义如下：假设 $X : \{x_n, n=1, 2, \dots, N\}$ ， $Y : \{y_n, n=1, 2, \dots, N\}$ 为两个随机变量， X 是具有 k 阶的马尔科夫过程， Y 是具有 l 阶的马尔科夫过程，可以得到下式：

$$p(x_{n+1} | x_n, x_{n-1}, \dots, x_1) = p(x_{n+1} | x_n, x_{n-1}, \dots, x_{n-k+1}) \quad (5)$$

上式表示 X 的下一个状态 x_{n+1} 与它自身的前 k 个状态有关。同样的 Y 的下一个状态 y_{t+1} 与它自身的前 l 个状态有关，设 $x_n^{(k)} = (x_n, x_{n-1}, \dots, x_{n-k+1})$ ， $y_n^{(l)} = (y_n, y_{n-1}, \dots, y_{n-l+1})$ ，为长度分别是 k 和 l 的时间序列。

考虑 Y 向 X 传递的信息：

$$p(x_{n+1} | x_n^{(k)}) = p(x_{n+1} | x_n^{(k)}, y_n^{(l)}) \quad (6)$$

可以认为 Y 的状态与 X 上的转移概率无关。假设下个状态 x_{n+1} 取决于自身的前 k 个状态与 Y 的前 l 个状态，传递熵的公式定义为：

$$T_{Y \rightarrow X}(k, l) = \sum p(x_{n+1}, x_n^{(k)}, y_n^{(l)}) \log \frac{p(x_{n+1} | x_n^{(k)}, y_n^{(l)})}{p(x_{n+1} | x_n^{(k)})} \quad (7)$$

传递熵被认为是在数据充足的情况下，发现时间序列因果关联的最稳定的方法，因此在传递熵的基础上构建因果贝叶斯网络。

从图 6 可以看出，要构造双偏振雷达参数的因果贝叶斯网络，需要找到每个节点的直接原因，在本节中通过信息分解的方法找寻每个节点的直接原因^[14]。将参数的直接原因叫做解释参数，参数本身叫做响应参数。通过信息分解的方法来寻找生成每个参数的下一步状态 $x_{i,t+1}$ 的信息来源。在复杂系统中，下一步状态 $x_{i,t+1}$ 的信息来源除了不确定性或者随机性的信息 U_{x_i} 以外，还包括系统中 X_i 的解释参数贡献的信息 $I(V_{x_i})$ ， V_{x_i} 表示系统中响应参数的解释参数集合。因此 X_i 的熵可以表示为：

$$H_{x_i} = I(V_{x_i}) + U_{x_i} \quad (8)$$

响应参数 X_i 的解释参数除了系统中其他参数贡献的信息外，还包括自身的历史信息，因此解释参数的信息 $I(V_{x_i})$ 可以进一步分解为：

$$I(V_{x_i}) = A_{x_i} + T_{V_{x_i}/x_i} \quad (9)$$

因此参数 X_i 的下一个状态 $x_{i,t+1}$ 的信息来源如下式所示：

$$H_{x_i} = A_{x_i} + T_{V_{x_i}/x_i} + U_{x_i} \quad (10)$$

$T_{V_{x_i}/x_i}$ 代表除去自身的历史信息外，其他解释参数贡献的信息。参数自身的历史信息 A_{x_i} 通过主动信息存储来获得，同过主动信息存储得到参数 X_i 的前 k 步是 $x_{i,t+1}$ 的信息来源。假设参数 X_i 的解释参数除了自身的历史信息外还包括参数集合 $\mathbf{Z}_{x_i} = \{Z_{x_{i_1}}, Z_{x_{i_2}}, \dots, Z_{x_{i_Q}}\}$ ， Q 为参数 X_i 的直接原因参数个数，因此 $T_{V_{x_i}/x_i}$ 可以如下式表示：

$$T_{V_{x_i}/x_i} = \sum_{j=1}^Q T_{Z_{x_{i_j}} \rightarrow x_i} \quad (11)$$

使用多元传递熵的方法来确定 $\mathbf{Z}_{x_i} = \{Z_{x_{i_1}}, Z_{x_{i_2}}, \dots, Z_{x_{i_Q}}\}$ ，多元传递熵使用贪婪算法通过计算响应参数与目标参数之间的传递熵，筛选出传递熵值显著的目标参数，并通过优化的方法得到贡献信息最大的马尔科夫阶数 l_i ，最后通过剪枝的方法剔除冗余的因果关系，剪枝的比例不大于 50%，得到响应参数的系统解释参数集合 $\mathbf{Z}_{x_i} = \{Z_{x_{i_1}}, Z_{x_{i_2}}, \dots, Z_{x_{i_Q}}\}$ ， $\mathbf{l} = \{l_1, l_2, \dots, l_Q\}$ 代表对应每个解释参数的马尔科夫阶数。具体步骤如下所示：

步骤 1：初始化 $\mathbf{Z}_{x_i} = \varnothing$ ；

步骤 2：对于每个可能的解释参数 $\mathbf{Z}_{x_i} \in \mathbf{D} \setminus \mathbf{Z}_{x_i}$ ， $\mathbf{D}$ 是系统中所有参数数量，计算 $T_{Z_{x_i} \rightarrow x_i | \mathbf{Z}_{x_i}}$ 。

步骤 3：选择为 X_i 的下一个状态提供了统计显著的信息且提供了最大增量信息贡献

($T_{Z_{X_i} \rightarrow X_i | Z_{X_i}}$) 的参数 $Z_{X_i} \in D \setminus Z_{X_i}$ 。

步骤 4: 重复 2、3 步骤, 直到没有参数 Z_{X_i} 可以添加到 Z_{X_i} 中。

步骤 5: 当找不到提供统计显著信息贡献 $T_{Z_{X_i} \rightarrow X_i | Z_{X_i}}$ 的源 Z_{X_i} 时, 考虑联合参数 $Z_{X_i} = \{Z_{X_i,a}, Z_{X_i,b} | Z_{X_i,a}, Z_{X_i,b} \in D \setminus Z_{X_i}\}$ 而不是单个参数 Z_{X_i} , 然后重复步骤 2 和 3。

步骤 6: 经过上述步骤, 确定了 Z_{X_i} (即没有进一步的参数向目标参数提供统计上显著的信息), 再测试在给定了其它解释参数的前提下, 每个 $Z_{X_i} \in Z_{X_i}$ 是否添加了具有统计显著性的信息量 $T_{Z_{X_i} \rightarrow X_i | Z_{X_i} \setminus Z_{X_i}}$ 。如果参数未通过检验, 则将该参数从 Z_{X_i} 中删除。需要说明的一点是, 如果有多个参数未通过检验, 那么只删除提供信息 ($T_{Z_{X_i} \rightarrow X_i | Z_{X_i} \setminus Z_{X_i}}$) 最少的参数 Z_{X_i} 。

因果发现方法的具体步骤如图 7。

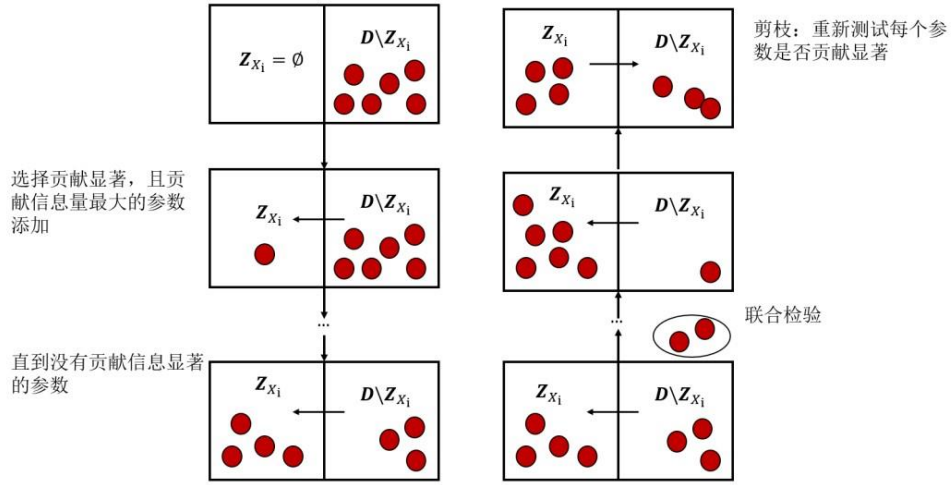


图 7 因果发现流程图

因此生成 $x_{i,t+1}$ 的完整信息来源如下式所示:

$$H_{X_i} = A_{X_i}(k) + \sum_{j=1}^Q T_{Z_{X_{ij}} \rightarrow X_i}(k, l_j) + U_{X_i} \quad (12)$$

通过上式可以构造出 Z_H 的因果贝叶斯网络, 为结构因果方程的进一步确立提供指导。

2.3.2 基于随机过程的结构因果方程

理论上, 在得到了系统中参数的因果贝叶斯网络结构后, 可以通过各种参数学习方法获得式 (4) 中因果贝叶斯网络的条件概率, 可以知道数据生成机制。但是对于具有连续变量和树宽的模型, 条件概率的学习通常是非常耗时的。因此, 为了降低学习成本, 引入 SCM 来描述数据生成过程, 同时使用该模型进行预测。

解释参数对响应参数的传递熵可以表示为:

$$T_{Z_{X_{ij}} \rightarrow X_i}(k, l_j) = \sum p(x_{i,t+1}, x_{i,t+1}^{(k)}, z_{X_{ij}}^{(l_j)}) \log \frac{p(x_{i,t+1} | x_{i,t}^{(k)}, z_{X_{ij}}^{(l_j)})}{p(x_{i,t+1} | x_{i,t}^{(k)})} \quad (13)$$

由上式可知, 传递熵计算过程中要估计大量的概率分布, 为了加快传递熵的计算。对于多元高斯分布, 假设有随机向量 $\mathbf{\Gamma}$, 它分为分量 $\mathbf{A}$ 和 $\mathbf{B}$, $\mathbf{A}$ 和 $\mathbf{B}$ 都服从多元高斯分布, 相应的均值向量为 μ_A 和 μ_B , 方差协方差矩阵可以分为以下四个部分:

$$\mathbf{\Gamma} = \begin{pmatrix} \mathbf{A} \\ \mathbf{B} \end{pmatrix} \sim N \left(\begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix}, \begin{pmatrix} \Sigma A & \Sigma AB \\ \Sigma BA & \Sigma B \end{pmatrix} \right) \quad (14)$$

其中, ΣA 是随机向量 $\mathbf{A}$ 的方差协方差矩阵, ΣB 是随机向量 $\mathbf{B}$ 的方差协方差矩阵, ΣBA 包含 $\mathbf{A}$ 元素和 $\mathbf{B}$ 对应元素之间的协方差。

那么，假设 $\mathbf{A}$ 取一个特定值 $\mathbf{a}$ ， $\mathbf{B}$ 的条件分布也将是具有条件期望的多元高斯分布，即：

$$\mathbf{E}(\mathbf{B} | \mathbf{A} = \mathbf{a}) = \sum_{BA} \Sigma_A^{-1} \mathbf{a} + \boldsymbol{\mu}_B - \sum_{BA} \Sigma_A^{-1} \boldsymbol{\mu}_A \quad (15)$$

由上式可知，如果随机变量 $\mathbf{A}$ 、 $\mathbf{B}$ 服从多元高斯分布，那么在给定 $\mathbf{A}$ 的情况下， $\mathbf{B}$ 的期望可以表示为 $\mathbf{B}$ 的线性函数，因此对于概率密度 $p(x_{i,t+1} | x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)})$ ，在传递熵的计算过程中 X_i 、 $\mathbf{Z}_{X_i}$ 服从多元高斯分布，将其划分为两部分 $\mathbf{B} = x_{i,t+1}$ ， $\mathbf{A} = p(x_{i,t+1}^{(k)}, z_{x_{ij_t}}^{(l_j)})$ ，则有：

$$\mathbf{E}(\mathbf{B} | \mathbf{A} = (x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)})) = \sum_{BA} \Sigma_A^{-1} \mathbf{a} + \boldsymbol{\mu}_B - \sum_{BA} \Sigma_A^{-1} \boldsymbol{\mu}_A \quad (16)$$

令 $\sum_{BA} \Sigma_A^{-1} = \mathbf{C}$ ， $\boldsymbol{\mu}_B - \sum_{BA} \Sigma_A^{-1} \boldsymbol{\mu}_A = \mathbf{U}$ ，则有：

$$\mathbf{E}(\mathbf{B} | \mathbf{A} = (x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)})) = \mathbf{C}(x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)}) + \mathbf{U} \quad (17)$$

又因：

$$\mathbf{E}(x_{i,t+1} | \mathbf{A} = (x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)})) = x_{i,t+1} \quad (18)$$

因此，结构因果模型如下式所示：

$$x_{i,t+1} = \mathbf{C}(x_{i,t}^{(k)}, z_{x_{ij_t}}^{(l_j)}) + \mathbf{U} \quad (19)$$

上式表明， $x_{i,t+1}$ 可以用其解释参数集合线性表示，其中 $\mathbf{C}$ 代表系数矩阵， $\mathbf{U}$ 代表噪声。其系数会随着时间的变化、干预的影响而发生改变。在获得了表示数据生成过程的结构因果模型后，需要估计参数 $\mathbf{C}$ 和 $\mathbf{U}$ 。常用的参数估计方法有矩估计、最小二乘估计和最大似然估计。本文采用最小二乘估计对未知参数进行估计，以简化计算过程。

2.4 模型求解

2.4.1 数据选择

为了对模型进行更精确的求解，在实验过程中根据“华为杯第二十届中国研究生数学建模竞赛 F 题附件数据下载及赛提数据格式说明”中描述对所有变量进行了归一化处理。

2.4.2 实验设置

在实验过程中，除了要预报区域的 Z_H 、 Z_{DR} 、 K_{DP} 外，还选了其对应关联区域（区域半径为 4）的 Z_H 、 Z_{DR} 、 K_{DP} 作为特征值对要预报区域的 Z_H 进行临近预报。实验中训练集的比例为 0.8，测试集的比例为 0.2。

2.4.3 对比算法

为了进一步验证本文提出方法提出的有效性，将本文提出的方法与核回归、多层感知机（MLP）、LSTM 方法进行了比较。核回归方法是一种非参数的回归方法，对数据分布不做要求；多层感知机是一种非线性的方法，理论上可以拟合任意的函数关系；LSTM 方法是循环神经网络的代表，广泛应用在时序数据预测中。选取了核回归、多层感知机、LSTM 这三种具有代表性的方法进行比较，能更有效地反映本文提出方法的有效性。

2.5 结果展示与分析

2.5.1 因果贝叶斯网络

通过基于多元传递熵的因果贝叶斯网络方法得到的因果贝叶斯网络如下图 8 所示（以区域 1 为例）：

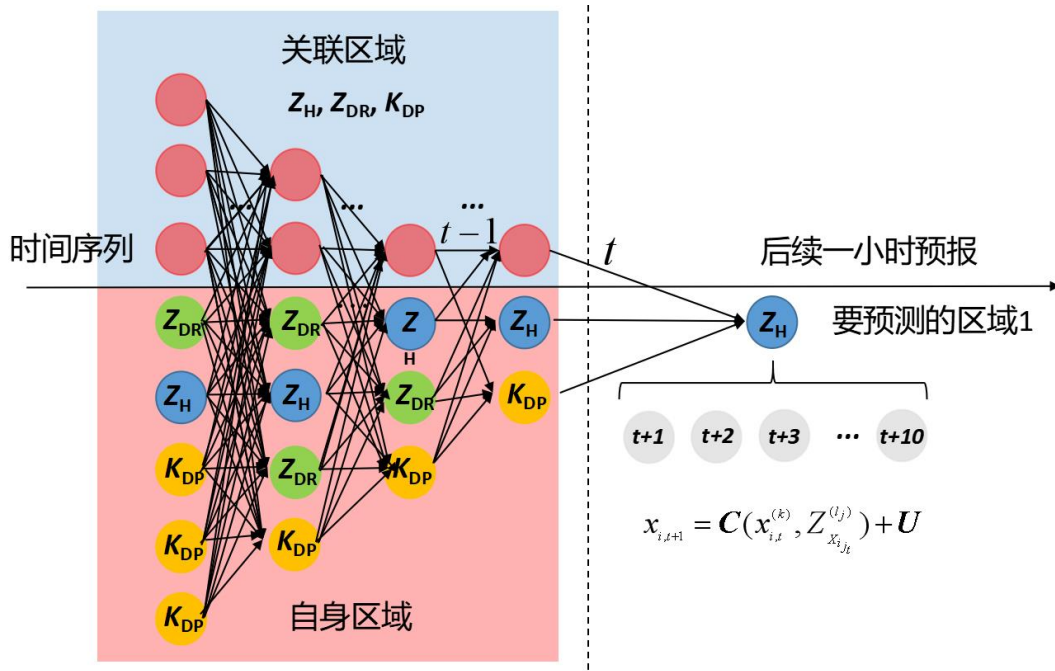


图 8 因果贝叶斯网络

2.5.2 算法有效性分析

根据推导出的因果动态贝叶斯网络，使用最小二乘估计进行参数估计，构造了结构因果模型，对 Z_H 后续一小时（10 帧）的值进行了预报，使用均方误差（MSE）、平均绝对误差（MAE）来衡量本文提出方法的有效性，MSE，MAE 的计算方法如式（20）、式（21）。

$$MSE = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2 \quad (20)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y - \hat{y}| \quad (21)$$

其中， n 代表预测样本的总数量， y 代表实际值， $\hat{y}$ 代表预测值。将本文提出的方法与核回归、MLP、LSTM 这三个方法进行了比较，比较结果如表 1 所示。

表 1 Z_H 临近预报结果比较

等高面/km	方法	MAE	MSE
1	本文方法	0.043322	0.012231
	核回归	0.070773	0.017063
	MLP	0.01767	0.001753
	LSTM	0.080925	0.012756
3	本文方法	0.017103	0.002149
	核回归	0.051766	0.009033
	MLP	0.02424	0.002254
	LSTM	0.056406	0.006877
7	本文方法	0.017975	0.002611
	核回归	0.051415	0.009154
	MLP	0.022791	0.002758
	LSTM	0.052994	0.007898

表 1 为 Z_H 的临近预报结果。从表 1 我们发现一个有意思的现象，当等高面为 1km 时，MLP 算法表现优异，当等高面为 3km/5km 时，本文提出的方法性能凸显。当等高面分别为 3km/7km 时，本文提出的算法在 MAE 方面相比较核回归、MLP、LSTM 方面提高了约 66.96%、29.44%、69.68%以及 65.04%、21.13%、66.08%。在 MSE 方面相比较核回归、MLP、LSTM 方面提高了约 76.21%、4.66%、68.75%和 71.48%、5.33%、66.94%。虽然当等高面为 1km 时，本文效果不如 MLP，但是在 3km/7km 等高面时，本文提出的算法均表现了优异性能。因此，本文提出的算法是有效的。

2.5.3 不同区域降水规律对算法性能影响

为了验证不同加区域降水规律对算法预测性能的影响，以 data_dir_225 这一降水事件为例，使用 31-40 时间段的双偏振雷达信息来对 41-50 时间段的 Z_H 进行预报，不同等高面下的预测初始时刻 Z_H 分布如图 9 所示，得到不同时间段的预报偏差如图 10 所示。

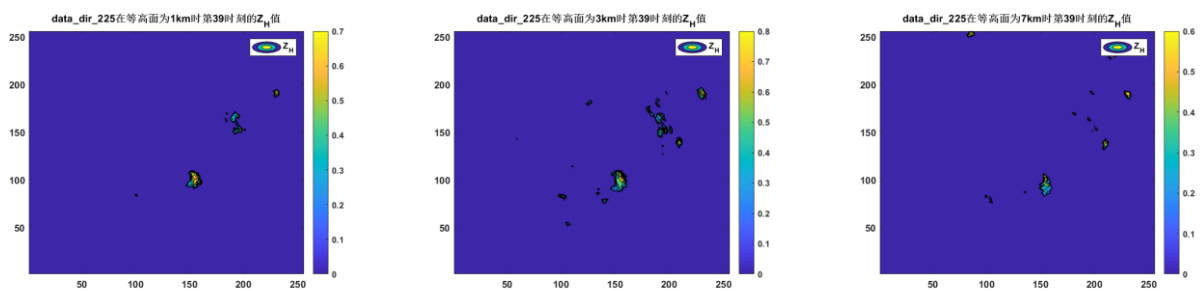
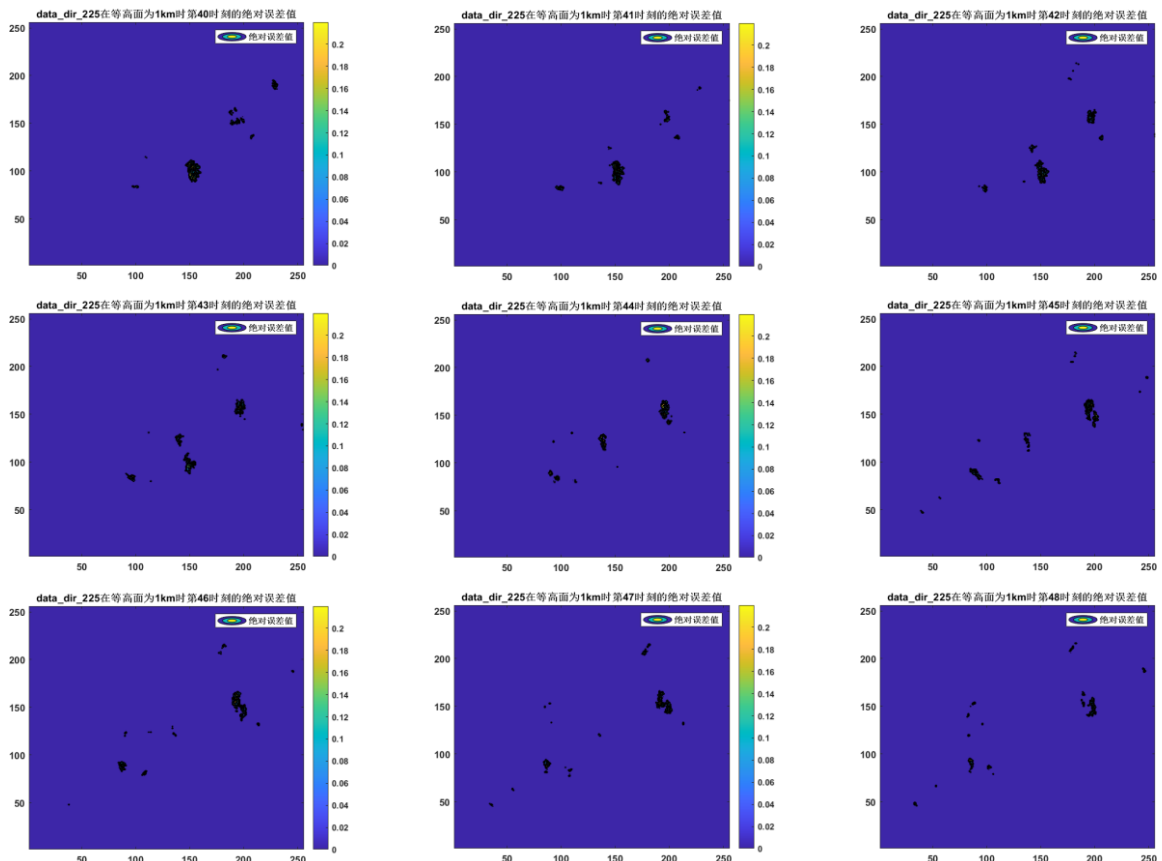
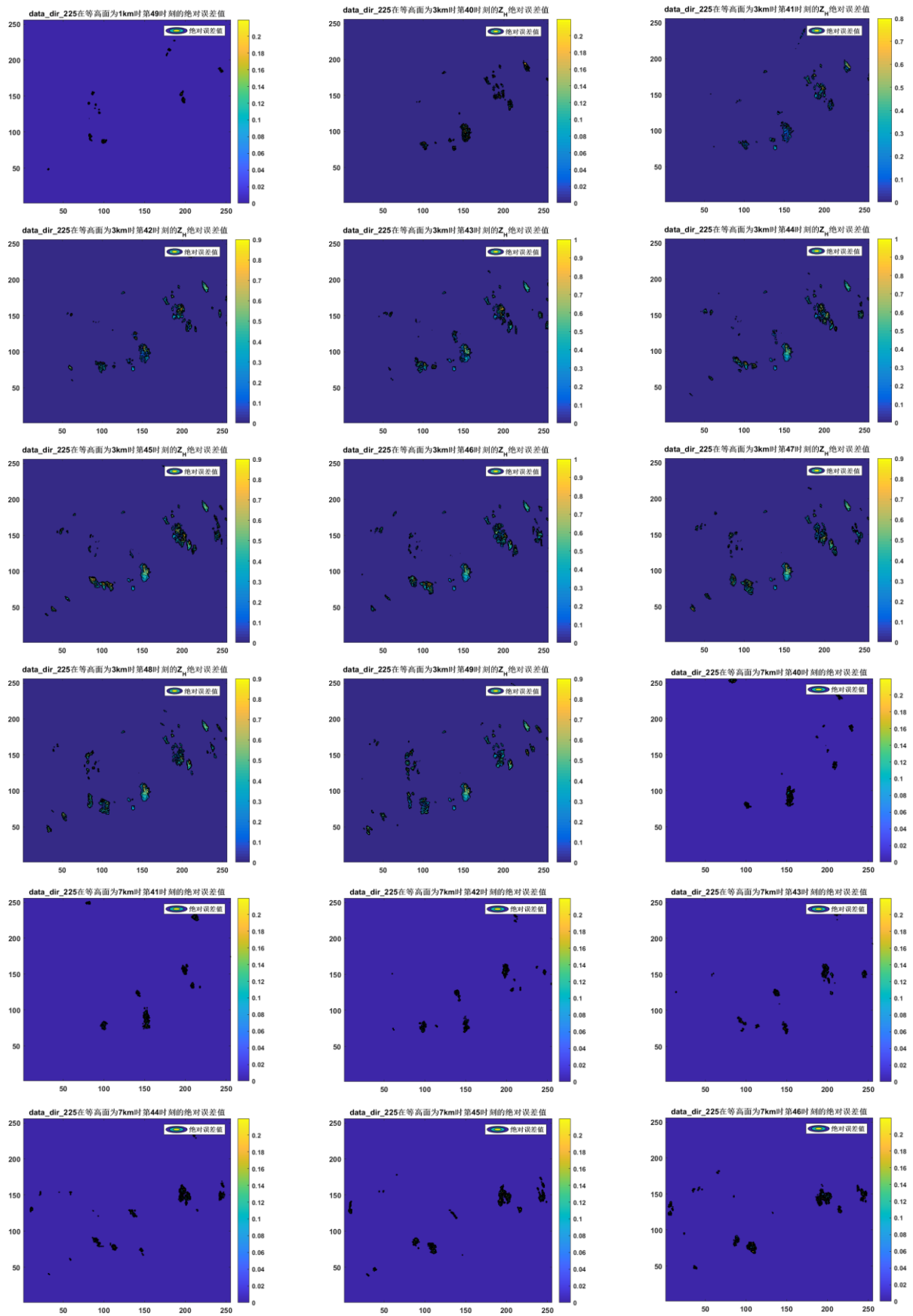


图 9 data_dir_225 在不同等高面下初始时刻的 Z_H 值

图 9 为 data_dir_225 在不同等高面下初始时刻的 Z_H 值。根据图 9 可知，同一地点在不同等高面 Z_H 数值有较大差别，预测时需要对不同等高面分开预测。图 10 展示了等高面为 1km、3km 和 7km 时，40-49 时间段内的 Z_H 预报绝对误差值（以本文提出的算法作为预报）。





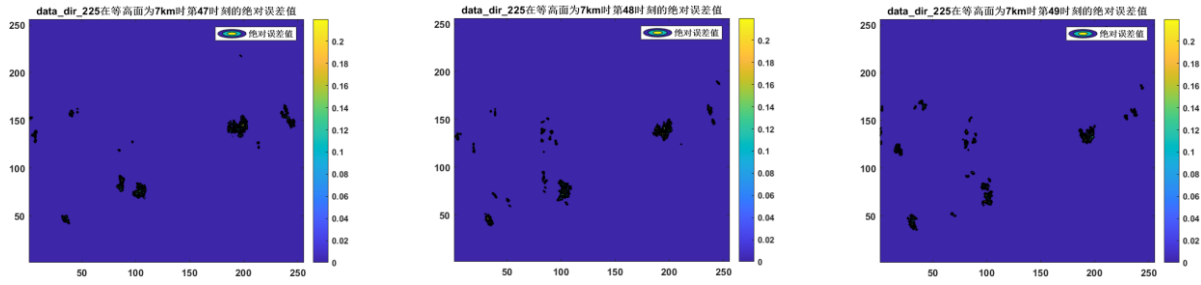


图 10 data_dir_225 在不同等高面下 40-49 时间段内的 Z_H 预报绝对误差值

图 10 为 data_dir_225 在不同等高面下 40-49 时间段内的 Z_H 预报绝对误差值。根据图 10 可知，不同区域的预测精度差异较大。有的地区能够实现精准预测，而有的地区误差比较大。经过分析数据可知，预测精度较高的地区，要么是常年不下雨，即降水总量为 0，要么降水比较规律，所提出的算法能够很好的学习到这种规律。而对于常年降水不规律的地区，降水差异比较大，如[128, 125], [130, 128]等区域，这在下一章会详细介绍。因此，我们猜测，本文提出的算法受不同区域的降水规律影响，且降水规律性与算法的预报误差成反比。即：出现题目二中“回归到平均”的问题，为了解决问题 2，第三章提出了基于极值理论与干预影响动态建模的预报模糊效应缓解方法。

2.6 本章小结

本节主要设计了结构因果模型来有效应用双偏振变量对 Z_H 进行预报。首先对不同区域的降水总量进行统计，提出区域半径的概念，找到区域的关联区域。然后，在邻居区域的基础上，基于传递熵设计了因果贝叶斯网络，确定对 Z_H 进行预报的网络结构。最后，基于时间序列数据因果特性，推导了描述数据生成过程的结构因果方程，以实现对其的准确预测。主要结论如下：（1）当区域半径较小时区域与其邻居区域之间的空间相关性十分显著，并且随着区域半径增加，这种空间相关性逐渐减小。（2）当等高面为 1km 时，MLP 方法的预测性能优于本文提出的方法，但当等高面为 3、7km 时，相比较其他算法，本文提出的预测方法能够达到最优效果。（3）讨论了不同区域降水规律对预报误差的影响，在本章算例中验证了问题 2 的存在性。

第3章 基于极值理论与干预影响动态建模的预报模糊效应缓解

3.1 问题二分析

当前一些数据驱动算法在进行强对流预报时，倾向于生成接近于平均值的预报，即存在“回归到平均（Regression to the mean）”问题，因此预报总是趋近模糊（这点在第2章的结论也有提及）。问题二要求在问题一的基础上，设计数学模型以缓解预报的模糊效应。

“回归到平均”是指指的是在一系列随机变量中，一个极端的观测值往往会被一个更接近平均值的观测值所跟随。如果某个事件是极端气象事件，使用数据驱动的方法进行预测时会趋近于生成接近平均值的预报，这样无法预测极端气象事件的发生，造成了预测偏差。此外，极端气象事件发生之后，数据驱动模型也会更趋近于生成接近平均值的预报，无法预料到极端气象事件之后的发展。为了解决以上两个问题，首先，采用极值理论对极值的分布进行拟合，得到极值发生的统计规律。随后，根据极端事件发生的规律，预测极端事件的发生并定量计算极端现象所带来的干预影响。最后，在问题一结构因果模型的基础上，融合极值理论与动态干预影响卡尔曼滤波，提出了一种基于极值理论与干预影响动态建模的预报模糊效应缓解方法，主要流程如图11。

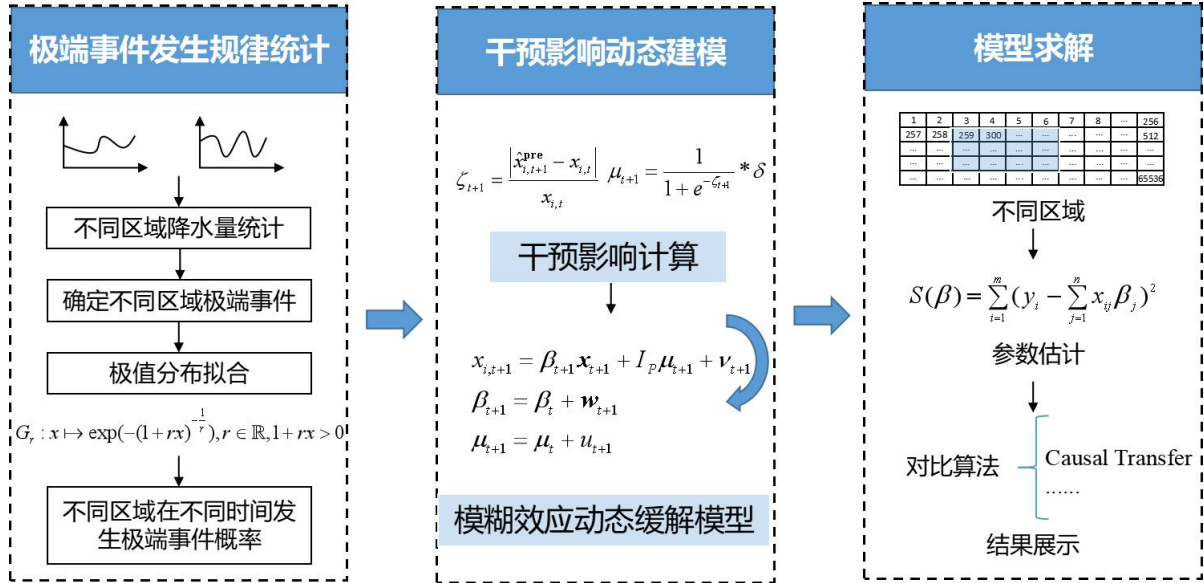


图 11 问题 2 求解思路

(1) **极端事件发生规律统计**：对不同区域的降水量进行统计，并使用极值理论对极端事件发生的规律进行统计，进行极值分布进行拟合，最后确定不同区域在不同时间发生极端事件的概率与极值。

(2) **干预影响动态建模**：根据极值发生的规律，预测极端事件的发生。将极端现象抽象为干预，定量计算极端现象带来的干预影响。在问题一模型的基础上，融入动态干预影响的卡尔曼滤波，对极端事件所带来的影响进行动态建模，并在后续预测中继续考虑这种影响，从而缓解“回归到平均”问题。

(3) **模型求解**：以不同区域为背景，通过极值理论判断当前是否为极值事件。如果为极值事件，采用 POT 方法得到极值作为预测值。反之，采用动态模型进行求解，并与典型的 Causal Transfer（CF）算法进行对比，得出相应结论。

3.2 极端事件发生规律统计

3.2.1 极值理论

极值理论的目标是找到极端事件的规律（例如日最高温度规律，或月最大潮高规律）。Fisher、Tippett 和后来的 Gnedenko 的结果表明，在弱条件下，这些极端事件具有相同的分布，无论原始事件是什么。例如，温度或潮汐高度的最大值或多或少具有相同的分布，而温度和潮汐高度的分布则不太可能相同。这种极端规律称为极值分布(EVD)，它们具有以下形式：

$$G_r : x \mapsto \exp(-(1+rx)^{-1/r}), r \in \mathbb{R}, 1+rx > 0 \quad (22)$$

普通标准分布的所有极值都遵循这样的分布，极值指数 r 取决于事件的原始分布。例如， $X_1, X_2, \dots, X_n$ 是 n 个独立同分布的变量（比如服从高斯分布 $N(0,1)$ ）， $M_n = \max_{1 \leq i \leq n} X_i$ 服从极值分布，极值指数 r 取决于它的原始分布（对于高斯分布 $r=0$ ）。

通过将极值分布拟合到未知的输入分布尾部，就可以评估潜在极端事件的概率。即，从给定的概率 q 可以计算出 z_q ，使得 $P(X > z_q) < q$ ，为了解决这个问题，我们需要估计 r 。存在几种估计方法，例如 Hill 估计和 Pichhands 估计，但它们只对特定的分布有效。近年来，随着大数据的发展，Peaks-Over-Threshold(POT)方法在求解极值分布发挥了重要作用。本节基于 POT 理论，对不同区域降水量的极值分布进行拟合。

3.2.2 区域降水量分析

为了对不同区域在不同时间的降水量进行分析，本节选择了 8 个地区（同第 2 章，其他地区降水量将以附件形式上传，名称为 sum_rain.mat）。根据图 12 可知，不同区域在不同时间的降水量具有显著差异。如区域 1、4、5。在区域 1 中，降水量大多集中在[0,75]和[180,250]区间，其他时间降水量较少。在区域 4 中，降水大多集中在[40,60]和[230,235]区间，其他时间降水量极低。区域 5 与区域 4 的降水量趋势几乎是相同的。这点从两个区域坐标也可以看出，区域 3 坐标为[128, 125]，区域 4 坐标为[130, 126]，两个地区的区域半径仅为“2”。这也证明了本文在第二章提出的“空间关联性”是存在的，且当区域半径较小时，这种“空间关联性”会很显著。

需要说明的是，其他地区在不同时间的降水量也有区别，但这种区别用肉眼很难区分，因此难以通过人工实现不同区域极端降水事件的预测。由于 POT 在求解极值分布上做出的极大贡献，本文基于 POT 理论，对不同区域的极值分布（极端降水量）进行求解，进而为极端气象事件预测提供基础。

3.2.3 基于 POT 的极端降水事件分布拟合

POT 方法的基本步骤是假设有 n 个观测值 $X_1, \dots, X_n$ ，并且已经确定了风险 q ，目标是计算验证 $P(X > z_q) < q$ 的第一个阈值 z_q 。这个想法是设置一个高阈值 t （例如，实际上是一个高经验分位数），检索峰值（超过 t ）并拟合 GPD（广义帕累托分布）到它们。这样就可以推断出极值的分布。因此，本节构建的基于极值理论与干预影响动态建模的模糊效应缓解主要思路如下：

Step1: 基于极值理论，判断当前预测事件是否为极端降水事件；

Step2: 如果预测事件是降水事件，基于 POT 求得极值，并将其作为预测值；

Step3: 采用模糊效应动态缓解模型，以提高预测精度；

根据上面步骤，可以有效缓解无法预测极端气象事件发生的问题。然而，当极端气象事件发生之后，后面的预测值仍会收到该“极端事件”的影响。为了解决无法准确预测

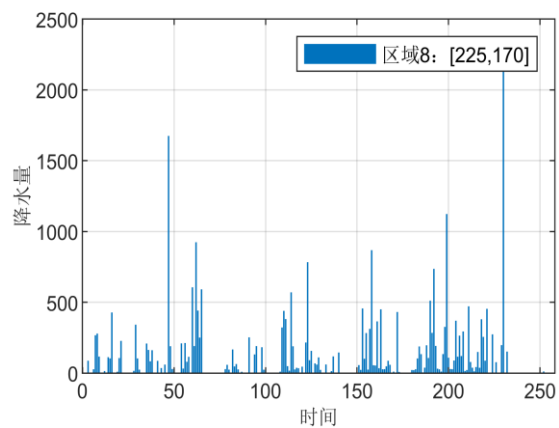
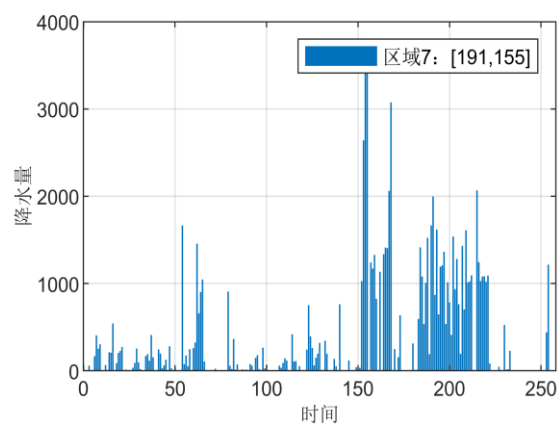
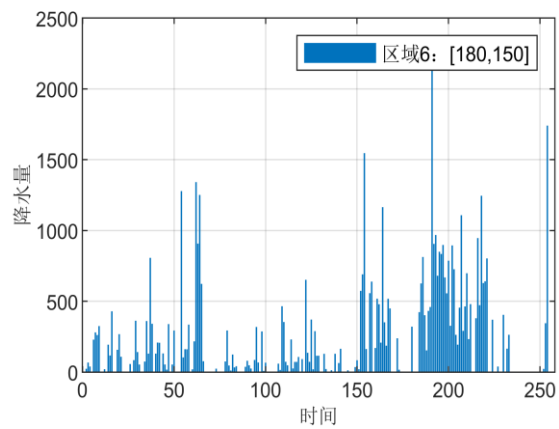
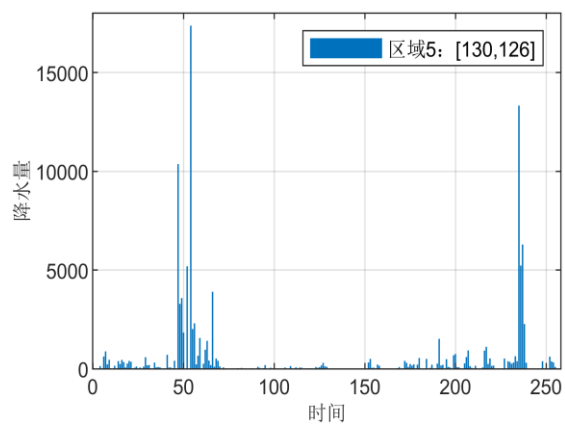
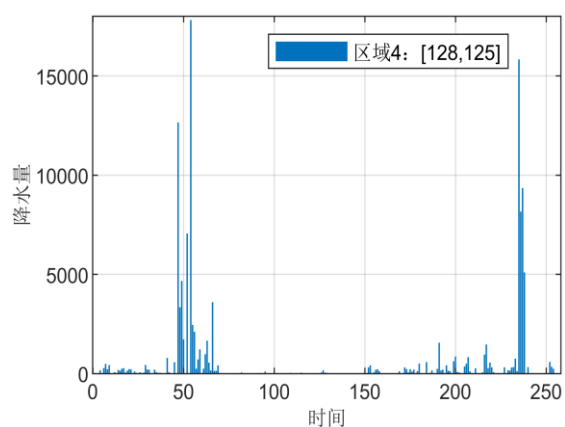
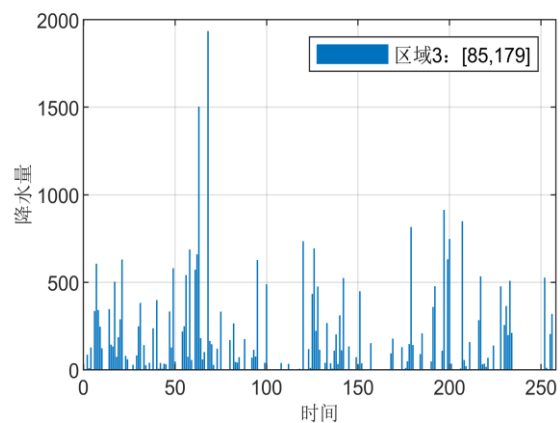
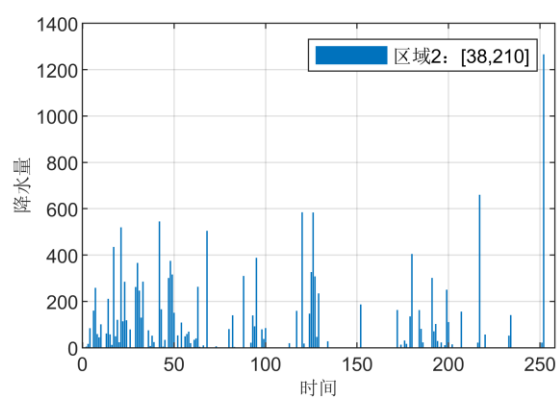
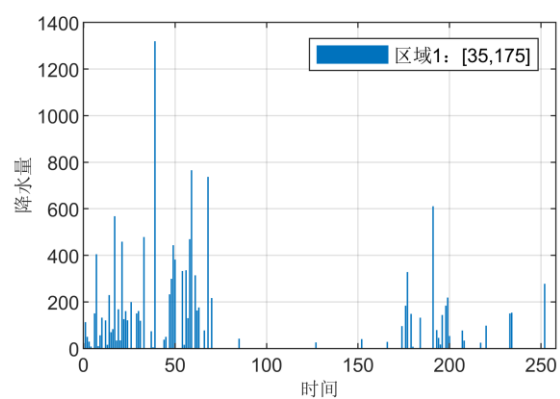


图 12 不同区域降水量分析

到极端气象事件之后数据的问题，本文第2章结构因果模型的基础上，将极端气象事件抽象成干预，并对极端气象事件所带来的影响进行动态建模，将这种影响融入结构因果模型中，得到了其随时间动态变化的状态空间模型，进而解决干预后的预测问题。

需要说明的是，极值理论与模糊效应动态缓解模型是相互影响的，即在预测极端事件后的降水数据时，仍需要利用 POT 理论确定极端降水阈值。当降水被分为极端降水事件之后，还需要采用动态模型修正。

3.3 干预影响动态模型

3.3.1 干预影响计算

干预可以分为硬干预与软干预，硬干预是指对个体施加的一个实验（政策、活动、环境变异）^[15]。软干预是指数据分布的变化、而这种数据分布的变化具体体现在数据规律的改变。因此，极端事件的发生可以抽象为软干预。为了解决“回归到平均（Regression to the mean）”中无法准确预测到极端气象时间之后数据的问题，本节提出了干预影响动态模型。主要思路为：计算极端事件所带来的初始影响，对这种影响进行动态建模，并将这种影响考虑到预测模型中，从而缓解预报的模糊效应问题。

假设干预发生在时间 t ，使用最小二乘法获得 $t+1$ 时刻的 β_{t+1} ，初始干预效果如下式所示：

$$\zeta_{t+1} = \frac{|\hat{x}_{i,t+1}^{\text{pre}} - x_{i,t}|}{x_{i,t}} \quad (23)$$

其中， $\hat{x}_{i,t+1}^{\text{pre}}$ 是在没有干预的情况下 $t+1$ 时刻的预测值， $x_{i,t}$ 是 t 时刻的实际值。为了使干预效果标准化，提出了标准化干预效果 μ_{t+1}

$$\mu_{t+1} = \frac{1}{1 + e^{-\zeta_{t+1}}} * \delta \quad (24)$$

其中， δ 是干预措施的实施程度。

3.3.2 融入动态干预影响的卡尔曼滤波

当软干预发生时，干预的效果将在干预后继续演变，并且可能随着时间的推移而增加或减少，具有时序依赖性^[16]。本节在问题一提出的预测模型的基础上提出了一个新的状态空间模型。该模型可以对干预效果的时间动态进行建模，在干预后进行精准预测，缓解预报的模糊效应。

干预可以导致因果系数的变化，并对参数产生动态影响。为了对这种变化进行建模，提出了一个状态空间模型，如下式所示。

$$\begin{aligned} x_{i,t+1} &= \beta_{t+1} \mathbf{x}_{t+1} + I_p \mu_{t+1} + v_{t+1} \\ \beta_{t+1} &= \beta_t + \mathbf{w}_{t+1} \\ \mu_{t+1} &= \mu_t + u_{t+1} \end{aligned} \quad (25)$$

在上式中， $\beta_{t+1} = \{\mathbf{C}_t, U_t\}$ ， v_{t+1} ， $\mathbf{w}_{t+1}$ 和 u_{t+1} 是噪声项， I_p 是软干预指标， $I_p = 1$ 代表干预， $I_p = 0$ 代表不干预， μ_t 表示在 t 时刻的干预效果。上式可以表示为：

$$x_{i,t+1} = (\mathbf{X}_{t+1} \quad I_p) \begin{pmatrix} \beta_{t+1} \\ \mu_{t+1} \end{pmatrix} + v_{t+1} = \begin{pmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{pmatrix} \begin{pmatrix} \beta_t \\ \mu_t \end{pmatrix} + \begin{pmatrix} \mathbf{w}_{t+1} \\ u_{t+1} \end{pmatrix} \quad (26)$$

让 $\mathbf{F}_{t+1} = (\mathbf{X}_{t+1} \quad \mathbf{I}_p)$, $\theta_{t+1} = \begin{pmatrix} \beta_{t+1} \\ \mu_{t+1} \end{pmatrix}$, $\mathbf{G}_{t+1} = \begin{pmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{pmatrix}$, $\mathbf{s}_{t+1} = \begin{pmatrix} \mathbf{w}_{t+1} \\ u_{t+1} \end{pmatrix}$ 。然后上式可以表示为:

$$x_{i,t+1} = \mathbf{F}_{t+1} \theta_{t+1} + v_{t+1} \quad (27)$$

$$\theta_{t+1} = \mathbf{G}_{t+1} \theta_t + \mathbf{s}_{t+1} \quad (28)$$

上式是卡尔曼滤波的标准形式。式(27)是观测方程, 式(28)是状态方程。假设干预发生在时间 t , 使用最小二乘法获得 $t+1$ 时刻的 β_{t+1} 。

为了最小化实际值 $x_{i,t}$ 和预测值 $\hat{x}_{i,t+1}^{\text{pre}}$ 之间的误差 e_{t+1} 。对于卡尔曼滤波, 更新方程如下式所示。

$$\theta_{t+1} = \mathbf{G}_{t+1} \theta_t \quad (29)$$

$$\mathbf{P}_{t+1} = \mathbf{G}_{t+1} \mathbf{P}_t \mathbf{G}_{t+1}^T + \mathbf{Q} \quad (30)$$

在上式中, 基于时刻 t 的状态递归地推导出时刻 $t+1$ 的状态。在上式中, 基于来自前一时刻的协方差矩阵 $\mathbf{P}_t$, 在时刻 $t+1$ 递归地获得协方差矩阵 $\mathbf{P}_{t+1}$, $\mathbf{Q}$ 是过程噪声的协方差矩阵, 接下来, 计算卡尔曼增益。

$$\mathbf{K}_{t+1} = \mathbf{P}_{t+1} \mathbf{F}_{t+1}^T (\mathbf{F}_{t+1} \mathbf{P}_{t+1} \mathbf{F}_{t+1}^T + \mathbf{R})^{-1} \quad (31)$$

在上式中, $\mathbf{R}$ 是误差方差矩阵。然后得到状态的后验估计 $\hat{\theta}_{t+1}$ 。

$$\hat{\theta}_{t+1} = \theta_{t+1} + \mathbf{K}_{t+1} e_{t+1} \quad (32)$$

计算后验协方差矩阵

$$\hat{\mathbf{P}}_{t+1} = (1 - \mathbf{K}_{t+1} \mathbf{F}_{t+1}) \mathbf{P}_{t+1} \quad (33)$$

然后计算最终的预测值。

$$\hat{x}_{i,t+1} = \mathbf{F}_{t+1} \hat{\theta}_{t+1} \quad (34)$$

递归地重复上述过程, 以估计干预的效果。卡尔曼滤波方法是在递归过程中不断更新测量信息, 并使用更新的测量数据来校正状态估计。对于多步预测, 使用预测值更新状态方程。该方法可以估计干预后的长期效果, 解决“回归到平均 (Regression to the mean)”问题。

3.3.3 基于极值理论与干预影响动态建模的预报模糊效应缓解

结合 3.2 与 3.3, 基于极值理论与干预影响动态建模的预报模糊效应缓解方法总结如下:

使用极值理论来拟合极端气象事件的分布, 得到极端气象事件的分布, 从而可以计算出发生极端气象事件的概率与极端事件值。当极端气象事件发生的概率较大时, 基于 POT 计算极值作为预测值。由于该极端气象事件可能会对后续值产生动态影响, 为了对这一影响进行动态建模, 将第二章构建的 SCM 模型融入动态干预影响的卡尔曼滤波, 该模型可以对极端事件所带来的影响进行动态更新, 并把这种影响冗余预报模型中。当极端气象事件发生的概率较小时, 按照 SCM 模型正常预测即可。

3.4 模型求解

3.4.1 数据选择

为了验证本章所提出的方法是否能够真正的缓解预报模糊效应，选取了极端气象事件进行预测（即通过极值分布极端事件发生概率超过 10%的降水事件），分别进行了极端气象 Z_H 预测与极端气象之后的 Z_H 预测两种实验。与本节相关代码及数据见附录（第三章）。

3.4.2 实验设置

在实验过程中，80%的数据为训练数据，20%的数据为测试数据，对 20%测试数据中的极端气象事件的 Z_H 值与极端气象事件之后 Z_H 值进行预报（10 帧）。

3.4.3 对比算法

在预测极端事件发生的概率与极端事件的取值时，将此方法与第二章中提出的 SCM 模型进行了比较。在预测极端事件发生后 Z_H 值时，将本章提出的方法与 CF 方法进行了比较，CF 方法是用来评估干预影响的典型方法^[18]。

3.5 结果展示与分析

3.5.1 极端气象事件 Z_H 预报结果

根据 POT 对区域的极端事件进行求解，使用 MSE、MAE 来衡量 POT 方法的有效性，MSE、MAE 的计算方法如式（20）、式（21）。将 POT 方法与第一章提出的 SCM 进行了比较，比较结果如表 2 所示。

表 2 不同算法下极端气象事件的 Z_H 预报结果

等高面/km	方法	MAE	MSE
1	POT	0.315502	0.177794
	SCM	0.389027	0.293495
3	POT	0.264498	0.126525
	SCM	0.328351	0.209729
7	POT	0.212427	0.078786
	SCM	0.251788	0.111931

表 2 为不同算法在极端气象事件的 Z_H 预报结果，根据表 2 可知，POT 算法在预测极端事件的 Z_H 时相比较第一章提出的 SCM 有较大的优越性，表现在其均方误差和最小平均误差都比 SCM 小。此外，根据表 2，我们看到一个有意思的现象，随着等高面的增加，POT 算法和 SCM 算法在预报 Z_H 时其 MAE 和 MSE 均有所降低，我们认为，这可能是不同等高面的数据质量导致的（或者当等高面为 7km 时，其数据更能反应极端事件的发生情况？），当等高面为 7km 时，极端事件的 Z_H 预测效果最好。

需要说明的是，上述结果是将所有地区的极端气象事件的 Z_H 值取其均值，在实验中我们发现，有的地区 POT 预测结果并不如 SCM（经过分析之后发现这类区域属于降水相对比较规律的区域）。但是在绝大多数区域中，POT 能够更加准确的预测出极端事件的 Z_H 值（降水并不规律）。因此，POT 方法不是在每个区域都有效的，当区域降水比较规律时，SCM 的预测效果会更好。

3.5.2 极端气象事件 1 小时后 Z_H 预报

“回归到平均”问题可以分为两个子问题。其一是通过过去的历史规律无法预测当前的极端事件，其二是当产生极端事件之后，后续预报会受当前极端事件的影响。3.5.1 节证

明了 POT 算法再预测极端事件的优越性，然而，当发生极端事件之后，仅采用第二章提出的 SCM 模型很难获得极端事件之后的数值，因此，本文将极端事件抽象为干预，提出了干预影响动态建模的预报模糊效应缓解方法。

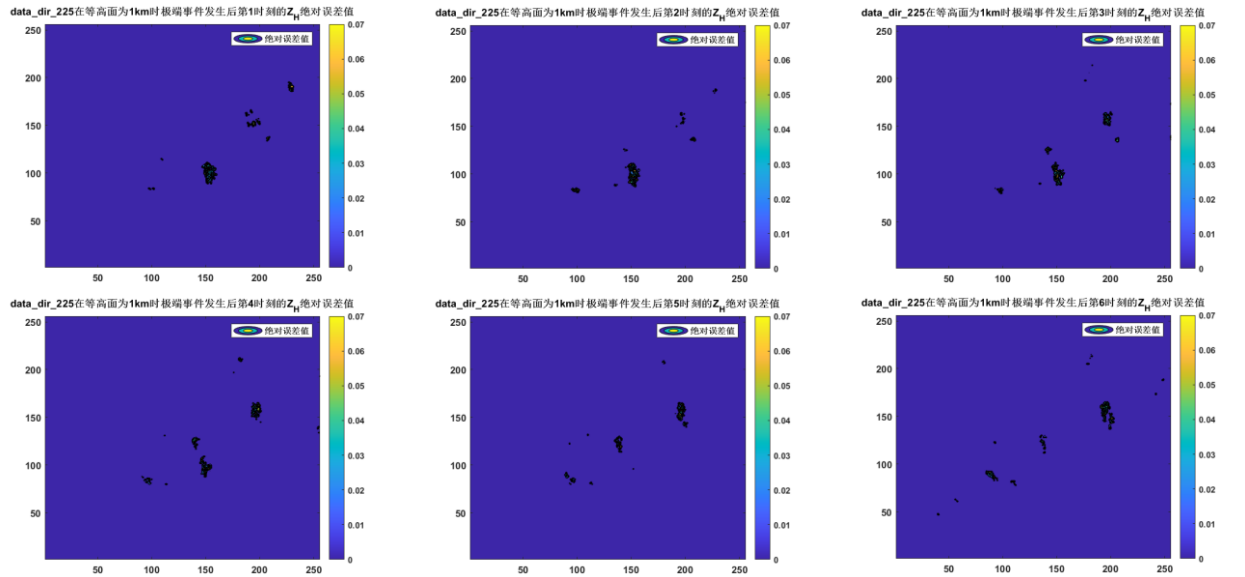
为了验证干预影响动态建模的预报模糊效应缓解方法，以 data_dir_225 这一降水事件为例，根据 CF 和本文提出的方法分别对其极端事件之后的 Z_H 进行预报，最终效果的 MSE 和 MAE 如表 3 所示。

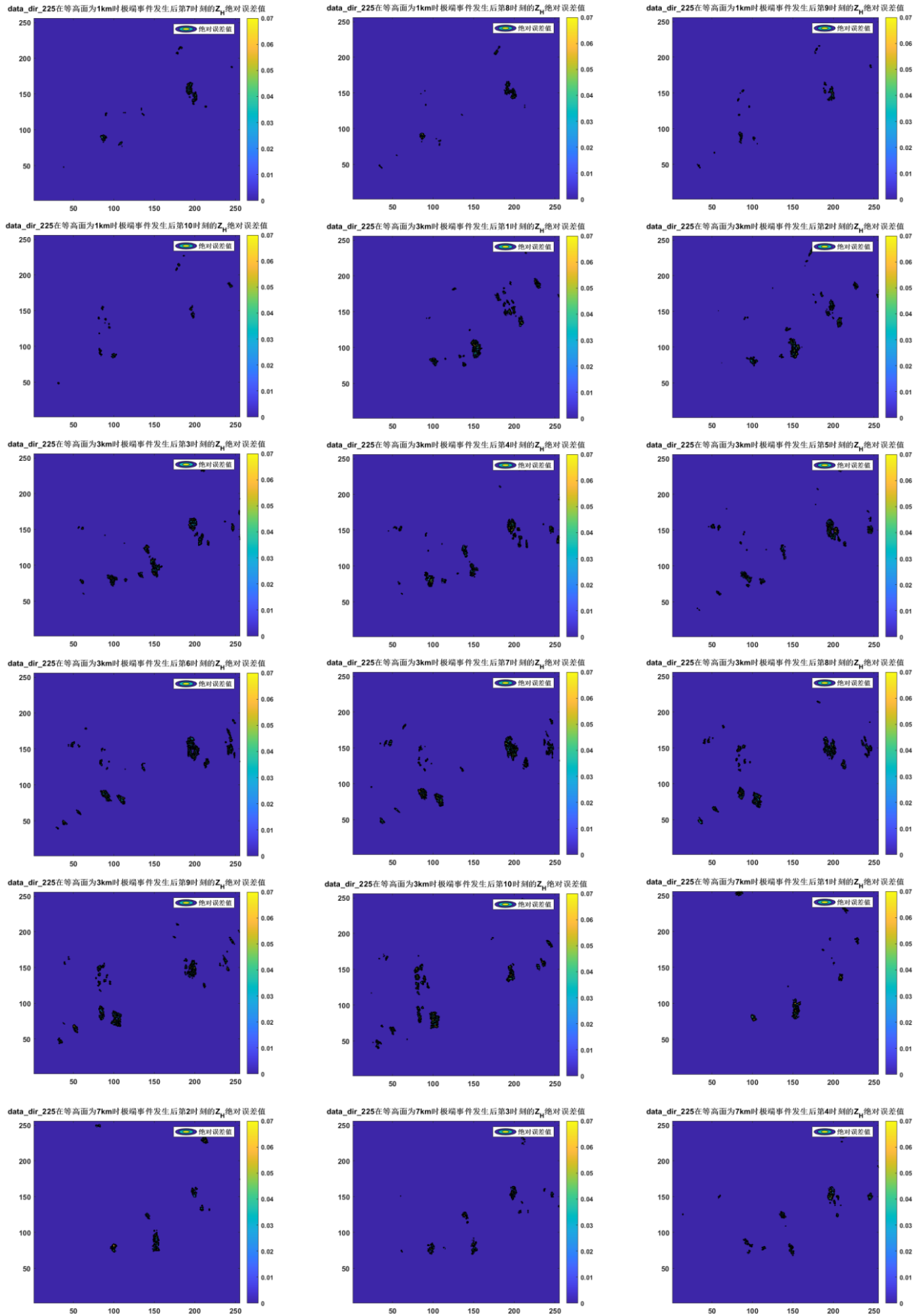
表 3 不同算法下极端气象事件之后一小时的 Z_H 预报结果			
等高面/km	方法	MAE	MSE
1	本文方法	0.008982	0.001306
	CF	0.013202	0.001652
3	本文方法	0.008758	0.001353
	CF	0.015569	0.001953
7	本文方法	0.020687	0.004981
	CF	0.016311	0.019109

表 3 为不同算法下极端气象事件之后一小时的 Z_H 预报结果。根据表 3 可知，在预测极端事件之后的 Z_H 时，本文提出的干预影响动态建模的预报模糊效应缓解方法全面优于 CF 上（仅在 7km 的 MAE 上，本文方法的精度低于 CF 方法），这体现在其 MAE 和 MSE 数值均远低于 CF 预测得到的数值。其 MAE 和 MSE 降低了约 14.76%和 66.37%。因此，本文提出的干预影响动态建模的预报模糊效应缓解方法能够对极端事件之后的 Z_H 进行有效预测。

此外，当等高面为 1km 时，本文提出的方法相比较 CF 方法在 MAE 和 MSE 方面分别提升了约 31.96%和 20.94%。当等高面为 3km 时，本文提出的方法相比较 CF 方法在 MAE 和 MSE 方面分别提升了约 43.75%30.72%。当登高面为 7km 时，本文提出的方法相比较 CF 方法在 MAE 和 MSE 方面分别提升了约-26.83%和 73.93%。3.5.1 表明，当等高面为 7km 时，其极端事件的 Z_H 预测效果会好，然而在本节实验中，当等高面的高度为 7km 时，其极端事件之后的预测效果反而会差。这表明，本文提出的方法只能够对“回归到平均”这类问题进行缓解，但是并不能完全消除这种影响。

图 13 展示了等高面为 1km、3km 和 7km 时，在发生极端事件后，未来十个时间段内的 Z_H 预报绝对误差值（以本文提出的算法作为预报）。





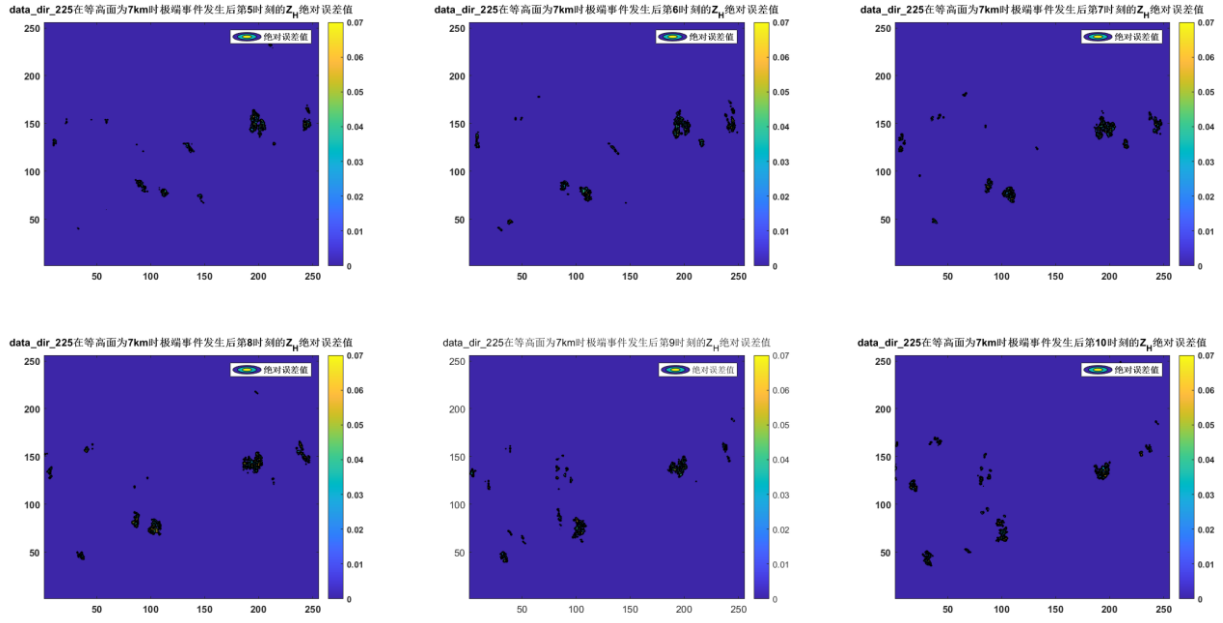


图 13 data_dir_225 极端事件后不同等高面下未来十个时间段内的 Z_H 预报绝对误差值

根据图 13 可知，不同区域的预测精度差异较大。有些地区预测精度较高，而有些地区预测精度较低。但是，对比图 10 可以看到，图 10 中预测偏差在 0.2，而本节中预测偏差为 0.07（不规律降水）。因此，本节设计的预报模糊效应缓解方法能够有效缓解“回归到平均”导致的预报模糊的问题。但是，从图 13 和图 10 对比也能看到，本节提出的方法仅能够对“回归到平均”效应进行缓解，并不能消除。

3.6 本章小结

本章主要研究了基于极值理论与干预影响动态建模的预报模糊效应缓解。结合本文研究的问题，将“回归到平均”问题分解为两个子问题。一是预测极端气象事件的 Z_H 值，另一个是预测极端事件发生之后的 Z_H 值。针对第一个子问题，本文采用极值理论对极值的分布进行拟合，得到极值发生的统计规律。随后，根据极端事件发生的规律，预测极端事件的发生定量计算极端现象所带来的干预影响。最后，在问题一结构因果模型的基础上，融合极值理论与动态干预影响卡尔曼滤波，提出了一种基于极值理论与干预影响动态建模的预报模糊效应缓解方法。主要得到以下结论：**1. 当降水不规律时，POT 方法能够有效预测到极端事件。当地区降水规律性增加时，POT 性能会下降，甚至不如 SCM，这也间接说明了 SCM 能够在稳态数据中保持良好的性能。****2. 干预影响动态建模的预报模型能够对极端事件之后的 Z_H 值进行有效预测，但是这各方法只能够对“回归到平均”的问题进行缓解，并不能完全消除这类影响。**

第 4 章 基于时空因果图神经网络的定量降水估计

4.1 问题三分析

问题三要求利用题目提供的 Z_H 、 Z_{DR} 和降水量数据，设计适当的数学模型，利用 Z_H 及 Z_{DR} 进行定量降水估计。

由于要估计的自身区域及相关区域的 Z_H 、 Z_{DR} 和降水量数据蕴含着许多信息，这种信息不仅包括时序因果性、还存在空间关联性。为了对这种时序因果性与空间关联性建模，获得可靠的定量降水估计结果，本节在时空图神经网络的基础上，提出了一种基于时空因果图神经网络的定量降水估计方法，主要流程如图 14 所示：

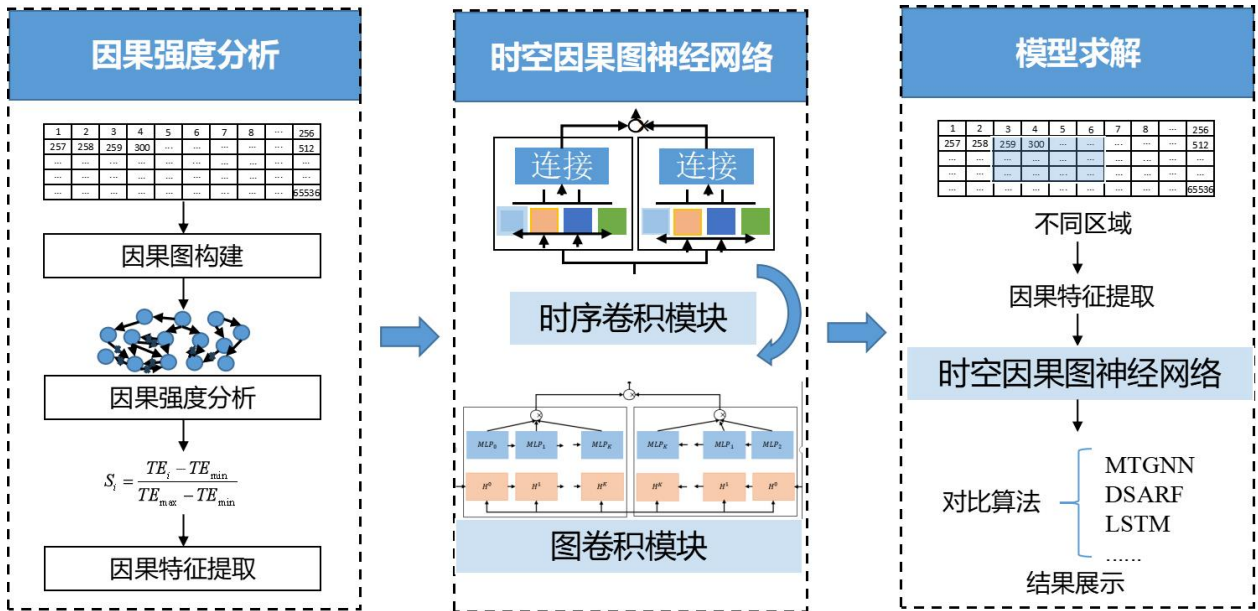


图 14 问题 3 求解思路

(1) **因果强度分析：**构造不同区域 Z_H 、 Z_{DR} 和降水量数据的因果图，并计算出因果强度；

(2) **时空因果图神经网络：**结合因果图与因果强度，使用因果图卷积与时序卷积分别对空间因果特征与时序因果特征进行提取，构建时空因果图神经网络；

(3) **模型求解：**针对不同区域，使用构建的时空因果图神经网络对降水量进行估计，并与传统的图神经网络（MTGNN）、Deep Switching Auto-Regression Factorization (BSARF)、长短记忆神经网络（LSTM）等基线算法进行对比，得出相应结论。

4.2 Z_H 与 Z_{DR} 因果强度分析

由于因果图与因果强度作为时空因果图神经网络的输入，是因果特征提取的前提。本节首先构造自身区域与相关区域的 Z_H 、 Z_{DR} 、降水量之间的因果图，然后计算因果关系之间的因果强度。

在因果图的构建方面，使用第 2 章中提出的多元传递熵方法找到不同变量之间的因果关系，进而构造因果图。在因果强度计算方面，设各个变量之间计算得到的传递熵值为 $[TE_1, \dots, TE_n]$ ， n 为传递熵的个数，最大的传递熵值为 $TE_{\max}$ ，最小的传递熵值为 $TE_{\min}$ ，使

用最大最小归一化得到因果变量之间的因果强度。如式（35）所示：

$$S_i = \frac{TE_i - TE_{\min}}{TE_{\max} - TE_{\min}} \quad (35)$$

其中， S_i 表示变量之间的因果强度。

4.3 定量降水估计模型

基于时空因果图神经网络的定量降水估计模型一共包括因果图结构提取、图卷积、时序卷积、输出卷积四个模块，具体结构如图 15 所示。

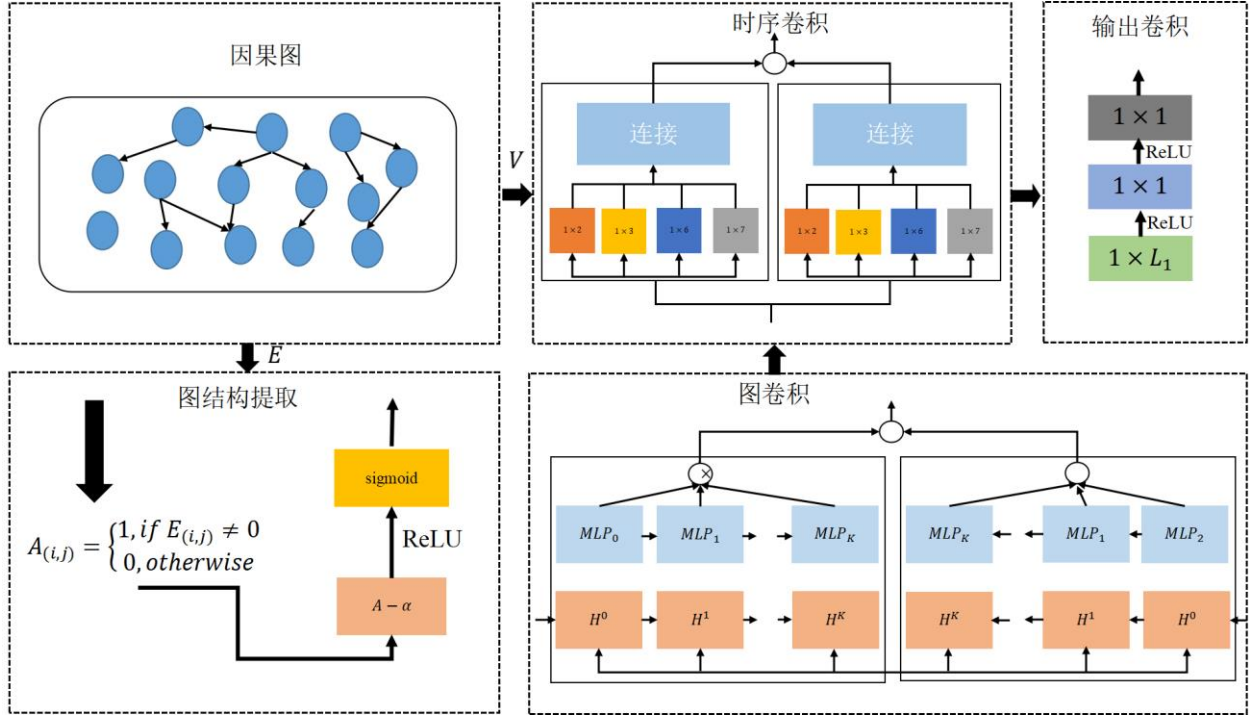


图 15 基于时空因果图神经网络的定量降水估计模型

4.3.1 因果图结构提取

图结构提取模块：根据 4.2 中学习到的参数之间的因果图与因果强度，参数之间的因果关系反映在顶点属性及其连接关系中，为了使用时空序列图神经网络的学习模式，图结构提取模块用于学习图的连接关系。

首先，根据式（36）从将输入图的边关系 E 变换为顶点的邻接矩阵 A 。然后，对输入图集 G 的邻接矩阵 A 进行计数，以获得其平均统计邻接矩阵 A_v ，如式（36）所示。

$$A_{(i,j)} = \begin{cases} 1, & \text{if } E_{(i,j)} \neq 0 \\ 0, & \text{otherwise} \end{cases} \quad (36)$$

$$A_v = \text{softmax}(\text{ReLU}(A - \alpha)) \quad (37)$$

其中 α 是阈值系数，当顶点之间的连接关系值（因果强度）超过阈值系数时，该连接关系被认为是有效的。否则，视为无效。该算法是一种稀疏邻接矩阵并降低后续图卷积计算成本的策略。

4.3.2 时序因果性-时序卷积

图信息的时间因果性特征主要由一组具有扩展一维卷积滤波器的时间模块提取。它主

要包括两个扩张的起始层。一个是用于滤波的正切双曲激活函数，另一个是作为控制信息传输的门控模块的 sigmoid 函数。

为了确保该模块能够应用于不同的时间模式，膨胀初始层采用了多种滤波器尺寸的模式，例如 1×2 、 1×3 、 1×6 、 1×7 。通过组合滤波器大小的不同模块，可以发现隐藏在长期信号模式中的规律和特性。相反，短期信号模式可以同时被精确地表示。扩张的起始层可以由公式 (38) 表示。四个滤波器的输出将根据最大滤波器进行截断，并跨信道维度进行级联。扩张卷积由公式 (39) 表示，其中 d 是扩张系数

$$z = \text{concat}(\text{conv}(z, f_{1 \times 2}), \text{conv}(z, f_{1 \times 3}), \text{conv}(z, f_{1 \times 6}), \text{conv}(z, f_{1 \times 7})) \quad (38)$$

$$\text{conv}(z, f_{1 \times k}(t)) = \sum_{s=0}^{k-1} f_{1 \times k}(t) z(t - d \times s) \quad (39)$$

4.3.3 空间关联性-图卷积

图卷积模块主要用于处理图信息中的空间因果性，该模块通过两个混合跳传播层处理每个节点的流入和流出信息，可以将节点信息与相邻节点信息融合。混合跳传播层包括两个步骤：1) 信息传播步骤和 2) 信息选择步骤。

这两个步骤可以分别由式 (38) 和式 (39) 表示，其中， β 用于控制保持根节点原始状态的比率， K 是传播深度， H_{in} 是前一层中输出的输入隐藏状态， H_{out} 是当前层 $H^0 = H_{in}$ 的输出隐藏状态。

$$H^k = \beta H_{in} + (1 - \beta) \tilde{A} H^{k-1} \quad (40)$$

$$H_{out} = \sum_{i=0}^K H^i W^k \quad (41)$$

其中 $\tilde{A} = \tilde{D}^{-1}(A + I)$ ， $\tilde{D} = 1 + \sum_j A_{ij}$ 。

跳跃连接层：跳跃连接层实际上是 $1 \times L_i$ 标准卷积层，其中 $1 \times L_i$ 是第 i 个跳跃连接层的输入长度。它的主要功能是标准化跳转到输出模块的信息，使它们具有相同的输出序列长度。

输出模块：包括两个 1×1 标准卷积模块，将最终结果转换为了所需的输出维度序列，即 $256\text{km} \times 256\text{km}$ 的平面区域。

4.4 模型求解

4.4.1 数据选择

为了评估基于时空因果图神经网络在定量降水估计方面的效果，选取了 208 次降水事件作为训练集，52 次降水事件作为测试集，使用 MAE 和 MSE 与估计效果进行评估。相关代码与数据见附件。

4.4.2 对比算法

为了验证本文方法的有效性，将本文提出的方法与传统图神经网络（Multivariate Time Series Forecasting with Graph Neural Networks, MTGNN）^[20]、DSARF（Deep Switching Auto-Regression Factorization）^[21]、LSTM 方法进行比较。

4.5 结果展示与分析

4.5.1 方法有效性分析

以 Z_H 和 Z_{DR} 为网络输入，降水量为输出。根据 MTGNN、DSARF、LSTM 和本文提出的方法分别对降水量进行估计，得到的估计结果的 MSE 和 MAE 如表 4 所示。

表 4 不同算法下降水量估计结果

方法	MAE	MSE
本文方法	0.542312	18.321134
MTGNN	0.552417	18.347673
DSARF	0.730806	32.798002
LSTM	0.982949	14.624969

表 4 为采用不同算法对降水量估计的 MAE 和 MSE。根据表 5 可知，本文提出的方法在求解具有时序因果性与空间因果性的降水量问题具有优越性，主要体现在其 MAE 和 MSE 相比较其他方法均有所降低。此外，我们还发现，时空神经网络对求解这类带有时空关系降水量估计的问题是有有效的，DSARF 和 LSTM 表现性能较差。而在本文引入因果关系后，图神经网络的模型精度会有明显提升，MAE 和 MSE 提升了约 1.83%和 0.15%。基于上述分析，可以得到结论如下：1. 时空图神经网络在求解例如降水这种带有复杂时空约束的问题具有一定优越性，相比较传统的 DSARF、LSTM 算法，其性能更加显著。2. 将时间序列因果信息与空间因果信息引入到图神经网络能够提高其模型精度。

4.5.2 不同区域降水量估计值

为了清晰描绘采用本文提出方法下，不同区域的降水量估计效果，绘制了其估计值与误差值（估计值-实际值=误差值）。受篇幅限制，本节仅分析一类事件的结果，如图 16 所示。

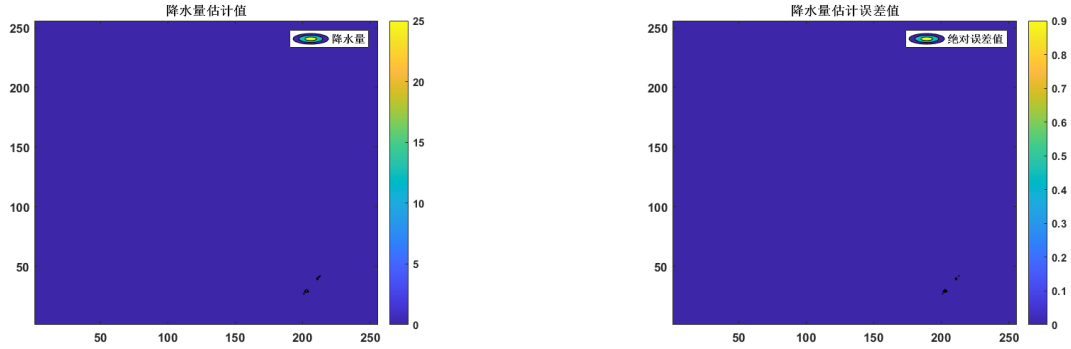


图 16 基于时空因果图神经网络的定量降水估计结果

从图 16 可以看出，本文提出的基于时空因果图神经网络的定量降水方法误差较小，具有较高的准确性。同时从图中可以看出，降水的区域面积较小，这是由于在同一时刻，大部分地区都在降水的可能性较小导致的。

4.6 本章小结

本节研究了基于时空因果图神经网络的定量降水估计问题。首先，构造 Z_H 、 Z_{DR} 和降水量数据的因果图，并计算出因果强度；然后，使用因果图卷积与时序卷积对空间因果特征与时序因果特征进行提取，构建时空因果图神经网络网络结构。最后，训练网络结构，得到定量降水估计结果。本章主要结论如下：1. 时空图神经网络在求解例如降水这种带有复杂时空约束的问题具有一定优越性，相比较传统的 DSARF、LSTM 算法，其性能更加显著。2. 将时间序列因果信息引入到图神经网络能够提高模型准确度，在本文讨论的降水估计问题中，MAE 提升了约 1.83%。

第5章 基于反事实推理的数据融合策略优化

5.1 问题四分析

问题四要求设计数学模型来评估双偏振雷达资料在强对流降水临近预报中的贡献，并优化数据融合策略，以便更好地应对突发性和局地性强的强对流天气。这主要涉及两个问题，一是建立合适的数学模型来评估双偏振雷达资料在临近预报中的贡献。二是优化数据融合策略，以更好地应对突发性和局地性强的强对流天气。在求解本节问题之前，需要对本节涉及的假设进行说明。

(1) 题目中提供了双偏振雷达的水平反射率因子、差分反射率、比差分相移等数据。本文假设采用单偏振雷达，在同一时空维度观测相同目标获得的水平反射率因子与双偏振雷达的数据是相同的。

(2) 数据融合策略优化涉及范围比较广，包括同一设备的多源信息融合，也包括不同设备之间的多源信息融合。本节仅研究在同一双偏振雷达中，如何对获得的水平反射因子与差分反射率两类数据融和优化的问题。不考虑单偏振雷达与双偏振雷达之间的数据优化，即：属于同一设备的多源信息融合。

基于上述假设，本章提出了基于反事实推理的数据融合策略优化方法，主要流程如图17所示。

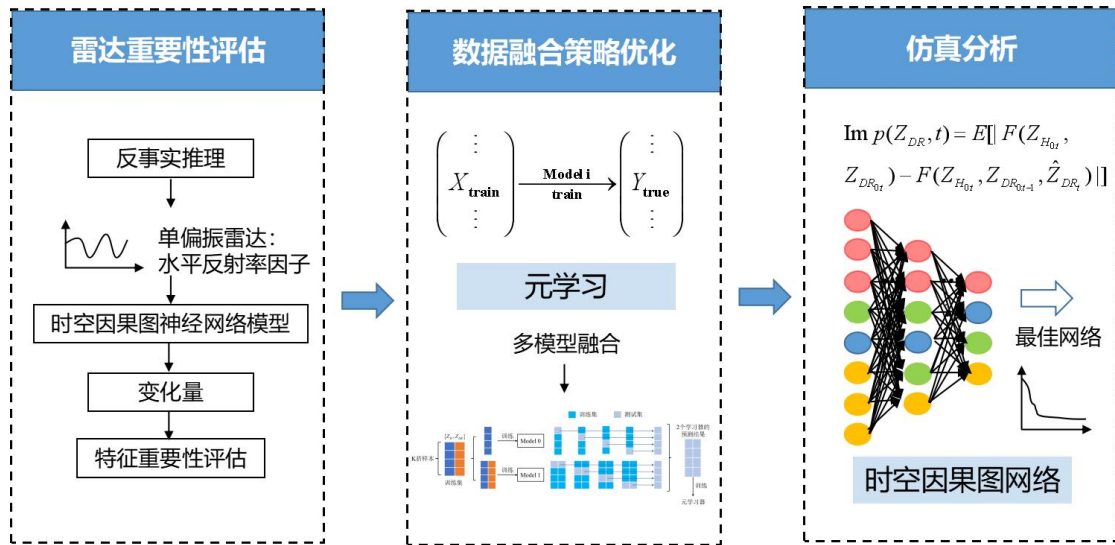


图17 问题4求解思路

(1) **双偏振雷达重要性评估**: 在第三节训练好的时空因果图神经网络模型基础上，将双偏振雷达数据的重要性定义为将输入数据变为传统雷达数据所引起的模型输出的变化，从而解释双偏振雷达重要性评估问题；

数据融合策略优化: 以第四章的模型为基本框架，通过输入不同组合的雷达观测数据 ($\{Z_H\}$, $\{Z_H, Z_{DR}\}$) 来训练模型预报降水量。然后，以不同模型的预报值作为输入，训练一个元学习器来输出不同模型预报值的集成结果，目的是使得集成输出误差变低。当输出误差达到最小时，理论上此时元学习模型代表最优融合策略；

(2) **仿真分析**: 采用反事实推理方法评估双偏振雷达在强对流降水临近预报中的贡献，并构建集成模型对最优策略（模型精度最优）进行确定。

5.2 基于反事实推理的双偏振雷达重要性评估

反事实推理（Counterfactual reasoning）不是基于实际发生的事情，而是基于一个假设的、与现实相反的情境。通过反事实推理，人们可以更好地理解现实中的因果关系、做出相关评估。在本节中，提出了一种基于反事实推理双偏振雷达资料重要性评估方法，将双偏振雷达资料的资料性的重要性定义为将其替换为传统雷达资料所引起的模型输出的变化，从而来解释双偏振雷达资料在强对流降水临近预报中的贡献，本文使用的预测模型为第四章中用于定量降水估计的时空因果图神经网络模型，如果双偏振雷达对模型输出的影响较大，那么它则被认为是重要的。

为了评估 Z_H 、 Z_{DR} 的特征重要性，提出了一种基于反事实的方法。根据观察值 Z_{H_t} （时间 t 的特征 Z_H ）、 Z_{DR_t} （时间 t 的特征 Z_{DR} ）在时间 t 对模型输出（降水量）的影响来赋予其重要性。用反事实 $\hat{Z}_{H_t}$ 或 $\hat{Z}_{DR_t}$ 替换观察值 Z_{H_t} 、 Z_{DR_t} ，以评估这种影响。

为了定量评估这种影响，对每个时间点上的 Z_H 、 Z_{DR} 特征赋予重要性特征，如式(42-43)所示。

$$\text{Im } p(Z_H, t) = E[|F(Z_{H_{0t}}, Z_{DR_{0t}}) - F(Z_{H_{0t-1}}, \hat{Z}_{H_t}, Z_{DR_{0t}})|] \quad (42)$$

$$\text{Im } p(Z_{DR}, t) = E[|F(Z_{H_{0t}}, Z_{DR_{0t}}) - F(Z_{H_{0t}}, Z_{DR_{0t-1}}, \hat{Z}_{DR_t})|] \quad (43)$$

其中 $|\cdot|$ 代表取绝对值， $\hat{Z}_{H_t}$ ， $\hat{Z}_{DR_t}$ 代表反事实样本，反事实样本从传统雷达资料中提取。 F 表示训练好的用于定量降水估计的时空因果图神经网络模型。通过此方法，可以计算双偏振雷达在任意时刻 t 的特征重要性，重要性特征越大，代表该特征越重要，根据此方法来评估双偏振雷达在强对流降水临近预报中的贡献。

5.3 数据融合策略优化

5.3.1 常见的数据融合策略优化方法

数据融合策略优化是指在数据融合过程中，通过合理的策略和方法，将来自不同来源、不同类型的数据进行整合和处理，以提高数据的质量和值。常见的数据融合策略优化方法有以下几种：

(1) 加权平均（Weighted Average）：对多个模型的预测结果进行加权平均，权重可以根据模型的性能进行分配。优点是简单易实现，能够平衡多个模型的贡献，缺点是需要手动设置权重，且对于权重的选择比较主观；

(2) 投票法（Voting）：多个模型的预测结果通过投票决定最终的预测结果，可以采用多数投票、加权投票等方式。优点是简单直观，能够充分利用多个模型的优势，缺点是对权重选择比较主观，且无法处理模型预测结果相差较大的情况。该方法与加权平均法类似；

(3) 堆叠（Stacking）：将多个模型的预测结果作为输入，训练一个元模型来融合这些预测结果。优点是能够学习到模型之间的关联性，能够更好地融合多个模型的优势，缺点是需要额外的训练过程，计算复杂度较高。

(4) 融合模型（Ensemble Model）：使用元学习算法，如随机森林、梯度提升树等，将多个模型组合成一个更强大的模型。优点是能够充分利用多个模型的优势，缺点是需要额外的训练过程，计算复杂度较高。

(5) 样本选择（Sample Selection）：根据模型在不同样本上的性能，选择具有较高性能的样本进行融合。优点是能够充分利用模型在不同样本上的优势，缺点是需要额外的样

本选择过程，可能会引入样本选择偏差。

5.3.2 基于元学习的数据融合优化策略

本节受上述方法（4）启发，采用元学习的方法对数据融合策略进行优化^[22]。主要思路如下：

Step1: 分别使用 $\{Z_H\}$ ， $\{Z_H, Z_{DR}\}$ 作为输入数据对时空因果图神经网络进行训练，分别得到基学习器 Model 0、Model 1，训练过程表示如下：

$$\begin{pmatrix} \vdots \\ X_{\text{train}} \\ \vdots \end{pmatrix} \xrightarrow[\text{train}]{\text{Model } i} \begin{pmatrix} \vdots \\ Y_{\text{true}} \\ \vdots \end{pmatrix} \quad (44)$$

Step2: 为了保证测试的均匀性，将训练集数据划分为 K 部分，利用其中的 K-1 份作为训练，剩余的一份作为测试，最后取平均测试误差作为泛化误差，这样训练集的所有样本都必然会成为训练数据同时也必然有机会成为一次测试集，可以更好地利用训练集数据；

Step3: 训练后的模型 Model i 分别在 X_{train} 和 X_{test} 上预测，得到预测标签分别是 P_i ， T_i ：

$$\begin{pmatrix} \vdots \\ X_{\text{train}} \\ \vdots \end{pmatrix} \xrightarrow[\text{test}]{\text{Model } i} \begin{pmatrix} \vdots \\ P_i \\ \vdots \end{pmatrix} \quad (45)$$

$$\begin{pmatrix} \vdots \\ X_{\text{test}} \\ \vdots \end{pmatrix} \xrightarrow[\text{test}]{\text{Model } i} \begin{pmatrix} \vdots \\ T_i \\ \vdots \end{pmatrix} \quad (46)$$

Step4: 分别把 P_i 以及 T_i 合并，得到一个新的训练集和测试集，新的训练集和测试集都包含了几个基学习器的预测结果。利用上述训练集构建一个新的时空因果图神经网络模型，并用测试集测试模型效果：

$$\begin{pmatrix} \vdots & \dots & \vdots \\ P_1 & \dots & P_n \\ \vdots & \dots & \vdots \end{pmatrix} \xrightarrow[\text{train}]{\text{Model}} \begin{pmatrix} \vdots \\ Y_{\text{true}} \\ \vdots \end{pmatrix} \quad (47)$$

$$\begin{pmatrix} \vdots & \dots & \vdots \\ T_1 & \dots & T_n \\ \vdots & \dots & \vdots \end{pmatrix} \xrightarrow[\text{test}]{\text{Model}} \begin{pmatrix} \vdots \\ Y_{\text{pre}} \\ \vdots \end{pmatrix} \quad (48)$$

最终采用的数据融合模型如图 18 所示。理论上，当模型预测精度最高时，其对应的策略则为最优融合策略。

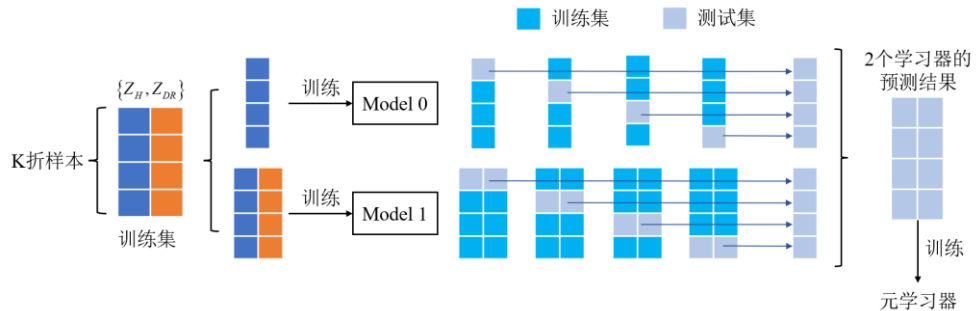


图 18 数据融合策略模型

5.4 模型求解与分析

5.4.1 数据选择

选取了 208 次降水事件作为训练集，52 次降水事件作为测试集，使用 MAE 和 MSE 与估计效果进行评估。相关代码与数据见附件。

5.4.2 双偏振雷达资料雷达重要性评估

为了评估双偏振资料雷达的重要性，提出了一种基于反事实推理的特征重要性评估方法，并进行实验验证。在实验中，依旧使用第四章提出的时空因果图神经网络作为估计模型。首先为了评估 Z_H 的特征重要性，在测试集中，将 $\{Z_H, Z_{DR}\}$ 替换为反事实 $\{Z_{DR}\}$ ，从而计算模型输出的变化。根据式(42)计算重要性特征，得到 Z_H 的重要性特征分数为 0.01606；随后为了评估 Z_{DR} 的特征重要性。在测试集中，将 $\{Z_H, Z_{DR}\}$ 替换为反事实 $\{Z_H\}$ ，根据式(43)计算特征重要性分数，得到 Z_{DR} 的特征重要性为 0.00948。以上结果说明， Z_H 和 Z_{DR} 在强对流降水临近预报中都起到了重要作用，但是相对来说， Z_H 在强对流降水临近预报中起到的作用更为显著一些。

同时，为了进一步体现双偏振雷达资料在预测方面的有效性，通过更改模型的输入进行评估，构建了三个模型输入分别为 $\{Z_H, Z_{DR}\}$ ， $\{Z_H\}$ ， $\{Z_{DR}\}$ ，输出为降水量的时空因果图神经网络模型，对降水量进行估计，得到的估计结果如表 4 所示。

表 4 不同模型下降水量估计结果

模型	MAE	MSE
$\{Z_H, Z_{DR}\}$	0.7599	41.2141
$\{Z_H\}$	0.7607	41.2538
$\{Z_{DR}\}$	0.7686	41.2421

从表 4 可以看出，当输入信息为 Z_H 、 Z_{DR} 时，降水量的估计误差最小，输入信息为 Z_H 时次之，输入信息为 Z_{DR} 的效果最差。输入信息为 $\{Z_H\}$ 、 $\{Z_{DR}\}$ 相比较输入信息为 $\{Z_H, Z_{DR}\}$ 的估计量在 MAE 上分别下降了约 0.01%和 1.13%。以上结果也充分展示了双偏振雷达信息在强对流降水临近预报中的贡献，双偏振雷达信息缺一不可，少了任意一个都会造成预报性能的下降。

5.4.3 数据融合策略有效性分析

为了更好地对双偏振雷达信息进行融合，提出了一种基于元学习的数据融合策略方法，为了评估该方法在强对流临近预报中的有效性，将该元学习方法与 Model 0 $\{Z_H, Z_{DR}\}$ 、Model 1 $\{Z_H\}$ 进行比较，得到的结果如表 5 所示。

表 5 不同算法下降水量估计结果

方法	MAE	MSE
元学习	0.8675	27.5983
Model 0	0.7599	41.2141
Model 1	0.7607	41.2538

从表 5 可以看出，元学习方法在 MSE 指标上有显著下降，而在 MAE 指标上有略微上升。相比较 Model 0 与 Model 1，元学习方法在 MSE 上分别下降了约 33.04%和 33.1%。而在 MAE 上分别上升了约 12.4%和 12.3%。我们认为，由于元学习在学习过程中会把预测的数据作为训练数据进行训练，这会导致 0 值区域的误差不断累加从而导致 MAE 的上升。同时，这种累计误差可能会使降水区域的预测值不断趋近于极值，导致 MSE 显著下降。此实验证明了这种方法能更好地应对突发性和局地性强的强对流天气。

5.5 本章小结

本章研究了基于反事实推理的数据融合策略优化问题。首先，在第四章时空因果图神经网络的基础上，将模型的输入替换为反事实样本，计算模型输出的变化，从而对双偏振雷达资料在强对流降水临近预报中的贡献进行评估。为了更好地应对突发性和局地性强的强对流天气，提出了基于元学习的数据融合策略优化方法。该方法以第四章模型为基本框架，通过输入不同组合的双偏振雷达观测数据（ $\{Z_H\}$ ， $\{Z_H, Z_{DR}\}$ ）来训练模型得到基学习器。然后，以不同基学习器的降雨预报值作为输入，训练一个元学习器来输出不同模型预报值的集成结果。当输出误差达到最小时，理论上此时元学习模型代表最优融合策略。主要结论如下：**1. Z_H 和 Z_{DR} 在强对流降水临近预报中都起到了重要作用，但是相对来说， Z_H 在强对流降水临近预报中起到的作用更为显著（ $0.01606 > 0.00948$ ）；2. 基于元学习的数据融合策略方法能够提高预测效果，但该效果只体现在 MSE 显著下降，而 MAE 却略有上升。我们认为，由于元学习在学习过程中会把预测的数据作为训练数据进行训练，这回导致 0 值区域的误差不断累加从而导致 MAE 的上升。同时，这种累计误差可能会使降水区域的预测值不断趋近于极值，导致 MSE 显著下降。因此，这种方法能更好地应对突发性和局地性强的强对流天气。**

第6章 模型的改进与推广

6.1 模型的优点

(1) 从“时序因果性”与“空间关联性”两个维度入手，提出了基于结构因果模型的 Z_H 临近预报方法来解决有效应用双偏振变量改进强对流预报问题。该模型不仅考虑自身区域的相关信息，还包括其邻居区域信息，考虑信息较为全面；

(2) 将“回归到平均”分解为两个子问题。一是前期极端气象事件较少，无法预测极端气象事件。二是极端气象事件发生，对后续气象预测造成影响，导致无法准确预测到极端气象之后的数据。针对这两类子问题，本文分别基于统计中极值理论进行极端事件预测和因果推理中“干预”思想进行动态建模以缓解预报的模糊效应，可解释性较强；

(3) 利用时空因果图神经网络在求解复杂时空依赖关系的优越性，将其应用到定量降水估计中；

(4) 提出了一种基于反事实推理的数据融合策略优化方法，该方法利用反事实与元学习的优越性，能够对双偏振雷达在强对流降水临近预报中的贡献进行评估。并且元学习在学习过程中会把预测的数据作为训练数据进行训练，这回导致 0 值区域的误差不断累加从而导致 MAE 的上升。同时，这种累计误差可能会使降水区域的预测值不断趋近于极值，导致 MSE 显著下降，可以更好地应对突发性和局地性强的强对流天气。

(5) 强对流降水临近预报本质上也是一个时序预测问题，本文从时序数据普遍存在的因果关系入手，研究了基于因果推理的强对流降水临近预报问题。希望能够对相关研究人员提供一些启发。

6.2 模型的缺点

(1) 虽然从“时序因果性”与“空间关联性”两个维度入手，提出了基于结构因果模型的 Z_H 临近预报方法能有效应用双偏振变量改进强对流预报问题。但是，该方法需要从大量参数中找其相关参数（本文中不仅是 Z_H 、 Z_{DR} 、 K_{DP} 参数及马尔科夫阶数，还包括关联区域的 Z_H 、 Z_{DR} 、 K_{DP} 参数及马尔科夫阶数）。寻找这些参数需要确定典型区域的区域半径、寻找邻居区域并确定对应参数的阶数，消耗了大量的计算时间。

(2) 利用统计中极值理论对极端事件分布进行拟合，并得到极端事件发生的情况。将极值作为极端事件的预测值是一个欠考虑的问题。实际上，这是需要优化的，本文设定过于主观。为解决第二个问题，论文提出了基于干预影响动态建模的预报模糊效应缓解方法，该方法利用卡尔曼滤波更新原理，得到其随时间动态变化的状态空间模型，但该模型更新是基于线性变换的，时空问题是一个复杂的问题，本文模型建立比较简化。

(3) 时空因果图神经网络框架通过集成图神经网络和时间学习算法，能够提取复杂的时空依赖问题，也很适合求解本文中不同区域的降水量预测问题，采用不同的策略会导致算法性能不同。然而，受时间限制，本文中仅使用时序卷积模块提取 Z_H 和 Z_{DR} 之间的时间相关性，使用图卷积模块提取不同区域之间的空间相关性，没有与其他情况作为对比，所获得的预测结果并不一定是最优的。

(4) 元学习在学习过程中会把预测的数据作为训练数据进行训练，这回导致 0 值区域的误差不断累加从而导致 MAE 的上升。

(5) 本文仅针对强对流降水临近预报问题进行了比较浅显的分析，所得出的结论仅

是在仿真结果中扩展得到的，还需进一步考量；

6.3 模型的改进与推广

（1）将非线性状态空间更新引入到模型更新中，使其预测效果更加准确；

（2）强对流降水临近预报是一个具有挑战性的问题。当前，物理神经网络（PINN）发展比较迅速，实际上，PINN 在航空航天、控制、电子等诸多领域发挥了重要作用。结合物理神经网络，在大模型的基础上对强对流降水临近预报，或许是一个值得探讨的方向。

参考文献

- [1] 郑永光,张小玲,周庆亮等.强对流天气短时临近预报业务技术进展与挑战[J].气象,2010,36(07):33-42.
- [2] 周康辉. 基于深度卷积神经网络的强对流天气预报方法研究[D].中国气象科学研究院,2022.DOI:10.27631/d.cnki.gzqky.2021.000006.
- [3] Fuhan Z,Xiaodong W,Jiping G, et al. RN-Net: A Deep Learning Approach to 0–2 Hour Rainfall Nowcasting Based on Radar and Automatic Weather Station Data[J]. Sensors,2021,21(6).
- [4] Lin W C, Tsai C F. Missing value imputation: a review and analysis of the literature (2006–2017)[J]. Artificial Intelligence Review, 2020, 53: 1487-1509.
- [5] 张秀秀,王慧,田双双等.高维数据回归分析中基于 LASSO 的自变量选择[J].中国卫生统计,2013,30(06):922-926.
- [6] 赵庆媛,叶春茂,鲁耀兵.基于随机森林的微动特征重要性评估研究[J].现代防御技术,2022,50(04):124-131.
- [7] Giandomenico C,Walter V A,Maria G B, et al. Auditing fairness under unawareness through counterfactual reasoning[J]. Information Processing and Management,2023,60(2).
- [8] Kwanseok O,Seok J Y,HeungIl S. Learn-Explain-Reinforce: Counterfactual Reasoning and Its Guidance to Reinforce an Alzheimer's Disease Diagnosis Model.[J]. IEEE transactions on pattern analysis and machine intelligence,2022,PP.
- [9] Emilia G,Lucio M,Elisa C, et al. Causal inference concepts applied to three observational studies in the context of vaccine development: from theory to practice.[J]. BMC medical research methodology,2021,21(1).
- [10] Eunjoo L,Hyejin P. Estimated Time to Complete Direct Nursing Interventions Using the Nursing Interventions Classification (NIC) at Eight Hospitals in South Korea.[J]. International journal of nursing knowledge,2018,29(2).
- [11] Rashmi Y,Kumar R Y,Kumar S, et al. Framingham Risk Score and Estimated 10-Year Cardiovascular Disease Risk Reduction by a Short-Term Yoga-Based LifeStyle Intervention.[J]. Journal of alternative and complementary medicine (New York, N.Y.),2017,23(9).
- [12] Giandomenico C,Walter V A,Maria G B, et al. Auditing fairness under unawareness through counterfactual reasoning[J]. Information Processing and Management,2023,60(2).
- [13] Kwanseok O,Seok J Y,HeungIl S. Learn-Explain-Reinforce: Counterfactual Reasoning and Its Guidance to Reinforce an Alzheimer's Disease Diagnosis Model.[J]. IEEE transactions on pattern analysis and machine intelligence,2022,PP.
- [14] Emilia G,Lucio M,Elisa C, et al. Causal inference concepts applied to three observational studies in the context of vaccine development: from theory to practice.[J]. BMC medical research methodology,2021,21(1).
- [15] Sarah W,Tobias G. The role of counterfactual reasoning in responsibility judgments[J]. Proceedings of the Annual Meeting of the Cognitive Science Society,2021,43(43).
- [16] Eunjoo L,Hyejin P. Estimated Time to Complete Direct Nursing Interventions Using the Nursing Interventions Classification (NIC) at Eight Hospitals in South Korea.[J].

- International journal of nursing knowledge,2018,29(2).
- [17]Rashmi Y,Kumar R Y,Kumar S, et al. Framingham Risk Score and Estimated 10-Year Cardiovascular Disease Risk Reduction by a Short-Term Yoga-Based LifeStyle Intervention.[J]. Journal of alternative and complementary medicine (New York, N.Y.),2017,23(9).
- [18]Shu Li, Peter Bühlmann. Estimating heterogeneous treatment effects in nonstationary time series with state-space models[J]. Statistics, 2018.
- [19]Quesada, D., Valverde, G., Larrañaga, P., Bielza, C. (2021). Longterm forecasting of multivariate time series in industrial furnaces with dynamic Gaussian Bayesian networks. Engineering Applications of Artificial Intelligence, 103(April), 104301.
- [20]Wu Z , Pan S , Long G ,et al. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks[J].ACM, 2020.DOI:10.1145/3394486.3403118.
- [21]Amirreza Farnoosh, Bahar Azari, Sarah Ostadabbas. Deep Switching Auto-Regressive Factorization:Application to Time Series Forecasting[J].2020.
- [22]田杨军, 数据挖掘终篇! 一文学习模型融合! 从加权融合到 stacking, boosting, <https://blog.csdn.net/Datawhale/article/details/105355270>,2023/9/26

附录

附录 A 第一问代码

- 寻找每个数字邻居（区域邻居）

```
1. % 找到每个数字邻居
2. function neighbor = findneighbor(num, R, type)
3. % num 数字
4. % R 半径
5. % type 范围模式 (1 为方形, 2 为圆形)
6. ori = trans_1dto2d(num);
7. neighborset_2d = [];
8. for i = 1:256
9.     for j = 1:256
10.        % 方形区域
11.        if type == 1
12.            if abs(i-ori(1)) <= R && abs(j - ori(2)) <= R
13.                neighborset_2d(end+1,:) = [i,j];
14.            end
15.        elseif type == 2
16.            if norm([i-ori(1), j - ori(2)]) <= R
17.                neighborset_2d(end+1,:) = [i,j];
18.            end
19.        else
20.            error('type = 1 or 2')
21.        end
22.    end
23. End
24.
25. for i = 1:length(neighborset_2d(:,1))
26.     neighbor(i) = trans_2dto1d(neighborset_2d(i,:));
27. end
28. end
29.
30. function output = trans_1dto2d(input)
31. x = floor(input/256) + 1;
32. y = mod(input, 256);
33. output = [x,y];
34. end
35. function output = trans_2dto1d(input)
36. x = input(1);
37. y = input(2);
38. output = (x-1)*256 + y;
39. end
```

- 计算互信息

```
1. function [Ixy,lambda]=MutualInfo(X,Y)
2. %%
3. X=X';
4. Y=Y';
5. d=2;
6. nx=length(X);
7. hx=(4/(d+2))^(1/(d+4))*nx^(-1/(d+4));
8. Xall=[X;Y];
9. sum1=0;
10. for is=1:nx
11.     pxy=p_mkde([X(is),Y(is)]',Xall,hx);
12.     px=p_mkde([X(is)],X,hx);
13.     py=p_mkde([Y(is)],Y,hx);
```

```

14.     sum1=sum1+log(pxy/(px*py));
15. end
16. Ixy=sum1/nx;
17. lambda=sqrt(1-exp(-2*Ixy));
18. end
19.
20. function [pxy]=p_mkde(x,X,h)
21. s1=size(X);
22. d=s1(1);
23. N=s1(2);
24. Sxy=cov(X');
25. sum=0;
26. detS=det(Sxy);
27. for ix=1:N
28.     p2=(x-X(:,ix))'*(Sxy^(-1))*(x-X(:,ix));
29.     sum=sum+1/sqrt((2*pi)^d*detS)*exp(-p2/(2*h^2));
30. end
31. pxy=1/(N*h^d)*sum;
32. end

```

● 主函数

```

1. local = 63694;
2. for R = 1:20
3.     R
4.     % R = 17;
5.     % find neighbor
6.     for i = 1:258
7.         sum_rain(i,258) = sum(sum_rain(i,1:257));
8.     end
9.     Neighbor = findneighbor(local,R,1);
10.    length(Neighbor)
11.
12.    %动态变化, 计算地区
13.    % 邻居所在行
14.    Y_various = sum_rain(Neighbor,1:257);
15.    for i = 1:length(Neighbor)
16.        if sum(sum_rain(local,1:257)) == 0 || sum(Y_various(i,:)) == 0
17.            Mi(i) = 0;
18.            pearson_corr(i) = 0;
19.            COR(i) = 0;
20.        else
21.
22.            % 互信息、皮尔逊相关系数、相关系数
23.            Mi(i) = MutualInfo(sum_rain(local,1:257)', Y_various(i,:));
24.            pearson_corr(i) = corr(sum_rain(local,1:257)', Y_various(i,:));
25.        end
26.    end
27.    M(R) = mean(Mi);
28.    Pear(R) = mean(pearson_corr);
29. end
30.
31. M = M';
32. Pear = Pear';

```

● Z_H预测

```

1. import numpy as np
2. import os
3. import xgboost as xgb
4. from sklearn.kernel_ridge import KernelRidge
5. from sklearn.neural_network import MLPRegressor
6. from sklearn import tree
7. import pandas as pd
8. from sklearn.linear_model import LinearRegression
9. from sklearn.linear_model import LogisticRegression

```

```

10. from sklearn.metrics import mean_squared_error
11. from sklearn.metrics import mean_absolute_error
12. from sklearn_extensions.kernel_regression import KernelRegression
13. from sklearn.ensemble import RandomForestRegressor
14. #特征选取
15. total_path=["D:\\BaiduNetdiskDownload\\NJU_CPOL_update2308\\NJU_CPOL_update2308\\ZDR\\", "E:\\NJU_CPOL_update2308\\KDP\\", "E:\\NJU_CPOL_update2308\\dBZ\\"]
16. info_list=[7,8,9,7,8,9,7,8,9]
17. max_length=9
18. m=np.zeros([256,256])#要预测的区域
19. n=np.zeros([256,256])
20. denggao_list=['1.0km', '3.0km', '7.0km']
21.
22. #####以下另外执行
23. #特征的合并
24. # denggao_list=['1km', '3km', '7km']
25. # feature_all=[]
26. # tezheng_path_list=["F:\\华为杯 2023\\code\\第一问\\ZDR\\", "F:\\华为杯 2023\\code\\第一问\\KDP\\", "F:\\华为杯 2023\\code\\第一问\\dBZ\\"]
27. # for k in range(258):
28. #     min=1000
29. #     min_array=[]
30. #     for i in range(len(tezheng_path_list)): #长度
31. #         for j in range(len(denggao_list)):
32. #             file_path = tezheng_path_list[i] + denggao_list[j] + "\\"
33. #             files = os.listdir(file_path)
34. #             input_path = file_path + files[k]
35. #             data = np.load(input_path)
36. #             if data.shape[0]<min:
37. #                 min=data.shape[0]
38. #
39. # #数组拼接
40. # #这个里面横向拼接,所有的横向拼接完了再纵向拼接
41. # heng_merge=[]
42. # for i in range(len(tezheng_path_list)): #长度
43. #     for j in range(len(denggao_list)):
44. #         file_path = tezheng_path_list[i] + denggao_list[j] + "\\"
45. #         files = os.listdir(file_path)
46. #         input_path = file_path + files[k]
47. #         data = np.load(input_path)
48. #         if i==0 and j==0:
49. #             heng_merge=data[:min,: ]
50. #         else:
51. #             heng_merge=np.concatenate((heng_merge, data[:min,: ]), axis=1)
52. #
53. # if k==0:
54. #     feature_all=heng_merge
55. # else:
56. #     feature_all=np.concatenate((feature_all, heng_merge), axis=0)
57. #
58. #print(feature_all)
59. #feature_all = feature_all[[not np.all(feature_all[i] == 0) for i in range(feature_all.shape[0])], :]
60. #print(feature_all)
61. # last_three=feature_all[:,feature_all.shape[1]-info_list[8]-info_list[7]-1]
62. # last_two=feature_all[:,feature_all.shape[1]-info_list[8]-1]
63. # last_one=feature_all[:,feature_all.shape[1]-1]
64. # update_feature_1=feature_all[:,0:feature_all.shape[1]-info_list[8]-info_list[7]-1]
65. # update_feature_2=feature_all[:,feature_all.shape[1]-info_list[8]-info_list[7]:feature_all.shape[1]-info_list[8]-1]
66. # update_feature_3=feature_all[:,feature_all.shape[1]-info_list[8]:feature_all.shape[1]-1]
67. # update_feature=np.concatenate([update_feature_1,update_feature_2,update_feature_3],axis=1)
68. # pd.DataFrame(update_feature).to_csv("F:\\华为杯 2023\\data\\feature.csv")
69. # pd.DataFrame(last_one).to_csv("F:\\华为杯 2023\\data\\target1.csv")
70. # pd.DataFrame(last_two).to_csv("F:\\华为杯 2023\\data\\target2.csv")
71. # pd.DataFrame(last_three).to_csv("F:\\华为杯 2023\\data\\target3.csv")

```

```

72. ###结构因果模型
73. # model = LinearRegression()
74. # train_rate=0.8
75. # train_size=int(update_feature.shape[0]*0.8)
76. # model_1km=model.fit(update_feature[0:train_size,:], last_three[0:train_size])
77. #
78. # result_1km=model_1km.predict(update_feature[train_size,:])
79. # model_3km=model.fit(update_feature[0:train_size,:], last_two[0:train_size])
80. # result_3km=model_3km.predict(update_feature[train_size,:])
81. # model_7km=model.fit(update_feature[0:train_size,:], last_one[0:train_size])
82. # result_7km=model_7km.predict(update_feature[train_size,:])

```

● 数据预处理

```

1. import numpy as np
2. import os
3. import xgboost as xgb
4. from sklearn.kernel_ridge import KernelRidge
5. from sklearn.neural_network import MLPRegressor
6. from sklearn import tree
7. import pandas as pd
8. from sklearn.linear_model import LinearRegression
9. from sklearn.linear_model import LogisticRegression
10. from sklearn.metrics import mean_squared_error
11. from sklearn.metrics import mean_absolute_error
12. from sklearn_extensions.kernel_regression import KernelRegression
13. from sklearn.ensemble import RandomForestRegressor
14. #特征选取
15. total_path=["D:\\BaiduNetdiskDownload\\NJU_CPOL_update2308\\NJU_CPOL_update2308\\ZDR\\", "E:\\NJU_CPOL_update2308\\KDP\\", "E:\\NJU_CPOL_update2308\\dBZ\\"]
16. info_list=[7,8,9,7,8,9,7,8,9]
17. max_length=9
18. m=np.zeros([256,256]) #要预测的区域
19. n=np.zeros([256,256])
20. denggao_list=['1.0km', '3.0km', '7.0km']
21.
22. for q in range(len(m)):
23.     features = []
24.     targets = []
25.     info_count = 0
26.     for i in range(len(total_path)):
27.         for j in range(len(denggao_list)):
28.             file_path = total_path[i] + "\\\" + denggao_list[j] + "\\\"
29.             child_files = []
30.             for root, folders, files in os.walk(file_path):
31.                 file_count = 0
32.                 if file_count == 0:
33.                     child_files = folders
34.                     break;
35.                 file_count = file_count + 1
36.             # 所有降水
37.             jiangshui_number = 0
38.             for k in range(len(child_files)):
39.                 child_path = file_path + child_files[k] + "\\\"
40.                 files = os.listdir(child_path)
41.                 len_rain = len(files) # 此次降雨持续时间
42.                 eve_rain = np.zeros(len_rain) # 添加每个区域的信息
43.                 file_count = 0
44.                 for file in files: # 每次降水中的信息
45.                     input_path = child_path + file
46.                     data = np.load(input_path)
47.                     if info_count == 0 or info_count == 1 or info_count == 2:
48.                         eve_rain[file_count] = (data[m[q]][n[q]] + 1) / 6
49.                     elif info_count == 3 or info_count == 4 or info_count == 5:
50.                         eve_rain[file_count] = (data[m[q]][n[q]] + 1) / 7
51.                     else:
52.                         eve_rain[file_count] = data[m[q]][n[q]] / 65
53.

```

```

54.         file_count = file_count + 1
55.         if info_count == 6 or info_count == 7 or info_count == 8:
56.             eve_features = np.zeros((len_rain - info_list[info_count],
info_list[info_count] + 1))
57.         else:
58.             eve_features = np.zeros((len_rain - info_list[info_count],
info_list[info_count]))
59.         if info_count == 6 or info_count == 7 or info_count == 8:
60.             for a in range(len_rain - max_length):
61.                 for b in range(info_list[info_count] + 1):
62.                     eve_features[a][b] = eve_rain[a + b]
63.         else:
64.             for a in range(len_rain - max_length):
65.                 for b in range(info_list[info_count]):
66.                     eve_features[a][b] = eve_rain[a + b]
67.
68.         if info_count == 0:
69.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\ZDR\\\\1km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
70.
71.         elif info_count == 1:
72.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\ZDR\\\\3km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
73.
74.         elif info_count == 2:
75.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\ZDR\\\\7km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
76.
77.         elif info_count == 3:
78.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\KDP\\\\1km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
79.
80.         elif info_count == 4:
81.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\KDP\\\\3km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
82.
83.         elif info_count == 5:
84.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\KDP\\\\7km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
85.
86.         elif info_count == 6:
87.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\dBZ\\\\1km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
88.
89.         elif info_count == 7:
90.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\dBZ\\\\3km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
91.
92.         elif info_count == 8:
93.             out_path = "F:\\\\                华                为                杯
2023\\\\code\\\\"+str(q+1)+"\\\\dBZ\\\\7km\\\\" + str(jiangshui_number) + ".np
np.save(out_path, eve_features)
94.
95.
96.         jiangshui_number = jiangshui_number + 1
97.
98.         info_count = info_count + 1

```

● 因果贝叶斯网络构建

1. """Perform network inference using multivariate transfer entropy.
- 2.
3. Estimate multivariate transfer entropy (TE) for network inference using a
4. greedy approach with maximum statistics to generate a non-uniform embedding
5. (Faes, 2011; Lizier, 2012). 迭代贪婪算法, 马尔科夫的阶数不是固定的
- 6.
7. Note:
8. Written for Python 3.4+
9. """

```

10. from .network_inference import NetworkInferenceTE, NetworkInferenceMultivariate
11. from .stats import network_fdr
12. from .results import ResultsNetworkInference
13.
14.
15. class MultivariateTE(NetworkInferenceTE, NetworkInferenceMultivariate):
16.     """Perform network inference using multivariate transfer entropy.
17.
18.     Perform network inference using multivariate transfer entropy (TE). To
19.     perform network inference call analyse_network() on the whole network or a
20.     set of nodes or call analyse_single_target() to estimate TE for a single
21.     target. See docstrings of the two functions for more information.
22.
23.     References:
24.
25.     - Schreiber, T. (2000). Measuring Information Transfer. Phys Rev Lett,
26.       85(2), 461–464. http://doi.org/10.1103/PhysRevLett.85.461
27.     - Vicente, R., Wibral, M., Lindner, M., & Pipa, G. (2011). Transfer
28.       entropy—a model-free measure of effective connectivity for the
29.       neurosciences. J Comp Neurosci, 30(1), 45–67.
30.       http://doi.org/10.1007/s10827-010-0262-3
31.     - Lizier, J. T., & Rubinov, M. (2012). Multivariate construction of
32.       effective computational networks from observational data. Max Planck
33.       Institute: Preprint. Retrieved from
34.       http://www.mis.mpg.de/preprints/2012/preprint2012\_25.pdf
35.     - Faes, L., Nollo, G., & Porta, A. (2011). Information-based detection
36.       of nonlinear Granger causality in multivariate processes via a
37.       nonuniform embedding technique. Phys Rev E, 83, 1–15.
38.       http://doi.org/10.1103/PhysRevE.83.051112
39.
40.     Attributes:
41.
42.         source_set : list
43.             indices of source processes tested for their influence on the
44.             target
45.         target : list
46.             index of target process
47.         settings : dict
48.             analysis settings
49.         current_value : tuple
50.             index of the current value in TE estimation, (idx process,
51.             idx sample)
52.         selected_vars_full : list of tuples
53.             samples in the full conditional set, (idx process, idx sample)
54.         selected_vars_sources : list of tuples
55.             source samples in the conditional set, (idx process, idx sample)
56.         selected_vars_target : list of tuples
57.             target samples in the conditional set, (idx process, idx sample)
58.         pvalue_omnibus : float
59.             p-value of the omnibus test
60.         pvalues_sign_sources : numpy array
61.             array of p-values for TE from individual sources to the target
62.         statistic_omnibus : float
63.             joint TE from all sources to the target
64.         statistic_sign_sources : numpy array
65.             raw TE values from individual sources to the target
66.         sign_omnibus : bool
67.             statistical significance of the over-all TE
68.     """
69.
70.     def __init__(self):
71.         super().__init__()
72.
73.     def analyse_network(self, settings, data, targets='all', sources='all'):
74.         """Find multivariate transfer entropy between all nodes in the network.
75.
76.         Estimate multivariate transfer entropy (TE) between all nodes in the
77.         network or between selected sources and targets.
78.

```

```

79. Note:
80.     For a detailed description of the algorithm and settings see
81.     documentation of the analyse_single_target() method and references
82.     in the class docstring.
83.
84. Example:
85.     >>> data = Data()
86.     >>> data.generate_mute_data(100, 5)
87.     >>> settings = {
88.     >>>     'cmi_estimator': 'JidtKraskovCMI',
89.     >>>     'n_perm_max_stat': 200,
90.     >>>     'n_perm_min_stat': 200,
91.     >>>     'n_perm_omnibus': 500,
92.     >>>     'n_perm_max_seq': 500,
93.     >>>     'max_lag_sources': 5,
94.     >>>     'min_lag_sources': 2
95.     >>> }
96.     >>> network_analysis = MultivariateTE()
97.     >>> results = network_analysis.analyse_network(settings, data)
98.
99. Args:
100.     settings : dict
101.         parameters for estimation and statistical testing, see
102.         documentation of analyse_single_target() for details, settings
103.         can further contain
104.
105.         - verbose : bool [optional] - toggle console output
106.           (default=True)
107.         - fdr_correction : bool [optional] - correct results on the
108.           network level, see documentation of stats.network_fdr() for
109.           details (default=True)
110.
111.     data : Data instance
112.         raw data for analysis
113.     targets : list of int | 'all' [optional]
114.         index of target processes (default='all')
115.     sources : list of int | list of list | 'all' [optional]
116.         indices of source processes for each target (default='all');
117.         if 'all', all network nodes excluding the target node are
118.         considered as potential sources and tested;
119.         if list of int, the source specified by each int is tested as
120.         a potential source for the target with the same index or a
121.         single target;
122.         if list of list, sources specified in each inner list are
123.         tested for the target with the same index
124.
125. Returns:
126.     ResultsNetworkInference instance
127.         results of network inference, see documentation of
128.         ResultsNetworkInference()
129.
130. """
131. # Set defaults for network inference.
132. settings.setdefault('verbose', True)
133. settings.setdefault('fdr_correction', True)
134.
135. # Check which targets and sources are requested for analysis.
136. if targets == 'all':
137.     targets = [t for t in range(data.n_processes)]
138. if sources == 'all':
139.     sources = ['all' for t in targets]
140. if (type(sources) is list) and (type(sources[0]) is int):
141.     sources = [sources for t in targets]
142. if (type(sources) is list) and (type(sources[0]) is list):
143.     pass
144. else:
145.     ValueError('Sources was not specified correctly: {0}.'.format(
146.         sources))
147. assert(len(sources) == len(targets)), ('List of targets and list of '
148.                                         'sources have to have the same

```

```

148.                                     'same length')
149.
150. # Perform TE estimation for each target individually
151. results = ResultsNetworkInference(n_nodes=data.n_processes,
152.                                   n_realisations=data.n_realisations(),
153.                                   normalised=data.normalise)
154.
155. for t in range(len(targets)):
156.     if settings['verbose']:
157.         print('\n##### analysing target with index {0} from list {1}
158.               .format(t, targets))
159.         res_single = self.analyse_single_target(
160.             settings, data, targets[t], sources[t])
161.         results.combine_results(res_single)
162.
163. # Get no. realisations actually used for estimation from single target
164. # analysis.
165. results.data_properties.n_realisations = (
166.     res_single.data_properties.n_realisations)
167.
168. # Perform FDR-correction on the network level. Add FDR-corrected
169. # results as an extra field. Network_fdr/combine_results internally
170. # creates a deep copy of the results.
171. if settings['fdr_correction']:
172.     results = network_fdr(settings, results)
173. return results
174.
175. def analyse_single_target(self, settings, data, target, sources='all'):
176.     """Find multivariate transfer entropy between sources and a target.
177.
178.     Find multivariate transfer entropy (TE) between all source processes
179.     and the target process. Uses multivariate, non-uniform embedding found
180.     through information maximisation. Multivariate TE is calculated in four
181.     steps:
182.
183.     (1) find all relevant variables in the target processes' own past, by
184.         iteratively adding candidate variables that have significant
185.         conditional mutual information (CMI) with the current value
186.         (conditional on all variables that were added previously)
187.     (2) find all relevant variables in the source processes' pasts (again
188.         by finding all candidates with significant CMI)
189.     (3) prune the final conditional set by testing the CMI between each
190.         variable in the final set and the current value, conditional on all
191.         other variables in the final set
192.     (4) statistics on the final set of sources (test for over-all transfer
193.         between the final conditional set and the current value, and for
194.         significant transfer of all individual variables in the set)
195.
196.     Note:
197.         For a further description of the algorithm see references in the
198.         class docstring.
199.
200.     Example:
201.
202.     >>> data = Data()
203.     >>> data.generate_mute_data(100, 5)
204.     >>> settings = {
205.     >>>     'cmi_estimator': 'JidtkraskovCMI',
206.     >>>     'n_perm_max_stat': 200,
207.     >>>     'n_perm_min_stat': 200,
208.     >>>     'n_perm_omnibus': 500,
209.     >>>     'n_perm_max_seq': 500,
210.     >>>     'max_lag_sources': 5,
211.     >>>     'min_lag_sources': 2

```

```

212.         >>> target = 0
213.         >>> sources = [1, 2, 3]
214.         >>> network_analysis = MultivariateTE()
215.         >>> results = network_analysis.analyse_single_target(settings,
216.         >>>                                     data, target,
217.         >>>                                     sources)
218.
219.     Args:
220.         settings : dict
221.             parameters for estimation and statistical testing:
222.
223.             - cmi_estimator : str - estimator to be used for CMI
224.               calculation (for estimator settings see the documentation in
225.               the estimators_* modules)
226.             - max_lag_sources : int - maximum temporal search depth for
227.               candidates in the sources' past in samples
228.             - min_lag_sources : int - minimum temporal search depth for
229.               candidates in the sources' past in samples
230.             - max_lag_target : int [optional] - maximum temporal search
231.               depth for candidates in the target's past in samples
232.               (default=same as max_lag_sources)
233.             - tau_sources : int [optional] - spacing between candidates in
234.               the sources' past in samples (default=1)
235.             - tau_target : int [optional] - spacing between candidates in
236.               the target's past in samples (default=1)
237.             - n_perm_* : int [optional] - number of permutations, where *
238.               can be 'max_stat', 'min_stat', 'omnibus', and 'max_seq'
239.               (default=500)
240.             - alpha_* : float [optional] - critical alpha level for
241.               statistical significance, where * can be 'max_stats',
242.               'min_stats', 'omnibus', and 'max_seq' (default=0.05)
243.             - add_conditionals : list of tuples | str [optional] - force
244.               the estimator to add these conditionals when estimating TE;
245.               can either be a list of variables, where each variable is
246.               described as (idx process, lag wrt to current value) or can
247.               be a string: 'faes' for Faes-Method (see references)
248.             - permute_in_time : bool [optional] - force surrogate
249.               creation by shuffling realisations in time instead of
250.               shuffling replications; see documentation of
251.               Data.permute_samples() for further settings (default=False)
252.             - verbose : bool [optional] - toggle console output
253.               (default=True)
254.
255.         data : Data instance
256.             raw data for analysis
257.         target : int
258.             index of target process
259.         sources : list of int | int | 'all' [optional]
260.             single index or list of indices of source processes
261.             (default='all'), if 'all', all network nodes excluding the
262.             target node are considered as potential sources
263.
264.     Returns:
265.         ResultsNetworkInference instance
266.             results of network inference, see documentation of
267.             ResultsNetworkInference()
268.
269.     """
270.     # Check input and clean up object if it was used before.
271.     self._initialise(settings, data, sources, target)
272.
273.     # Main algorithm.
274.     print('\n----- (1) include target candidates')
275.     self._include_target_candidates(data)
276.     print('\n----- (2) include source candidates')
277.     self._include_source_candidates(data)
278.     print('\n----- (3) prune source candidate')
279.     self._prune_candidates(data)
280.     print('\n----- (4) final statistics')
281.     self._test_final_conditional(data)

```

```

281.
282. # Clean up and return results.
283. if self.settings['verbose']:
284.     print('final source samples: {0}'.format(
285.         self._idx_to_lag(self.selected_vars_sources)))
286.     print('final target samples: {0}\n\n'.format(
287.         self._idx_to_lag(self.selected_vars_target)))
288. results = ResultsNetworkInference(
289.     n_nodes=data.n_processes,
290.     n_realisations=data.n_realisations(self.current_value),
291.     normalised=data.normalise)
292. results._add_single_result(
293.     target=self.target,
294.     settings=self.settings,
295.     results={
296.         'sources_tested': self.source_set,
297.         'current_value': self.current_value,
298.         'selected_vars_target': self._idx_to_lag(
299.             self.selected_vars_target),
300.         'selected_vars_sources': self._idx_to_lag(
301.             self.selected_vars_sources),
302.         'selected_sources_pval': self.pvalues_sign_sources,
303.         'selected_sources_te': self.statistic_sign_sources,
304.         'omnibus_te': self.statistic_omnibus,
305.         'omnibus_pval': self.pvalue_omnibus,
306.         'omnibus_sign': self.sign_omnibus,
307.         'te': self.statistic_single_link
308.     })
309. self._reset() # remove attributes
310. return results
311.

```

附录 B 区域的最大互信息数均值与皮尔逊相关系数均值

- 区域: [35,175]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.34449537571613	0.928890966432660
2	1.09544383215743	0.866034295893229
3	0.992501087887492	0.836735657057104
4	0.898422290547738	0.803240359371226
5	0.823804565199356	0.772494801171036
6	0.770408221686518	0.752329594697698
7	0.731499129777881	0.738059958935346
8	0.697156415441559	0.723961113575211
9	0.666700369712517	0.710111838483618
10	0.638535765963032	0.695217069010175
11	0.614476293978038	0.682829218902006
12	0.591122953091101	0.669981185145773
13	0.568744840086309	0.657239096013809
14	0.548031109023511	0.644849842595836
15	0.529889819338091	0.634087633204081
16	0.512880076196521	0.622746802552644
17	0.496756325695362	0.611858965823695
18	0.481212563256325	0.601056325599532
19	0.466556325632566	0.590725636956632
20	0.453325698433569	0.581456986563256

- 区域: [38,210]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.49308001604399	0.963064324366822
2	1.25514191869736	0.934725836368048
3	1.12052355783563	0.912381106791694
4	1.02649111721299	0.890406627429643
5	0.949587425073533	0.868383736616112
6	0.884803205032976	0.847488231643686
7	0.829389009956546	0.826554012220321
8	0.786585785281840	0.809666074554134
9	0.750525020329225	0.795252764045928
10	0.719924469896044	0.782496088153086
11	0.691710257606005	0.769423615782748
12	0.664866451554650	0.756210068663649
13	0.639465130612602	0.742480099199197

14	0.615410023687087	0.728375438880959
15	0.592890541812958	0.714660349392188
16	0.571508829129536	0.700860259159805
17	0.552662521069154	0.688368410986195
18	0.533905001094405	0.674484189988629
19	0.515250380041605	0.659587356430610
20	0.497156597412636	0.644255701342580

● 区域: [85,179]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.41867015264913	0.947808995918855
2	1.23310197701983	0.927824221623548
3	1.11714832851465	0.911090878769232
4	1.03410964748312	0.896834521444576
5	0.965431939248100	0.882843957207802
6	0.908954840285983	0.868776137636064
7	0.863789406710294	0.856809097493768
8	0.829300942076563	0.847708922939627
9	0.798849915172108	0.839014044288678
10	0.771520442658555	0.830556947831470
11	0.745068935253789	0.822345588311053
12	0.721421887019355	0.814456074892142
13	0.698503862094952	0.806082221039483
14	0.676956615213426	0.797605210714440
15	0.657958842687205	0.790687338535985
16	0.640665326380954	0.784401061863562
17	0.624292543257496	0.778086884345193
18	0.609215161351706	0.772456287401890
19	0.595495366492503	0.767290320376359
20	0.582648308300868	0.762360932453394

● 区域: [128,175]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.59539124426159	0.968782058416364
2	1.36711755469435	0.940075585236359
3	1.13847198410085	0.902895685998331
4	0.981611660379782	0.866732559023146
5	0.871516541621381	0.847399087287060
6	0.785085010481939	0.827947453862746
7	0.725581962148129	0.813806356512308
8	0.676863412534457	0.800292492394138

9	0.634740939830689	0.786880655439110
10	0.597072996991738	0.771905749785743
11	0.562129696763973	0.753978870229760
12	0.529271827702391	0.735557963216964
13	0.496946566281891	0.713253519745031
14	0.465506220892075	0.687816041391449
15	0.435727432573351	0.661333476211845
16	0.407431478317626	0.633723632889686
17	0.381011824644854	0.605153667757162
18	0.357925767419097	0.579583845196737
19	0.337551234618410	0.556457529733660
20	0.319670139034225	0.535755041120200

● 区域: [130,126]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.56031729104377	0.946809269149351
2	1.21849337889427	0.904158028691816
3	1.02478727508776	0.863850613110097
4	0.922038704814787	0.852530679346119
5	0.847057598131448	0.841675814843585
6	0.784289975109490	0.828346297188176
7	0.720031211825442	0.807741609184611
8	0.666348675870691	0.787201734008095
9	0.620767009560160	0.767221344578162
10	0.583264156721095	0.748731286908387
11	0.550363787768921	0.731026569018645
12	0.522286592145219	0.713785137168417
13	0.494498360571322	0.694908839353925
14	0.467999196182965	0.675829443951682
15	0.443616357326699	0.657480834987996
16	0.418718471733240	0.636222512557969
17	0.395249081969049	0.615224517566383
18	0.373674660544624	0.594608118041597
19	0.354591533084310	0.575656437334828
20	0.337576848843985	0.558090895310406

● 区域: [180,150]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.10778516073572	0.883758378687997
2	0.840931318043275	0.810687098019898
3	0.710706613452281	0.768905918447476

4	0.638273917441552	0.746559258597394
5	0.596332772634265	0.734508144108572
6	0.562032315048503	0.722239194254990
7	0.530825231775854	0.705048941847571
8	0.499899362144147	0.682944349142404
9	0.471804271337739	0.660375040187073
10	0.448176768345903	0.639568131555909
11	0.427864709323486	0.621711159166354
12	0.411868670385199	0.608854160715258
13	0.399055037084622	0.599122450918370
14	0.388228847202168	0.590725785810947
15	0.377658737801170	0.580865419598941
16	0.366933769685283	0.569528732116140
17	0.355834700952306	0.557086231001814
18	0.344946425353435	0.544272923965211
19	0.335700044071001	0.534481592809269
20	0.326882382807620	0.525408215606278

● 区域: [191,155]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.28950914945814	0.845968076314826
2	0.782972352496199	0.658798046497778
3	0.565616626341243	0.556004581015129
4	0.458957128549639	0.499555692918192
5	0.396195059069585	0.460189002943631
6	0.359741135142963	0.436727308871583
7	0.330288189802845	0.415908858538088
8	0.307517895054824	0.398911495952296
9	0.288480220486832	0.382847177082600
10	0.273151138231665	0.368280859724140
11	0.260452278100682	0.355463411851277
12	0.248557143735882	0.343060582113018
13	0.237683562969031	0.331386874674090
14	0.227136677504205	0.320202094424541
15	0.218141534564618	0.310947896860404
16	0.210237569110602	0.302643419024664
17	0.203070474248850	0.294839483253404
18	0.196834092403613	0.287405552737459
19	0.191224441764208	0.280012110749741
20	0.185865563993668	0.272121534870376

● 区域: [225,170]

邻居半径/R	最大互信息系数均值	皮尔逊相关系数均值
1	1.18127223526845	0.919555279267686
2	0.989790693801335	0.883866521848354
3	0.854657229924500	0.839018010926913
4	0.767833770830423	0.801966048970359
5	0.706089019642102	0.773831160855524
6	0.665404499249535	0.755941908588474
7	0.639565298278682	0.744685044110823
8	0.618597234280294	0.736654331800658
9	0.600333468911446	0.728782686486563
10	0.582717195268734	0.720435624672844
11	0.566142012965508	0.711663945183210
12	0.549512991079415	0.701457407265997
13	0.532550363128359	0.689613190234201
14	0.515626731852912	0.676471698888266
15	0.498430511264026	0.661837893168505
16	0.482379291040389	0.648425290028061
17	0.467396188302583	0.635931968154496
18	0.453374420257827	0.624340355658198
1	1.18127223526845	0.919555279267686
2	0.989790693801335	0.883866521848354

附录 C 第二问代码

● 极值理论

```
1. import matplotlib.pyplot as plt
2. import pandas as pd
3. import datetime
4. import math
5. import numpy as np
6. import os
7. from typing import List
8. import matplotlib.pyplot as plt
9. import numpy as np
10. import pandas as pd
11. import ads_evt as spot
12. from sklearn.metrics import mean_squared_error
13. from sklearn.metrics import mean_absolute_error
14. denggao_list=['1km', '3km', '7km']
15. feature_all=[]
16. tezheng_path_list=["F:\\华为杯 2023\\data\\sum_inter\\8\\dBZ\\"]
17. # mse1=[]
18. # mae1=[]
19. # mse2=[]
20. # mae2=[]
21. mae_value_1km = []
22. mse_value_1km = []
23. mae_value_3km = []
24. mse_value_3km = []
25. mae_value_7km = []
26. mse_value_7km = []
27. for k in range(50):
28.     min=1000
29.     min_array=[]
30.     for i in range(len(tezheng_path_list)): #长度
31.         for j in range(len(denggao_list)):
32.             file_path = tezheng_path_list[i] + denggao_list[j] + "\\
33.             files = os.listdir(file_path)
34.             input_path = file_path + files[k]
35.             data = np.load(input_path)
36.             # print(sum(data))
37.             if sum(data)>0:
38.                 init_data = data.shape[0] - 2
39.                 proba = 1e-1
40.                 depth = 5
41.                 models: List[spot.SPOTBase] = [
42.                     # spot.SPOT(q=proba),
43.                     # spot.dSPOT(q=proba, depth=depth),
44.                     # spot.biSPOT(q=proba),
45.                     # The original implementation of bidSPOT uses n_points=8 fo
46.                 r _grimshaw by default
47.                     spot.bidSPOT(q=proba, depth=depth, n_points=8),
48.                 ]
49.                 thresholds = 0
50.                 for alg in models:
51.                     alg.fit(init_data=init_data, data=data)
52.                     alg.initialize()
53.                     results = alg.run()
54.                     thresholds = results['upper_thresholds']
55.                     thresholds = results['upper_thresholds'][0]
```

● 融入干预影响的卡尔曼滤波

```
1. import numpy as np
2. import os
3. import math
4. from filterpy.kalman import KalmanFilter
```

```

5. from filterpy.common import Q_discrete_white_noise
6. import xgboost as xgb
7. from sklearn.kernel_ridge import KernelRidge
8. from sklearn.neural_network import MLPRegressor
9. import pandas as pd
10. from sklearn.linear_model import LinearRegression
11.
12. from sklearn.metrics import mean_squared_error
13. from sklearn.metrics import mean_absolute_error
14. from sklearn_extensions.kernel_regression import KernelRegression
15. #特征选取
16. total_path=["D:\\BaiduNetdiskDownload\\NJU_CPOL_update2308\\NJU_CPOL_update2308
    \\ZDR\\", "E:\\NJU_CPOL_update2308\\KDP\\", "E:\\NJU_CPOL_update2308\\dBZ\\"]
17. info_list=[7,8,9,7,8,9,7,8,9]
18. max_length=9
19. m=np.zeros([256,256]) #要预测的区域的行与列
20. n=np.zeros([256,256]) #要预测的区域的行与列
21. denggao_list=['1.0km', '3.0km', '7.0km']
22. features=[]
23. targets=[]
24. info_count=0
25. #####以下另外执行
26. #特征的合并
27. denggao_list=['1km', '3km', '7km']
28. feature_all=[]
29. tezheng_path_list=["F:\\华为杯 2023\\code\\第一问\\ZDR\\", "F:\\华为杯 2023\\code\\第
    一问\\KDP\\", "F:\\华为杯 2023\\code\\第一问\\dBZ\\"]
30. for k in range(258):
31.     min=1000
32.     min_array=[]
33.     for i in range(len(tezheng_path_list)): #长度
34.         for j in range(len(denggao_list)):
35.             file_path = tezheng_path_list[i] + denggao_list[j] + "\\
36.             files = os.listdir(file_path)
37.             input_path = file_path + files[k]
38.             data = np.load(input_path)
39.             if data.shape[0]<min:
40.                 min=data.shape[0]
41. #数组拼接
42. #这个里面横向拼接,所有的横向拼接完了再纵向拼接
43. heng_merge=[]
44. for i in range(len(tezheng_path_list)): #长度
45.     for j in range(len(denggao_list)):
46.         file_path = tezheng_path_list[i] + denggao_list[j] + "\\
47.         files = os.listdir(file_path)
48.         input_path = file_path + files[k]
49.         data = np.load(input_path)
50.         if i==0 and j==0:
51.             heng_merge=data[:min,:]
52.         else:
53.             heng_merge=np.concatenate((heng_merge, data[:min,:]), axis=1)
54.     if k==0:
55.         feature_all=heng_merge
56.     else:
57.         feature_all=np.concatenate((feature_all, heng_merge), axis=0)
58.
59. #print(feature_all)
60. #feature_all = feature_all[[not np.all(feature_all[i] == 0) for i in range(featur
    e_all.shape[0])], :]
61. #print(feature_all)
62. last_three=feature_all[:, feature_all.shape[1]-info_list[8]-info_list[7]-1]
63. last_two=feature_all[:, feature_all.shape[1]-info_list[8]-1]
64. last_one=feature_all[:, feature_all.shape[1]-1]
65. update_feature_1=feature_all[:, 0:feature_all.shape[1]-info_list[8]-info_list[7]
    -1]
66. update_feature_2=feature_all[:, feature_all.shape[1]-info_list[8]-info_list[7]:f
    eature_all.shape[1]-info_list[8]-1]
67. update_feature_3=feature_all[:, feature_all.shape[1]-info_list[8]:feature_all.sh
    ape[1]-1]

```

```

68. update_feature=np.concatenate([update_feature_1,update_feature_2,update_feature
   _3],axis=1)
69. pd.DataFrame(update_feature).to_csv("F:\\华为杯 2023\\data\\feature.csv")
70. pd.DataFrame(last_one).to_csv("F:\\华为杯 2023\\data\\target1.csv")
71. pd.DataFrame(last_two).to_csv("F:\\华为杯 2023\\data\\target2.csv")
72. pd.DataFrame(last_three).to_csv("F:\\华为杯 2023\\data\\target3.csv")
73.
74. ###结构因果模型
75. model = LinearRegression()
76. train_rate=0.8
77. train_size=int(update_feature.shape[0]*0.8)
78. model_1km=model.fit(update_feature[0:train_size:], last_three[0:train_size])
79. c = model_1km.coef_
80. b = model_1km.intercept_
81. my_filter = KalmanFilter(dim_x=len(c) + 2, dim_z=1)
82. my_filter.Q = Q_discrete_white_noise(dim=2, dt=0.1, var=1e-2)
83. x = []
84. x.append(b)
85. for m in range(len(c)):
86.     x.append(c[m])
87. my_filter.F = np.identity(len(c) + 2, dtype=float)
88. my_filter.P = np.zeros((len(c) + 2, len(c) + 2))
89. my_filter.R = 0
90. x.append(0)
91. my_filter.x = x
92. my_filter.R = 0
93.
94. # print(my_filter.F)
95. result_sum=[]
96. #融入干预影响的卡尔曼滤波
97. for k in range(train_size,update_feature.shape[0]):
98.     flag=0
99.     predict_result=0
100.    print(update_feature[k,:])
101.    result = model_1km.predict(update_feature[k,:])
102.    predict_result=result[0]
103.    my_filter = KalmanFilter(dim_x=len(c) + 2, dim_z=1)
104.    x = []
105.    x.append(b)
106.    for m in range(len(c)):
107.        x.append(c[m])
108.    my_filter.F = np.identity(len(c) + 2, dtype=float)
109.    my_filter.P = np.zeros((len(c) + 2, len(c) + 2))
110.    my_filter.R = 0
111.    causal_effect = math.fabs(
112.        result[0] - last_three[k]) / last_three[k]
113.    causal_effect = 1 / (1 + math.exp(-causal_effect)) * 0.01
114.    # causal_effect=0.00001
115.    x.append(causal_effect)
116.
117.    my_filter.x = x
118.
119.    H = []
120.    H.append(1)
121.    remain_H = update_feature[k,:]
122.    # new_data.iloc[train_length + k:train_length + k + 1, 1:len(new_da
   ta)].iloc[0].values
123.    for m in range(len(remain_H)):
124.        H.append(remain_H[m])
125.    H.append(1)
126.    my_filter.H = np.array([H])
127.    my_filter.predict()
128.    predict_value = my_filter.H.dot(my_filter.x)[0]
129.    # data_pre.append(predict_value)
130.    z = last_three[k] # 真实值
131.    my_filter.update(z)
132.    result_sum.append(predict_value)

```

附录 D 第三问代码

● 降水数据预处理

```
1. import numpy as np
2. import os
3. import xgboost as xgb
4. from sklearn.kernel_ridge import KernelRidge
5. from sklearn.neural_network import MLPRegressor
6. from sklearn import tree
7. import pandas as pd
8. from sklearn.linear_model import LinearRegression
9. from sklearn.linear_model import LogisticRegression
10. from sklearn.metrics import mean_squared_error
11. from sklearn.metrics import mean_absolute_error
12. from sklearn.extensions.kernel_regression import KernelRegression
13. from sklearn.ensemble import RandomForestRegressor
14. #特征选取
15. total_path=["D:\\BaiduNetdiskDownload\\NJU_CPOL_update2308\\NJU_CPOL_update2308\\ZDR\\", "E:\\NJU_CPOL_update2308\\dBZ\\"]
16. info_list=[7,8,9,7,8,9,7,8,9]
17. max_length=9
18. m=np.zeros([256,256])#要预测的区域的行与列
19. n=np.zeros([256,256]) #要预测的区域的行与列
20. denggao_list=['1.0km', '3.0km', '7.0km']
21.
22. #降水数据数据预处理
23. # total_path=['F:\\华为杯 2023\\NJU_CPOL_kdpRain\\NJU_CPOL_kdpRain\\']
24. # info_list=[7,8,9,7,8,9,7,8,9]
25. # max_length=9
26. # m=[35,38,85,128,130,180,191,225] #要预测的区域的行与列
27. # n=[175,210,179,125,126,150,155,170] #要预测的区域的行与列
28. # denggao_list=['1.0km', '3.0km', '7.0km']
29. #
30. # for q in range(len(m)):
31. #     features = []
32. #     targets = []
33. #     info_count = 0
34. #     for i in range(len(total_path)):
35. #         file_path = total_path[i] + "\\
36. #         child_files = []
37. #         for root, folders, files in os.walk(file_path):
38. #             file_count = 0
39. #             if file_count == 0:
40. #                 child_files = folders
41. #                 break;
42. #             file_count = file_count + 1
43. #         # 所有降水
44. #         jiangshui_number = 0
45. #         #len(child_files)
46. #         for k in range(len(child_files)):
47. #             child_path = file_path + child_files[k] + "\\
48. #             files = os.listdir(child_path)
49. #             len_rain = len(files) # 此次降雨持续时间
50. #             eve_rain = np.zeros(len_rain) # 添加每个区域的信息
51. #             file_count = 0
52. #             for file in files: # 每次降水中的信息
53. #                 input_path = child_path + file
54. #                 data = np.load(input_path)
55. #                 eve_rain[file_count] = data[m[q]][n[q]]
56. #             file_count = file_count + 1
57. #         out_path = "F:\\
58. #         2023\\data\\sum_inter\\" + str(q + 1) + "\\rain\\" + str(
59. #             jiangshui_number) + ".npy"
60. #         np.save(out_path, eve_rain)
61. #
```

```

62. #             jiangshui_number = jiangshui_number + 1
63. #             info_count = info_count + 1
64. #             print(info_count)

```

● 时空图神经网络模型

```

1. import torch
2. import torch.nn as nn
3.
4. from model import layers
5.
6. class STGCNChebGraphConv(nn.Module):
7.     # STGCNChebGraphConv contains 'TGTND TGTND TNFF' structure
8.     # ChebGraphConv is the graph convolution from ChebyNet.
9.     # Using the Chebyshev polynomials of the first kind as a graph filter.
10.
11.     # T: Gated Temporal Convolution Layer (GLU or GTU)
12.     # G: Graph Convolution Layer (ChebGraphConv)
13.     # T: Gated Temporal Convolution Layer (GLU or GTU)
14.     # N: Layer Normalization
15.     # D: Dropout
16.
17.     # T: Gated Temporal Convolution Layer (GLU or GTU)
18.     # G: Graph Convolution Layer (ChebGraphConv)
19.     # T: Gated Temporal Convolution Layer (GLU or GTU)
20.     # N: Layer Normalization
21.     # D: Dropout
22.
23.     # T: Gated Temporal Convolution Layer (GLU or GTU)
24.     # N: Layer Normalization
25.     # F: Fully-Connected Layer
26.     # F: Fully-Connected Layer
27.
28.     def __init__(self, args, blocks, n_vertex):
29.         super(STGCNChebGraphConv, self).__init__()
30.         modules = []
31.         for l in range(len(blocks) - 3):
32.             modules.append(layers.STConvBlock(args.Kt, args.Ks, n_vertex, block
s[l][-1], blocks[l+1], args.act_func, args.graph_conv_type, args.gso, args.enab
le_bias, args.droprate))
33.         self.st_blocks = nn.Sequential(*modules)
34.         ko = args.n_his - (len(blocks) - 3) * 2 * (args.Kt - 1)
35.         self.ko = ko
36.         if self.ko > 1:
37.             self.output = layers.OutputBlock(ko, blocks[-3][-1], blocks[-2], b
locks[-1][0], n_vertex, args.act_func, args.enable_bias, args.droprate)
38.         elif self.ko == 0:
39.             self.fc1 = nn.Linear(in_features=blocks[-3][-1], out_features=block
s[-2][0], bias=args.enable_bias)
40.             self.fc2 = nn.Linear(in_features=blocks[-2][0], out_features=blocks
[-1][0], bias=args.enable_bias)
41.             self.relu = nn.ReLU()
42.             self.leaky_relu = nn.LeakyReLU()
43.             self.silu = nn.SiLU()
44.             self.dropout = nn.Dropout(p=args.droprate)
45.
46.         def forward(self, x):
47.             x = self.st_blocks(x)
48.             if self.ko > 1:
49.                 x = self.output(x)
50.             elif self.ko == 0:
51.                 x = self.fc1(x.permute(0, 2, 3, 1))
52.                 x = self.relu(x)
53.                 x = self.fc2(x).permute(0, 3, 1, 2)
54.
55.             return x
56.
57. class STGCNGraphConv(nn.Module):

```

```

58.     # STGCNGraphConv contains 'TGTND TGTND TNFF' structure
59.     # GraphConv is the graph convolution from GCN.
60.     # GraphConv is not the first-order ChebConv, because the renormalization tr
    ick is adopted.
61.     # Be careful about over-smoothing.
62.
63.     # T: Gated Temporal Convolution Layer (GLU or GTU)
64.     # G: Graph Convolution Layer (GraphConv)
65.     # T: Gated Temporal Convolution Layer (GLU or GTU)
66.     # N: Layer Normalization
67.     # D: Dropout
68.
69.     # T: Gated Temporal Convolution Layer (GLU or GTU)
70.     # G: Graph Convolution Layer (GraphConv)
71.     # T: Gated Temporal Convolution Layer (GLU or GTU)
72.     # N: Layer Normalization
73.     # D: Dropout
74.
75.     # T: Gated Temporal Convolution Layer (GLU or GTU)
76.     # N: Layer Normalization
77.     # F: Fully-Connected Layer
78.     # F: Fully-Connected Layer
79.
80.     def __init__(self, args, blocks, n_vertex):
81.         super(STGCNGraphConv, self).__init__()
82.         modules = []
83.         for l in range(len(blocks) - 3):
84.             modules.append(layers.STConvBlock(args.Kt, args.Ks, n_vertex, block
s[l][-1], blocks[l+1], args.act_func, args.graph_conv_type, args.gso, args.enab
le_bias, args.droprate))
85.         self.st_blocks = nn.Sequential(*modules)
86.         Ko = args.n_his - (len(blocks) - 3) * 2 * (args.Kt - 1)
87.         self.Ko = Ko
88.         if self.Ko > 1:
89.             self.output = layers.OutputBlock(Ko, blocks[-3][-1], blocks[-2], b
locks[-1][0], n_vertex, args.act_func, args.enable_bias, args.droprate)
90.         elif self.Ko == 0:
91.             self.fc1 = nn.Linear(in_features=blocks[-3][-1], out_features=block
s[-2][0], bias=args.enable_bias)
92.             self.fc2 = nn.Linear(in_features=blocks[-2][0], out_features=blocks
[-1][0], bias=args.enable_bias)
93.             self.relu = nn.ReLU()
94.             self.leaky_relu = nn.LeakyReLU()
95.             self.silu = nn.SiLU()
96.             self.do = nn.Dropout(p=args.droprate)
97.
98.         def forward(self, x):
99.             x = self.st_blocks(x)
100.            if self.Ko > 1:
101.                x = self.output(x)
102.            elif self.Ko == 0:
103.                x = self.fc1(x.permute(0, 2, 3, 1))
104.                x = self.relu(x)
105.                x = self.fc2(x).permute(0, 3, 1, 2)
106.
107.            return x

```

- 因果卷积层时序卷积层

```

1. import math
2. import torch
3. import torch.nn as nn
4. import torch.nn.functional as F
5. import torch.nn.init as init
6.
7. class Align(nn.Module):
8.     def __init__(self, c_in, c_out):
9.         super(Align, self).__init__()

```

```

10.         self.c_in = c_in
11.         self.c_out = c_out
12.         self.align_conv = nn.Conv2d(in_channels=c_in, out_channels=c_out, kernel_size=(1, 1))
13.
14.     def forward(self, x):
15.         if self.c_in > self.c_out:
16.             x = self.align_conv(x)
17.         elif self.c_in < self.c_out:
18.             batch_size, _, timestep, n_vertex = x.shape
19.             x = torch.cat([x, torch.zeros([batch_size, self.c_out - self.c_in, timestep, n_vertex]).to(x)], dim=1)
20.         else:
21.             x = x
22.
23.         return x
24.
25. class CausalConv1d(nn.Conv1d):
26.     def __init__(self, in_channels, out_channels, kernel_size, stride=1, enable_padding=False, dilation=1, groups=1, bias=True):
27.         if enable_padding == True:
28.             self.__padding = (kernel_size - 1) * dilation
29.         else:
30.             self.__padding = 0
31.         super(CausalConv1d, self).__init__(in_channels, out_channels, kernel_size, stride=stride, padding=self.__padding, dilation=dilation, groups=groups, bias=bias)
32.
33.     def forward(self, input):
34.         result = super(CausalConv1d, self).forward(input)
35.         if self.__padding != 0:
36.             return result[:, :, :, -self.__padding:]
37.
38.         return result
39.
40. class CausalConv2d(nn.Conv2d):
41.     def __init__(self, in_channels, out_channels, kernel_size, stride=1, enable_padding=False, dilation=1, groups=1, bias=True):
42.         kernel_size = nn.modules.utils._pair(kernel_size)
43.         stride = nn.modules.utils._pair(stride)
44.         dilation = nn.modules.utils._pair(dilation)
45.         if enable_padding == True:
46.             self.__padding = [int((kernel_size[i] - 1) * dilation[i]) for i in range(len(kernel_size))]
47.         else:
48.             self.__padding = 0
49.             self.left_padding = nn.modules.utils._pair(self.__padding)
50.         super(CausalConv2d, self).__init__(in_channels, out_channels, kernel_size, stride=stride, padding=0, dilation=dilation, groups=groups, bias=bias)
51.
52.     def forward(self, input):
53.         if self.__padding != 0:
54.             input = F.pad(input, (self.left_padding[1], 0, self.left_padding[0], 0))
55.         result = super(CausalConv2d, self).forward(input)
56.
57.         return result
58.
59. class TemporalConvLayer(nn.Module):
60.
61.     # Temporal Convolution Layer (GLU)
62.     #
63.     # |-----| * residual connection *
64.     #
65.     # |-----| casualconv2d -----+ -----|
66.     # -----|-----| -----| ⊙ ----->
67.     # |-----| casualconv2d --- sigmoid ---|
68.     #

```

```

69.
70.     #param x: tensor, [bs, c_in, ts, n_vertex]
71.
72.     def __init__(self, kt, c_in, c_out, n_vertex, act_func):
73.         super(TemporalConvLayer, self).__init__()
74.         self.kt = kt
75.         self.c_in = c_in
76.         self.c_out = c_out
77.         self.n_vertex = n_vertex
78.         self.align = Align(c_in, c_out)
79.         if act_func == 'glu' or act_func == 'gtu':
80.             self.causal_conv = CausalConv2d(in_channels=c_in, out_channels=2 *
c_out, kernel_size=(kt, 1), enable_padding=False, dilation=1)
81.         else:
82.             self.causal_conv = CausalConv2d(in_channels=c_in, out_channels=c_ou
t, kernel_size=(kt, 1), enable_padding=False, dilation=1)
83.         self.act_func = act_func
84.         self.sigmoid = nn.Sigmoid()
85.         self.tanh = nn.Tanh()
86.         self.relu = nn.ReLU()
87.         self.leaky_relu = nn.LeakyReLU()
88.         self.silu = nn.SiLU()
89.
90.     def forward(self, x):
91.         x_in = self.align(x)[: , : , self.kt - 1 : , :]
92.         x_causal_conv = self.causal_conv(x)
93.
94.         if self.act_func == 'glu' or self.act_func == 'gtu':
95.             x_p = x_causal_conv[: , : self.c_out , : , :]
96.             x_q = x_causal_conv[: , -self.c_out : , : , :]
97.
98.             if self.act_func == 'glu':
99.                 # Explanation of Gated Linear Units (GLU):
100.                 # The concept of GLU was first introduced in the paper
101.                 # "Language Modeling with Gated Convolutional Networks".
102.                 # URL: https://arxiv.org/abs/1612.08083
103.                 # In the GLU operation, the input tensor x is divided into two
tensors, x_a and x_b,
104.                 # along a specific dimension.
105.                 # In PyTorch, GLU is computed as the element-wise multiplicati
on of x_a and sigmoid(x_b).
106.                 # More information can be found here: https://pytorch.org/docs
/master/nn.functional.html#torch.nn.functional.glu
107.                 # The provided code snippet, (x_p + x_in) ⊙ sigmoid(x_q), is
an example of GLU operation.
108.                 x = torch.mul((x_p + x_in), self.sigmoid(x_q))
109.
110.             else:
111.                 # tanh(x_p + x_in) ⊙ sigmoid(x_q)
112.                 x = torch.mul(self.tanh(x_p + x_in), self.sigmoid(x_q))
113.
114.         elif self.act_func == 'relu':
115.             x = self.relu(x_causal_conv + x_in)
116.
117.         elif self.act_func == 'leaky_relu':
118.             x = self.leaky_relu(x_causal_conv + x_in)
119.
120.         elif self.act_func == 'silu':
121.             x = self.silu(x_causal_conv + x_in)
122.
123.         else:
124.             raise NotImplementedError(f'ERROR: The activation function {self.a
ct_func} is not implemented.')
125.
126.         return x
127.
128. class ChebGraphConv(nn.Module):
129.     def __init__(self, c_in, c_out, ks, gso, bias):
130.         super(ChebGraphConv, self).__init__()

```

```

131.         self.c_in = c_in
132.         self.c_out = c_out
133.         self.Ks = Ks
134.         self.gso = gso
135.         self.weight = nn.Parameter(torch.FloatTensor(Ks, c_in, c_out))
136.         if bias:
137.             self.bias = nn.Parameter(torch.FloatTensor(c_out))
138.         else:
139.             self.register_parameter('bias', None)
140.         self.reset_parameters()
141.
142.     def reset_parameters(self):
143.         init.kaiming_uniform_(self.weight, a=math.sqrt(5))
144.         if self.bias is not None:
145.             fan_in, _ = init._calculate_fan_in_and_fan_out(self.weight)
146.             bound = 1 / math.sqrt(fan_in) if fan_in > 0 else 0
147.             init.uniform_(self.bias, -bound, bound)
148.
149.     def forward(self, x):
150.         #bs, c_in, ts, n_vertex = x.shape
151.         x = torch.permute(x, (0, 2, 3, 1))
152.
153.         if self.Ks - 1 < 0:
154.             raise ValueError(f'ERROR: the graph convolution kernel size Ks has
to be a positive integer, but received {self.Ks}.')
155.         elif self.Ks - 1 == 0:
156.             x_0 = x
157.             x_list = [x_0]
158.         elif self.Ks - 1 == 1:
159.             x_0 = x
160.             x_1 = torch.einsum('hi,btij->bthj', self.gso, x)
161.             x_list = [x_0, x_1]
162.         elif self.Ks - 1 >= 2:
163.             x_0 = x
164.             x_1 = torch.einsum('hi,btij->bthj', self.gso, x)
165.             x_list = [x_0, x_1]
166.             for k in range(2, self.Ks):
167.                 x_list.append(torch.einsum('hi,btij->bthj', 2 * self.gso, x_li
st[k - 1]) - x_list[k - 2])
168.
169.         x = torch.stack(x_list, dim=2)
170.
171.         cheb_graph_conv = torch.einsum('btkhi,kij->bthj', x, self.weight)
172.
173.         if self.bias is not None:
174.             cheb_graph_conv = torch.add(cheb_graph_conv, self.bias)
175.         else:
176.             cheb_graph_conv = cheb_graph_conv
177.
178.         return cheb_graph_conv
179.
180. class GraphConv(nn.Module):
181.     def __init__(self, c_in, c_out, gso, bias):
182.         super(GraphConv, self).__init__()
183.         self.c_in = c_in
184.         self.c_out = c_out
185.         self.gso = gso
186.         self.weight = nn.Parameter(torch.FloatTensor(c_in, c_out))
187.         if bias:
188.             self.bias = nn.Parameter(torch.FloatTensor(c_out))
189.         else:
190.             self.register_parameter('bias', None)
191.         self.reset_parameters()
192.
193.     def reset_parameters(self):
194.         init.kaiming_uniform_(self.weight, a=math.sqrt(5))
195.         if self.bias is not None:
196.             fan_in, _ = init._calculate_fan_in_and_fan_out(self.weight)
197.             bound = 1 / math.sqrt(fan_in) if fan_in > 0 else 0

```

```

198.         init.uniform_(self.bias, -bound, bound)
199.
200.     def forward(self, x):
201.         #bs, c_in, ts, n_vertex = x.shape
202.         x = torch.permute(x, (0, 2, 3, 1))
203.
204.         first_mul = torch.einsum('hi,btij->bthj', self.gso, x)
205.         second_mul = torch.einsum('bthi,ij->bthj', first_mul, self.weight)
206.
207.         if self.bias is not None:
208.             graph_conv = torch.add(second_mul, self.bias)
209.         else:
210.             graph_conv = second_mul
211.
212.         return graph_conv
213.
214. class GraphConvLayer(nn.Module):
215.     def __init__(self, graph_conv_type, c_in, c_out, ks, gso, bias):
216.         super(GraphConvLayer, self).__init__()
217.         self.graph_conv_type = graph_conv_type
218.         self.c_in = c_in
219.         self.c_out = c_out
220.         self.align = Align(c_in, c_out)
221.         self.ks = ks
222.         self.gso = gso
223.         if self.graph_conv_type == 'cheb_graph_conv':
224.             self.cheb_graph_conv = ChebGraphConv(c_out, c_out, ks, gso, bias)
225.         elif self.graph_conv_type == 'graph_conv':
226.             self.graph_conv = GraphConv(c_out, c_out, gso, bias)
227.
228.     def forward(self, x):
229.         x_gc_in = self.align(x)
230.         if self.graph_conv_type == 'cheb_graph_conv':
231.             x_gc = self.cheb_graph_conv(x_gc_in)
232.         elif self.graph_conv_type == 'graph_conv':
233.             x_gc = self.graph_conv(x_gc_in)
234.         x_gc = x_gc.permute(0, 3, 1, 2)
235.         x_gc_out = torch.add(x_gc, x_gc_in)
236.
237.         return x_gc_out
238.
239. class STConvBlock(nn.Module):
240.     # STConv Block contains 'TGTND' structure
241.     # T: Gated Temporal Convolution Layer (GLU or GTU)
242.     # G: Graph Convolution Layer (ChebGraphConv or GraphConv)
243.     # T: Gated Temporal Convolution Layer (GLU or GTU)
244.     # N: Layer Normolization
245.     # D: Dropout
246.
247.     def __init__(self, kt, ks, n_vertex, last_block_channel, channels, act_func, graph_conv_type, gso, bias, droprate):
248.         super(STConvBlock, self).__init__()
249.         self.tmp_conv1 = TemporalConvLayer(kt, last_block_channel, channels[0], n_vertex, act_func)
250.         self.graph_conv = GraphConvLayer(graph_conv_type, channels[0], channels[1], ks, gso, bias)
251.         self.tmp_conv2 = TemporalConvLayer(kt, channels[1], channels[2], n_vertex, act_func)
252.         self.tc2_ln = nn.LayerNorm([n_vertex, channels[2]])
253.         self.relu = nn.ReLU()
254.         self.dropout = nn.Dropout(p=droprate)
255.
256.     def forward(self, x):
257.         x = self.tmp_conv1(x)
258.         x = self.graph_conv(x)
259.         x = self.relu(x)
260.         x = self.tmp_conv2(x)
261.         x = self.tc2_ln(x.permute(0, 2, 3, 1)).permute(0, 3, 1, 2)
262.         x = self.dropout(x)

```

```

263.
264.         return x
265.
266. class OutputBlock(nn.Module):
267.     # Output block contains 'TNFF' structure
268.     # T: Gated Temporal Convolution Layer (GLU or GTU)
269.     # N: Layer Normalization
270.     # F: Fully-Connected Layer
271.     # F: Fully-Connected Layer
272.
273.     def __init__(self, ko, last_block_channel, channels, end_channel, n_vertex
, act_func, bias, droprate):
274.         super(OutputBlock, self).__init__()
275.         self.tmp_conv1 = TemporalConvLayer(ko, last_block_channel, channels[0]
, n_vertex, act_func)
276.         self.fc1 = nn.Linear(in_features=channels[0], out_features=channels[1]
, bias=bias)
277.         self.fc2 = nn.Linear(in_features=channels[1], out_features=end_channel
, bias=bias)
278.         self.tc1_ln = nn.LayerNorm([n_vertex, channels[0]])
279.         self.relu = nn.ReLU()
280.         self.leaky_relu = nn.LeakyReLU()
281.         self.silu = nn.SiLU()
282.         self.dropout = nn.Dropout(p=droprate)
283.
284.     def forward(self, x):
285.         x = self.tmp_conv1(x)
286.         x = self.tc1_ln(x.permute(0, 2, 3, 1))
287.         x = self.fc1(x)
288.         x = self.relu(x)
289.         x = self.fc2(x).permute(0, 3, 1, 2)
290.
291.         return x

```

附录 E 第四问代码

● 元学习代码

```
1. import numpy as np
2. import os
3. import xgboost as xgb
4. import torch
5. from sklearn.kernel_ridge import KernelRidge
6. from sklearn.neural_network import MLPRegressor
7. from sklearn import tree
8. import pandas as pd
9. from sklearn.linear_model import LinearRegression
10. from sklearn.linear_model import LogisticRegression
11. from sklearn.metrics import mean_squared_error
12. from sklearn.metrics import mean_absolute_error
13. from sklearn_extensions.kernel_regression import KernelRegression
14. from sklearn.ensemble import RandomForestRegressor
15. from sklearn.model_selection import KFold
16. data=pd.read_csv("C:\\Users\\chensiya\\Downloads\\vel.csv")
17.
18. train_rate=0.8
19. train_size=int(len(data)*0.8)
20.
21. #模型一
22. model = torch.load("model1.pt") #加载训练好的时空因果图网络模型
23. model1=model.fit(data.iloc[0:train_size,0:6],data.iloc[0:train_size,6])
24. model = torch.load("model2.pt") #加载训练好的时空因果图神经网络模型
25. model2=model.fit(data.iloc[0:train_size,3:6],data.iloc[0:train_size,6])
26. result_train_1=model1.predict(data.iloc[0:train_size,0:6])
27. result_test_1=model1.predict(data.iloc[train_size:,0:6])
28. result_train_2=model2.predict(data.iloc[0:train_size,0:3])
29. result_test_2=model2.predict(data.iloc[train_size:,0:3])
30. new_label_1=np.concatenate([result_train_1,result_test_1],axis=0)
31. new_label_2=np.concatenate([result_train_2,result_test_2],axis=0)
32. # kf = KFold(n_splits=5)      K 折交叉验证
33. # for train_index, test_index in kf.split(X):
34. #     X_train, X_test = X[train_index], X[test_index]
35. #     y_train, y_test = y[train_index], y[test_index]
36. model=GNN() #调用 GNN 进行训练
37. model.fit(data.iloc[0:train_size,0:6],new_label_1[0:train_size])
38. predict_value=model.predict(data.iloc[train_size:,0:6])
39. print(len(predict_value))
40. mae=mean_absolute_error(result_test_1[5992:],data.iloc[train_size:,6].values[5992:])
41. mse=mean_squared_error(result_test_1[5992:],data.iloc[train_size:,6].values[5992:])
42. print(mae)
43. print(mse)
```